

中国大恒（集团）有限公司北京图像视觉技术分公司

Python 接口开发说明书

版本：V1.0.17

发布日期：2026-01-12

 HENG
IM GING | 大恒图像

本手册中所提及的其它软硬件产品的商标与名称，都属于相应公司所有。

本手册的版权属于中国大恒（集团）有限公司北京图像视觉技术分公司所有。未得到本公司的正式许可，任何组织或个人均不得以任何手段和形式对本手册内容进行复制或传播。

本手册的内容若有任何修改，恕不另行通知。

© 2026 中国大恒（集团）有限公司北京图像视觉技术分公司版权所有

网 站：www.daheng-imaging.com

公 司 总 机：010-82828878

客户服务热线：400-999-7595

销 售 信 箱：sales@daheng-imaging.com

支 持 信 箱：support@daheng-imaging.com

目录

1. 相机工作流程	1
1.1. 整体工作流程	1
1.2. 功能控制流程	2
1.3. 整体代码样例	3
2. 编程指引	5
2.1. 搭建编程环境	5
2.1.1. Linux	5
2.1.2. Windows	6
2.2. 快速上手	6
2.2.1. 引入库	6
2.2.2. 枚举设备	7
2.2.3. 打开关闭设备	8
2.2.4. 采集控制	9
2.2.5. 图像处理	15
2.2.6. 相机控制	18
2.2.7. 掉线重连	23
2.2.8. 导入导出相机配置参数	26
2.2.9. 属性设置 UI	27
2.2.10. 存图存视频	29
2.2.11. 错误处理	30
3. 附录	32
3.1. 功能类定义	32
3.1.1. Feature_s(推荐)	32
3.1.2. Feature(不再维护)	36
3.2. 数据类型定义	44
3.2.1. GxLogTypeList	44
3.2.2. GxDeviceClassList	45
3.2.3. GxAccessStatus	45
3.2.4. GxAccessMode	45
3.2.5. GxPixelFormatEntry	45
3.2.6. GxFrameStatusList	48

3.2.7. GxDeviceTemperatureSelectorEntry	48
3.2.8. GxPixelSizeEntry	48
3.2.9. GxPixelColorFilterEntry	48
3.2.10. GxAcquisitionModeEntry	49
3.2.11. GxTriggerSourceEntry	49
3.2.12. GxTriggerActivationEntry.....	49
3.2.13. GxExposureModeEntry	49
3.2.14. GxUserOutputSelectorEntry	49
3.2.15. GxUserOutputModeEntry	50
3.2.16. GxGainSelectorEntry.....	50
3.2.17. GxBlackLevelSelectEntry	50
3.2.18. GxBalanceRatioSelectorEntry.....	50
3.2.19. GxAALightEnvironmentEntry.....	50
3.2.20. GxUserSetEntry.....	50
3.2.21. GxAWBLampHouseEntry	51
3.2.22. GxUserDataFieldSelectorEntry	51
3.2.23. GxTestPatternEntry	51
3.2.24. GxTriggerSelectorEntry	51
3.2.25. GxLineSelectorEntry.....	51
3.2.26. GxDeviceSerialPortBaudRateEntry	52
3.2.27. GxSerialPortStopBitsEntry	52
3.2.28. GxLineModeEntry	52
3.2.29. GxLineSourceEntry	53
3.2.30. GxEventSelectorEntry	53
3.2.31. GxLutSelectorEntry	54
3.2.32. GxTransferControlModeEntry.....	54
3.2.33. GxTransferOperationModeEntry	54
3.2.34. GxTestPatternGeneratorSelectorEntry.....	54
3.2.35. GxChunkSelectorEntry	54
3.2.36. GxBinningHorizontalModeEntry	54
3.2.37. GxBinningVerticalModeEntry.....	54
3.2.38. GxSensorShutterModeEntry	54
3.2.39. GxAcquisitionStatusSelectorEntry	55
3.2.40. GxExposureTimeModeEntry	55
3.2.41. GxGammaModeEntry.....	55
3.2.42. GxLightSourcePresetEntry.....	55
3.2.43. GxColorTransformationModeEntry.....	55
3.2.44. GxColorTransformationValueSelectorEntry	55
3.2.45. GxAutoEntry	56
3.2.46. GxSwitchEntry	56
3.2.47. GxSensorBitDepthEntry	56
3.2.48. GxMultisourceSelectorEntry.....	56
3.2.49. GxDeviceTapGeometryEntry.....	56
3.2.50. GxEncoderSelectorEntry.....	56

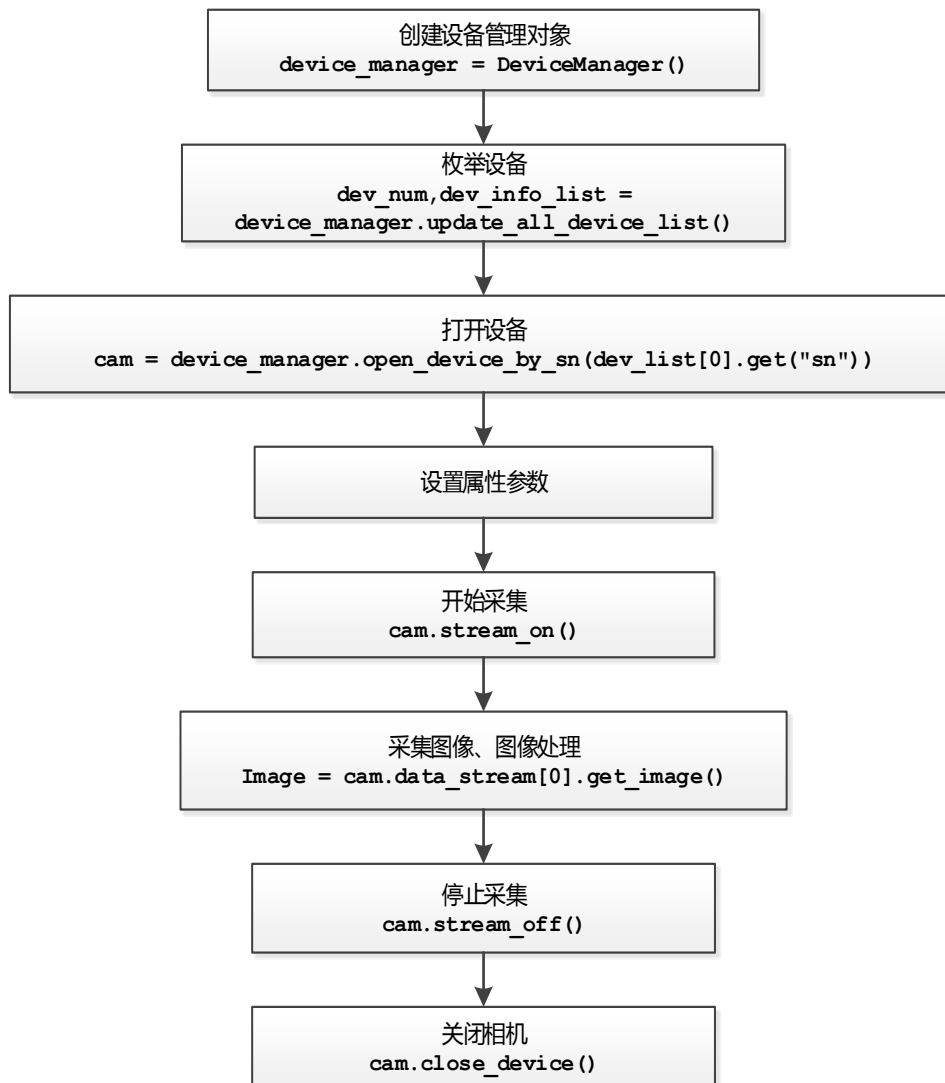
3.2.51. GxEncoderSourceAEntry	56
3.2.52. GxEncoderSourceBEntry	57
3.2.53. GxEncoderModeEntry	57
3.2.54. GxEncoderDirectionEntry	57
3.2.55. GxRegionSendModeEntry.....	57
3.2.56. GxShadingCorrectionModeEntry.....	57
3.2.57. GxFFCCGenerateStatusEntry	57
3.2.58. GxFFCCCoefficientEntry.....	58
3.2.59. GxDSNUSelectorEntry	58
3.2.60. GxDSNUGenerateStatusEntry	59
3.2.61. GxPRNUSelectorEntry	59
3.2.62. GxPRNUGenerateStatusEntry	59
3.2.63. GxCXPLinkConfigurationEntry	59
3.2.64. GxCXPLinkConfigurationPreferredEntry	60
3.2.65. GxCXPLinkConfigurationStatusEntry.....	60
3.2.66. GxCXPConectionSelectorEntry.....	60
3.2.67. GxCXPConectionTestModeEntry	60
3.2.68. GxSequencerFratureSelectorEntry	60
3.2.69. GxSequencerTriggerSourceEntry	61
3.2.70. GxRegionSelectorEntry	61
3.2.71. GxTimerSelectorEntry	61
3.2.72. GxTimerTriggerSourceEntry.....	61
3.2.73. GxNoiseReductionModeEntry	61
3.2.74. GxHDMRModeEntry.....	61
3.2.75. GxMGCMModeEntry	62
3.2.76. GxAcquisitionBurstModeEntry.....	62
3.2.77. GxSensorSelectorEntry	62
3.2.78. GxIMUConfigAccRangeEntry	62
3.2.79. GxIMUConfigAccOdrEntry	62
3.2.80. GxIMUConfigAccOdrLowPassFilterFrequencyEntry	63
3.2.81. GxIMUConfigGyroRangeEntry	63
3.2.82. GxIMUConfigGyroOdrEntry.....	63
3.2.83. GxIMUConfigGyroOdrLowPassFilterFrequencyEntry.....	63
3.2.84. GxIMUTemperatureOdrEntry.....	64
3.2.85. GxSerialportSelectorEntry.....	65
3.2.86. GxSerialportSourceEntry.....	65
3.2.87. GxSerialportBaundrateEntry	65
3.2.88. GxSerialporeStopBitsEntry.....	65
3.2.89. GxSerialportParityEntry	65
3.2.90. GxCounterSelectorEntry.....	65
3.2.91. GxCounterEventSourceEntry	66
3.2.92. GxCounterResetSourceEntry	66
3.2.93. GxCounterResetActivationEntry.....	66
3.2.94. GxCounterTriggerSourceEntry	66

3.2.95. GxTimerTriggerActivationEntry	67
3.2.96. GxStopAcquisitionModeEntry	67
3.2.97. GxDSSStreamBufferHandlingModeEntry	67
3.2.98. GxResetDeviceModeEntry	67
3.2.99. Dx BayerConvertType	67
3.2.100. DxValidBit	67
3.2.101. DxActualBits	68
3.2.102. DxImageMirrorMode	68
3.2.103. DxRGBChannelOrder	68
3.2.104. GxTLCClassList	68
3.2.105. GxImageInfo	68
3.2.106. GxIPCConfigureModeList	68
3.2.107. GxActionCommandResult	69
3.2.108. GxRegisterStackEntry	69
3.2.109. ColorTransformFactor	69
3.2.110. FlatFieldCorrectionParameter	69
3.2.111. GxWindowsID	70
3.2.112. GxWindowsShowMode	70
3.2.113. GxCFAMethod	70
3.2.114. GxImageFormatType	70
3.2.115. GxVideoFormatType	70
3.2.116. GxSaveImageInfo	71
3.2.117. GxRecordParam	71
3.3. 模块接口定义	71
3.3.1. DeviceManager	71
3.3.2. Device	80
3.3.3. Interface	88
3.3.4. DataStream	89
3.3.5. FeatureControl	93
3.3.6. RGBImage(不再维护)	99
3.3.7. RawImage	101
3.3.8. Buffer	108
3.3.9. ImageProcessConfig	109
3.3.10. Utility	114
3.3.11. ImageProcess	116
3.3.12. ImageFormatConvert	118
3.3.13. FlatFieldCorrection	122
3.3.14. Decompressor	123
3.3.15. MediaProc	124
3.3.16. VideoSaver	124
3.4. 属性参数 (不再维护, 推荐使用 Feature_s)	125
3.4.1. 设备属性参数	125

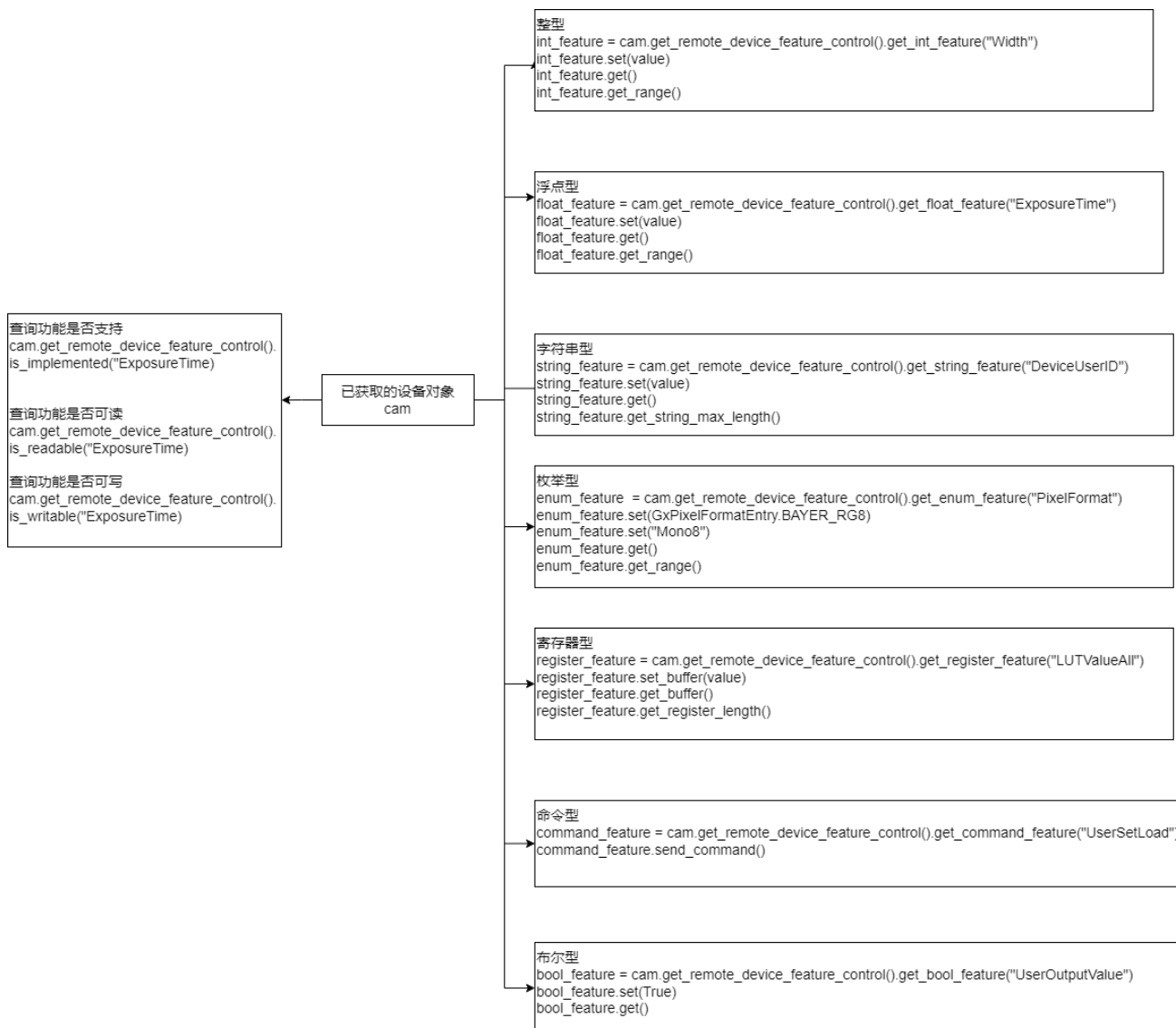
3.4.2. 流属性参数	134
4. 常见问题解答	135
5. 版本说明	136

1. 相机工作流程

1.1. 整体工作流程



1.2. 功能控制流程



1.3. 整体代码样例

```
# 用户可自定义调用前缀，样例中使用了 gx
import gxipy as gx

#枚举设备。dev_info_list 是设备信息列表，列表的元素个数为枚举到的设备个数，列
#表元素是字典，其中包含设备索引 (index)、ip 信息 (ip) 等设备信息
device_manager = gx.DeviceManager()
dev_num, dev_info_list = device_manager.update_all_device_list()
if dev_num == 0:
    sys.exit(1)

# 打开设备
# 获取设备基本信息列表
strSN = dev_info_list[0].get("sn")
#通过序列号打开设备
cam = device_manager.open_device_by_sn(strSN)

# 开始采集
cam.stream_on()

#获取流通道个数
#如果 int_channel_num == 1，设备只有一个流通道，列表 data_stream 元素个数
#为 1
#如果 int_channel_num>1，设备有多个流通道，列表 data_stream 元素个数大于 1
#目前千兆网相机、USB3.0、USB2.0 相机均不支持多流通道。
int_channel_num = cam.get_stream_channel_num()

#获取数据
#num 为采集图片次数
num = 1
for i in range(num):

    #从第 0 个流通道获取一幅图像
    raw_image = cam.data_stream[0].get_image()

    #从彩色原始图像获取 RGB 图像
    #创建格式转换器
    image_convert = device_manager.create_image_format_convert()

    #设置要转换的目标像素格式
    image_convert.set_dest_format(GxPixelFormatEntry.RGB8)
```

```

#设置要转换的有效位假设有效位有 12 位,选取高 8 位
image_convert.set_valid_bits(DxValidBit.BIT4_11)

#创建转换后的图像缓存
rgb_image_buffer_length =
    image_convert.image_format_convert_get_buffer_
    size_for_conversion(raw_image)
rgb_image_array = (c_ubyte * buffer_out_size)()
rgb_image = addressof(output_image_array)

#转换像素格式成 RGB8
image_convert.convert(raw_image, rgb_image,
    buffer_out_size, False)
if output_image is None:
    continue

#创建 numpy 数组
numpy_image = numpy.frombuffer(rgb_image_array,
    dtype=numpy.ubyte, count=rgb_image_buffer_length).\
    reshape(raw_image.frame_data.height,
    raw_image.frame_data.width, 3)

if numpy_image is None:
    continue

#显示并保存获得的 RGB 图片
image = Image.fromarray(numpy_image, 'RGB')
image.show()
image.save("image.jpg")

#停止采集,关闭设备
cam.stream_off()
cam.close_device()

```

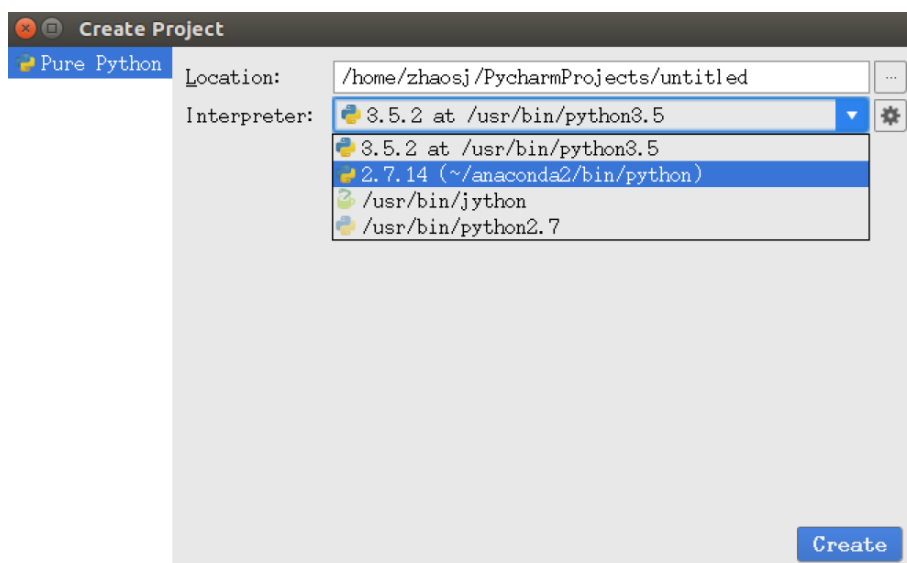
2. 编程指引

2.1. 搭建编程环境

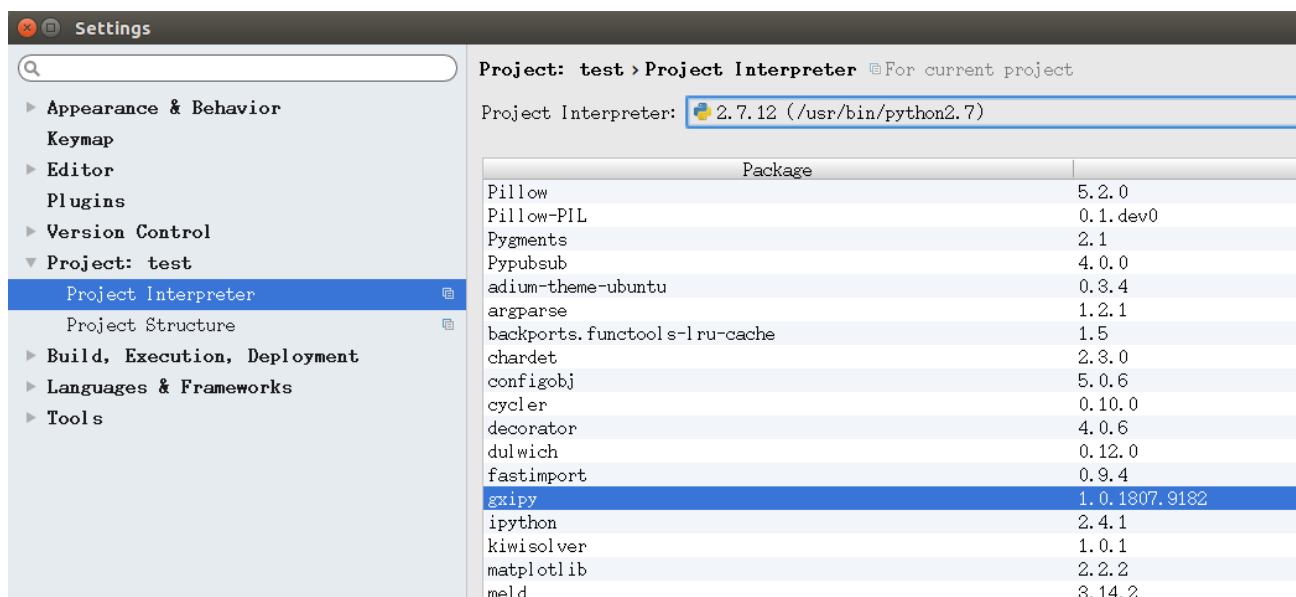
推荐用户优先使用 Python2.7、Python3.5 版本，本接口库已在上述两版本环境中测试通过。

2.1.1. Linux

- 1) 安装 pycharm-community（社区免费版本）：sudo apt-get install pycharm-community。
- 2) 新建 project，选择已安装 gxipy 库的 python 解释器，如果没有可选项，点击右侧的“设置”图标按钮，添加对应版本的 python 解释器。



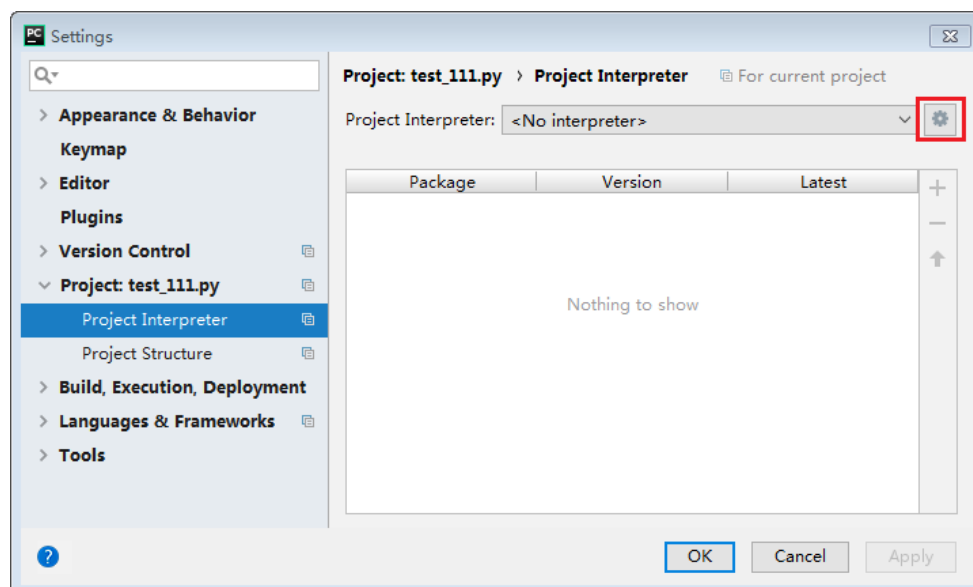
- 3) File->Settings->Project->Project Interpreter 可以查看 python 解释器已安装的 Package，如图表示 gxipy 库已正常安装，可以导入 gxipy 模块使用。



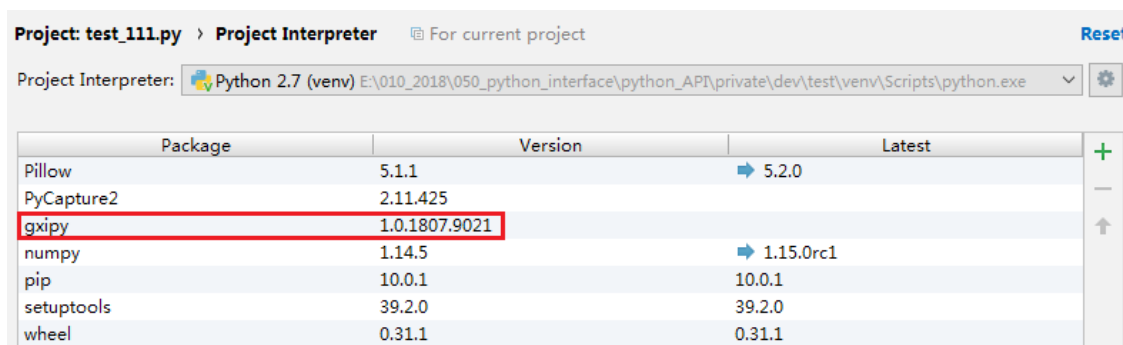
2.1.2. Windows

以 Python2.7、pycharm2018 平台为例，在 windows 7 64 位操作系统环境下演示安装 gxipy 库。初次安装 gxipy 库时：

1) 打开 pycharm2018 的工程配置 File->Settings->Project->Project Interpreter，点击下图红色方框出现的 Add。



2) 选择 New environment，选中 Inherit global site-packages 和 Make available to all projects，点击 OK。这时用户将在新建的 interpreter 中看到 gxipy 库已被成功加载到当前工程环境中。



2.2. 快速上手

2.2.1. 引入库

为使用已安装的库，在程序的开始引入库。在代码中任何使用库中的类、方法、数据类型时，请加自定义的前缀。

代码样例：

```
# 用户可自定义调用前缀，样例中使用了 gx
import gxipy as gx

device_manager = gx.DeviceManager()
```

2.2.2. 枚举设备

用户通过调用 [DeviceManager.update_device_list\(\)](#)枚举当前所有可用设备，函数返回值为设备数量 dev_num 和设备信息列表 dev_info_list。设备信息列表的元素个数为枚举到的设备个数，列表中元素的数据类型为字典，字典的键详见下表：

键名称	意义	类型
index	设备索引	整型
vendor_name	厂商名称	字符串
model_name	设备型号	字符串
sn	设备序列号	字符串
display_name	设备显示名称	字符串
device_id	设备标识	字符串
user_id	用户自定义名称	字符串
access_status	权限状态	GxAccessStatus
device_class	设备类	GxDeviceClassList
mac	mac 地址（GEV 相机特有）	字符串
ip	ip 地址（GEV 相机特有）	字符串
subnet_mask	子网掩码（GEV 相机特有）	字符串
gateway	网关（GEV 相机特有）	字符串
nic_mac	nic_mac 地址（GEV 相机特有）	字符串
nic_ip	nic_ip 地址（GEV 相机特有）	字符串
nic_subnet_mask	nic 子网掩码（GEV 相机特有）	字符串
nic_gateway	nic 网关（GEV 相机特有）	字符串
nic_description	nic 描述（GEV 相机特有）	字符串

枚举设备代码片段如下：

```
#枚举设备
device_manager = gx.DeviceManager()
dev_num, dev_info_list = device_manager.update_device_list()
if dev_num == 0:
    sys.exit(1)
```

注意：除上面的枚举接口，[DeviceManager](#) 还提供了另两个枚举接口 [DeviceManager.update_all_device_list\(\)](#) 和 [DeviceManager.update_device_list_ex\(\)](#)：

- 1) 对于非千兆网相机来说，这两个枚举接口功能上是一样的；
- 2) 对千兆网相机来说，库内部使用的枚举机制不一样：

update_all_device_list：使用全网枚举，能够枚举到局域网内的所有千兆网相机。

update_device_list：使用子网枚举，只能枚举到局域网内的同一网段的千兆网相机。

update_device_list_ex：使用全网枚举可以根据特定类型枚举设备 [update_device_list_ex](#) 枚举设备。

代码片段如下：

```
#枚举设备
device_manager = gx.DeviceManager()
dev_num, dev_info_list =
    device_manager.update_device_list_ex(
        GxTLClassList.TL_TYPE_CXP)

if dev_num == 0:
    sys.exit(1)
```

2.2.3. 打开关闭设备

用户通过调用以下五种不同的方式打开设备：

[DeviceManager.open_device_by_sn\(self, sn, access_mode=GxAccessMode.CONTROL\)](#)

[DeviceManager.open_device_by_user_id\(self, user_id, access_mode=GxAccessMode.CONTROL\)](#)

[DeviceManager.open_device_by_index\(self, index, access_mode=GxAccessMode.CONTROL\)](#)

[DeviceManager.open_device_by_ip\(ip, access_mode=GxAccessMode.CONTROL\)](#)

[DeviceManager.open_device_by_mac\(mac, access_mode=GxAccessMode.CONTROL\)](#)

sn：设备序列号

use_id：用户自定义名称

index：设备索引（1，2，3...）

mac：设备 mac 地址（非千兆网相机不可用）

ip：设备 ip 地址（非千兆网相机不可用）

注意：最后两个函数只针对千兆网相机使用。用户可以调用 [Device](#) 提供的 [Device.close_device\(\)](#) 接口来关闭设备，释放所有设备资源。

代码样例：

```
#用户可自定义调用前缀，样例中使用了 gx
import gxipy as gx

#枚举设备。dev_info_list 是设备信息列表，列表的元素个数为枚举到的设备个数，
#列表元素是
#字典，其中包含设备索引（index）、ip 信息（ip）等设备信息
device_manager = gx.DeviceManager()
tl_type = GxTLClassList.TL_TYPE_U3V | GxTLClassList.TL_TYPE_USB |
    GxTLClassList.TL_TYPE_GEV | GxTLClassList.TL_TYPE_CXP
dev_num, dev_info_list =
    device_manager.update_device_list_ex(tl_type)

if dev_num == 0:
    sys.exit(1)
```

```
# 打开设备

# 方法一
# 获取设备基本信息列表
str_sn = dev_info_list[0].get("sn")
# 通过序列号打开设备
cam = device_manager.open_device_by_sn(str_sn)

# 方法二
# 通过用户 ID 打开设备
# str_user_id = dev_info_list[0].get("user_id")
# cam = device_manager.open_device_by_user_id(str_user_id)
# 方法三
# 通过索引打开设备
# str_index = dev_info_list[0].get("index")
# cam = device_manager.open_device_by_index(str_index)

# 下面为只针对千兆网相机使用的打开方式

# 方法四
# 通过 ip 地址打开设备
# str_ip= dev_info_list[0].get("ip")
# cam = device_manager.open_device_by_ip(str_ip)

# 方法五
# 通过 mac 地址打开设备
# str_mac = dev_info_list[0].get("mac")
# cam = device_manager.open_device_by_mac(str_mac)

# 关闭设备
cam.close_device()
```

2.2.4. 采集控制

在设备正常开启并设置好相机采集参数后，用户可调用 [Device.stream_on\(\)](#)和 [Device.stream_off\(\)](#)执行开停采：

与采集相关的接口和控制都由 [DataStream](#) 提供，可通过设置循环次数控制采集图像次数。通过 [Device.get_stream_channel_num\(\)](#)接口获得设备的流通道个数。获取的图像使用 [RawImage.get_status\(\)](#)判断是否为残帧。代码如下：

- **get_image 方式**

```
# 开始采集
cam.stream_on()
```



```
# 获取流通道个数
# 如果 int_channel_num == 1, 设备只有一个流通道, 列表 data_stream 元素个数
# 为 1
# 如果 int_channel_num>1, 设备有多个流通道, 列表 data_stream 元素个数大于 1
# 目前千兆网相机、USB3.0、USB2.0 相机均不支持多流通道
# int_channel_num = cam.get_stream_channel_num()
# 获取数据
# num 为采集图片次数
num = 1
for i in range(num):
    # 打开第 0 通道数据流
    raw_image = cam.data_stream[0].get_image()
    if raw_image.get_status() == gx.GxFrameStatusList.INCOMPLETE:
        print("incomplete frame")

# 停止采集
cam.stream_off()
```

● dq_buf 方式

```
# 开始采集
cam.stream_on()
# 获取流通道个数
# 如果 int_channel_num == 1, 设备只有一个流通道, 列表 data_stream 元素个数
# 为 1
# 如果 int_channel_num>1, 设备有多个流通道, 列表 data_stream 元素个数大于 1
# 目前千兆网相机、USB3.0、USB2.0 相机均不支持多流通道
# int_channel_num = cam.get_stream_channel_num()
# 获取数据
# num 为采集图片次数
num = 1
for i in range(num):
    # 打开第 0 通道数据流
    raw_image = cam.data_stream[0].dq_buf()
    if raw_image.get_status() == gx.GxFrameStatusList.INCOMPLETE:
        print("incomplete frame")
    # dq_buf 必须与 q_buf 成对使用, 否则可能会导致无法持续采集
    cam.data_stream[0].q_buf(raw_image)

# 停止采集
cam.stream_off()
```

● 回调方式

```
# 定义采集回调函数
def capture_callback(raw_image):
    if raw_image.get_status() == gx.GxFrameStatusList.INCOMPLETE:
        print("incomplete frame")

# 注册回调
cam.data_stream[0].register_capture_callback(capture_callback)

# 开始采集
cam.stream_on()

# 等待一段时间，这段时间会自动调用采集回调函数
time.sleep(1)

# 停止采集
cam.stream_off()

# 注销回调
cam.data_stream[0].unregister_capture_callback()
```

● 帧信息

1. 原理：

开启帧信息后，相机将在采集返回的图像缓存之后紧接着附加一个帧信息块，其中包含 ID，长度以及用户希望获取的帧信息内容。

采集到图像作为第一个数据块放在首位，附加的帧信息块作为 2 号，3 号块依次添加。

下方的示意图展示了包含前置图像数据块和附加帧信息的数据块的集合示例。



手动解析时倒着从后往前解析，先读一个 32 位的长度，再读一个 32 位的 ID，紧接着读第一步读的长度内容。

注意：开启帧信息后图像由 GXGetPayloadSize()返回的大小会大于宽乘以高乘以像素位深的值。

2. 启用以及获取支持的帧信息种类

a) 具体可获得的帧信息数据可查看 GalaxyView 中属性树中帧信息控制部分，如图：

▼ 帧信息控制	
帧信息使能	false
时间戳	Not Available
帧号	Not Available
帧信息计数器值	Not Available
帧信息触发源	Not Available

图 2-1 网络帧信息节点

▼ 帧信息控制	
帧信息使能	false
帧信息项选择	FrameID
单项帧信息使能	false
帧号	Not Available
时间戳	Not Available

图 2-2 U3 帧信息节点

b) 开启帧信息

1) 将 ChunkModeActive 设置为 true

代码样例：

```
# 获取远端属性控制
remote_device_feature = cam.get_remote_device_feature_control()

# 开启 ChunkModeActive
if (remote_device_feature.is_implemented("ChunkModeActive") is not True):
    remote_device_feature.get_bool_feature("ChunkModeActive").set(True)
```

2) 如果相机支持 ChunkSelector 节点，则设置为如下可选的功能项，即想要开启的功能：

- FrameID
- Timestamp
- CounterValue

3) 通过将 ChunkEnable（块启用）参数设置为 true，启用所选块类型。

4) 针对每一个需要启用的帧信息，重复 2 到 3 步。

注意：以上 2 至 3 步仅在相机支持情况下设置

代码样例：

```
# 获取远端属性控制
remote_device_feature = cam.get_remote_device_feature_control()
```

```
# 启用帧 ID
if remote_device_feature.is_implemented("ChunkSelector")
    and remote_device_feature.is_writable("ChunkSelector"):
    remote_device_feature.get_enum_feature(
        "ChunkSelector").set("FrameID")

if remote_device_feature.is_implemented("ChunkEnable")
    and remote_device_feature.is_writable("ChunkEnable"):
    remote_device_feature.get_bool_feature(
        "ChunkEnable").set(True)
```

3. 通过帧信息属性控制器获取帧信息

- a) 通过采集回调获取的 RawImage 对象来获取帧信息属性控制对象，利用帧信息属性控制对象获取帧信息内容。

代码样例：

```
# 定义采集回调函数
def capture_callback(raw_image):

# 创建帧信息属性控制
try:
    if raw_image.get_status() == gx.GxFrameStatusList.SUCCESS:
        chunk_data_feature_ctl = image.
                                get_chunk_data_feature_control()

# 获取帧 ID
        chunk_id = chunk_data_feature_ctl.
                                get_int_feature("ChunkFrameID").get()
        print(f"ChunkFrameID: {chunk_id}>")
except Exception as ex:
    print(f"error: {str(ex)}")

# 注册回调
cam.data_stream[0].register_capture_callback(capture_callback)

# 开始采集
cam.stream_on()

# 等待一段时间，这段时间会自动调用采集回调函数
time.sleep(1)

# 停止采集
cam.stream_off()

# 注销回调
cam.data_stream[0].unregister_capture_callback()
```

- b) 通过 dq_buf 获取的 RawImage 对象来获取帧信息属性控制对象，利用帧信息属性控制对象获取帧信息内容。

代码样例：

```
# 开始采集
cam.stream_on()

# 打开第 0 通道数据流 并获取一张图像
raw_image = cam.data_stream[0].dq_buf()

# 创建帧信息属性控制
try:
    if raw_image.get_status() == gx.GxFrameStatusList.SUCCESS:
        chunk_data_feature_ctl = image.
            get_chunk_data_feature_control()

    # 获取帧 ID
    chunk_id = chunk_data_feature_ctl.
        get_int_feature("ChunkFrameID").get()
    print(f"ChunkFrameID: {chunk_id}>")

    # dq_buf 必须与 q_buf 成对使用，否则可能会导致无法持续采集
    cam.data_stream[0].q_buf(raw_image)
except Exception as ex:
    print(f"error: {str(ex)}")

    # dq_buf 必须与 q_buf 成对使用，否则可能会导致无法持续采集
    cam.data_stream[0].q_buf(raw_image)

# 停止采集
cam.stream_off()
```

- c) 通过 get_image 获取的 RawImage 对象来获取帧信息属性控制对象，利用帧信息属性控制对象获取帧信息内容。

代码样例：

```
# 开始采集
cam.stream_on()

# 打开第 0 通道数据流 并获取一张图像
raw_image = cam.data_stream[0].get_image()

# 创建帧信息属性控制
try:
    if raw_image.get_status() == gx.GxFrameStatusList.SUCCESS:
        chunk_data_feature_ctl = image.
            get_chunk_data_feature_control()
```

```
# 获取帧 ID
chunk_id = chunk_data_feature_ctl.
                get_int_feature("ChunkFrameID").get()
print(f"ChunkFrameID: {chunk_id}>")
except Exception as ex:
    print(f"error: {str(ex)}")

# 停止采集
cam.stream_off()
```

2.2.5. 图像处理

● 图像格式转换

图像格式转换的对象是采集图像 [DataStream.get_image\(\)](#)、[DataStream.dq_buf\(\)](#)得到的 raw_image。

功能描述：

支持任意像素格式的转换。详见的 [ImageFormatConvert.convert\(\)](#)接口。

代码样例：

```
# 创建设备管理器
device_manager = gx.DeviceManager()

# 枚举设备
dev_num, dev_info_list = device_manager.update_device_list()
if dev_num == 0:
    sys.exit(1)

#打开第一台设备
cam = self.device_manager.open_device_by_index(1)

# 开采
cam.stream_on()

# 获取 raw 图，假设其像素格式为 BayerRG12
raw_image = cam.data_stream[0].get_image()

# 停采
cam.stream_off()

# 创建格式转换器
image_convert = device_manager.create_image_format_convert()

# 设置要转换的目标像素格式
image_convert.set_dest_format(GxPixelFormatEntry.RGB8)

# 设置要转换的有效位
image_convert.set_valid_bits(DxValidBit.BIT4_11)
```

创建转换后的图像缓存

```
buffer_out_size =  
    image_convert.get_buffer_size_for_conversion(raw_image)  
output_image_array = (c_ubyte * buffer_out_size)()  
output_image = addressof(output_image_array)
```

转换像素格式成 RGB8

```
image_convert.convert(raw_image, output_image,  
                      buffer_out_size, False)  
if output_image is None:  
    continue
```

创建 numpy 数组

```
numpy_image = numpy.frombuffer(rgb_image_array, dtype=numpy.ubyte,  
                               count=rgb_image_buffer_length). \  
    reshape(raw_image.frame_data.height,  
            raw_image.frame_data.width, 3)  
  
if numpy_image is None:  
    continue
```

之后，用户可根据获取的 numpy_array 显示、保存图像

● 图像质量提升

本接口库还提供了软件端的图像质量提升接口，用户可以有选择的进行颜色校正、对比度、Gamma 等图像质量提升的操作。用户可调用 ImageProcess 的接口 ImageProcess.image_improvement()实现功能。一个简单的使用范例如下：

假设对像素格式为 RGB8 的图像做图像质量提升

创建设备管理器

```
device_manager = gx.DeviceManager()
```

打开第一台设备

```
cam = device_manager.open_device_by_index(1)
```

创建图像质量提升处理对象

```
image_process = device_manager.create_image_process()
```

创建图像质量提升参数配置对象

```
image_config = cam.create_image_process_config()
```

创建属性控制器

```
remote_device_feature = cam.get_remote_device_feature_control()
```

#获取对比度、Gamma 参数

```

if remote_device_feature.is_readable("GammaParam"):
    gamma_param =
        remote_device_feature.get_float_feature("GammaParam").get()
else:
    gamma_param = 0.1

if remote_device_feature.is_readable("ContrastParam"):
    contrast_param =
        remote_device_feature.get_int_feature("ContrastParam").get()
else:
    contrast_param = 0

#设置对比度、Gamma 等参数
image_config.set_contrast_param(gamma_param)
image_config.set_gamma_param(contrast_param)

# 创建图像质量提升后存放的 Buffer 缓存
output_image_array = (c_ubyte * raw_image.frame_data.image_size)()
output_image = addressof(output_image_array)

# 采集获取图像 raw_image，调用图像处理接口实现质量提升
image_process.image_improvement(raw_image, output_image,
                                image_config)

```

1) 颜色校正

设备属性参数名称：ColorCorrectionParam

名词解释：提高相机色彩还原度，使图像更加接近人眼视觉感受。

2) 对比度调节

Contrast 值：整型，范围[-50, 100]，缺省值为 0

设备属性参数名称：ContrastParam

名词解释：图像明亮部分与黑暗部分的亮度比称为对比度，又叫反差。对比度高或者反差大的图像，其中被摄景物的轮廓较清楚，图像也较清晰；反之，对比度低的图像轮廓不清，图像也不太清晰。

3) Gamma 调节

Gamma 值：整型或浮点型，范围[0.1, 10.0]，缺省值为 1

设备属性参数名称：GammaParam

名词解释：Gamma 调节是为了让显示器的输出尽量接近输入。

● 图像显示和保存

调用 PIL (Python Imaging Library) 的接口 Image.fromarray()，将 numpy 数组转换成 Image 图像，显示并保存。代码如下：

1) 黑白相机

```

# 显示并保存获得的黑白图片
image = Image.fromarray(numpy_image, 'L')

```



```
image.show()
image.save("acquisition_mono_image.jpg")
```

2) 彩色相机

显示并保存获得的彩色图片

```
image = Image.fromarray(numpy_image, 'RGB')
image.show()
image.save("acquisition_RGB_image.jpg")
```

2.2.6. 相机控制

● 属性控制种类

属性控制分如下几种。

- 1) Device.get_local_device_feature_control(): 获取本地设备属性控制器。
- 3) Device.get_remote_device_feature_control(): 获取远端设备属性控制器。
- 4) DeviceManagetr.get_interface().get_feature_control(): 获取 interface 属性控制器。
- 5) Device.get_stream().get_feature_control(): 获取流属性控制器。

通过 GalaxyView 演示程序打开设备，见右侧属性控制树，在设备属性页中分三个区间，上面区间对应功能是 Device.get_remote_device_feature_control() 返回的属性集合（图 2-3），中间区间对应功能是 Device.get_local_device_feature_control() 返回属性集合（图 2-3），下面区间对应功能是 Device.get_stream().get_feature_control() 返回的属性集合（图 2-3）。

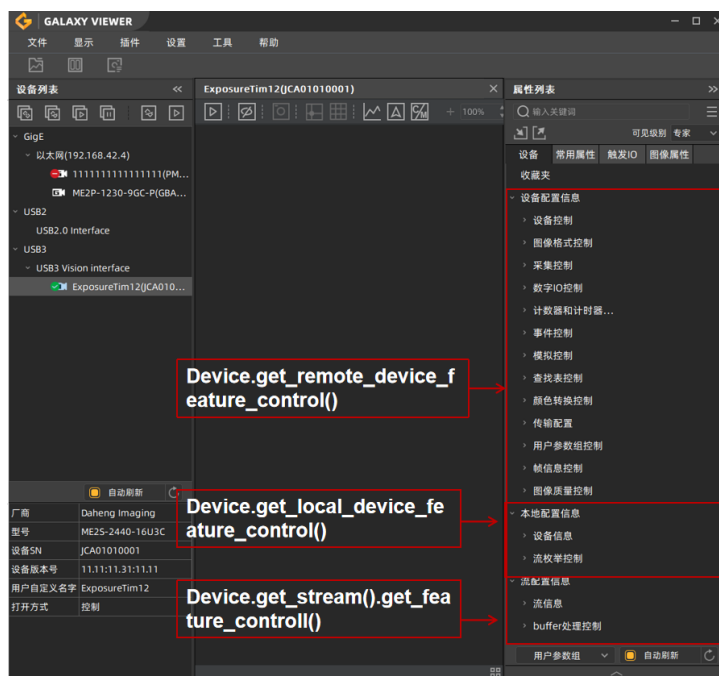


图 2-3 远端、本地和流属性集合

通过 GalaxyView 演示程序点击 Interface 节点，见右侧属性控制树，在属性页对应功能是 DeviceManagetr.get_interface().get_feature_control() 返回的属性集合（图 2-4）。

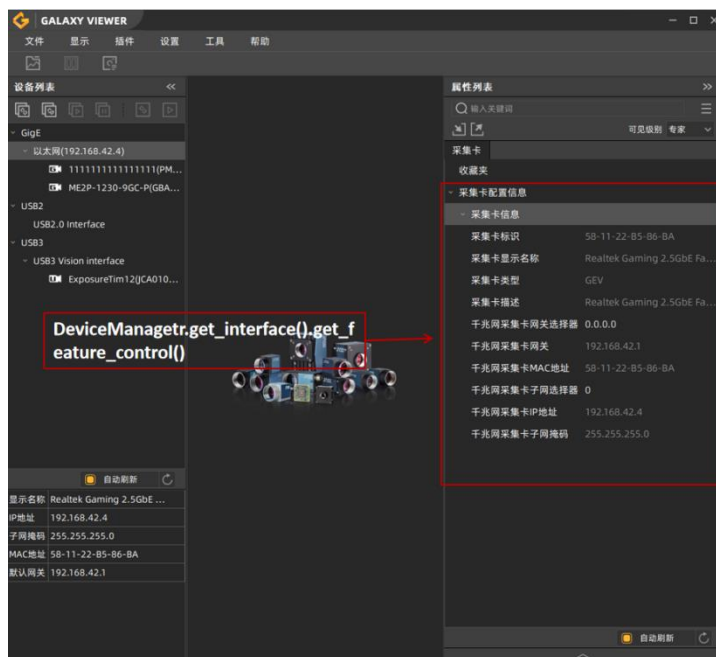


图 2-4 Interface 属性集合

● 属性数据节点访问类型

属性数据节点的访问分三种类型：是否实现、是否可读、是否可写。接口设计如下：

FeatureControl.is_implement(feature_name) 此功能是否支持

FeatureControl.is_readable(feature_name) 此功能是否可读

FeatureControl.is_writable(feature_name) 此功能是否可写

建议用户在操作属性参数前，先查询属性的访问类型。

代码样例：

```
# 查询数据节点是否支持
remote_device_feature = cam.get_remote_device_feature_control()
is_implemented =
    remote_device_feature.is_implemented("PixelFormat")
if is_implemented == True:
    # 查询数据节点是否可写
    is_writable = remote_device_feature.is_writable("PixelFormat")
    if is_writable == True:
        # 设置像素格式
        remote_device_feature.get_enum_feature("PixelFormat").
            set("Mono8")

# 查询数据节点是否可读
is_readable = remote_device_feature.is_readable("PixelFormat")
if is_readable == True:
    # 打印像素格式
    value, str_value =
        remote_device_feature.get_enum_feature("PixelFormat").get()
    print(str_value)
```

- 整型数据节点

相关接口：

IntFeature_s.set(int_value) //设置当前值
IntFeature_s.get() //读取当前值
IntFeature_s.get_range() //获取最小值、最大值、步长

代码样例：

```
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 获取图像宽度数据节点对象
width_feature = remote_device_feature.get_int_feature("Width")

# 获取图像宽度可设置范围
width_range = width_feature.get_range()

# 设置当前图像宽度为范围内任意值
Width_feature.set(800)

# 获取当前图像宽度
int_Width_value = width_feature.get()
```

- 浮点型数据节点

相关接口：

FloatFeature_s.set(float_value) //设置当前值
FloatFeature_s.get() //读取当前值
FloatFeature_s.get_range() //获取最小值、最大值、步长、单位、单位是否有效

代码样例：

```
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 获取曝光时间数据节点对象
exposure_time_feature =
    remote_device_feature.get_float_feature("ExposureTime")

# 获取曝光值可设置范围和最大值
float_range = rexposure_time_feature.get_range()
float_max = float_range["max"]

# 设置当前曝光值范围内任意值
exposure_time_feature.set(10.0)

# 获取当前曝光值
float_exposure_value = exposure_time_feature.get()
```

- 枚举型数据节点

相关接口：

EnumFeature_s.set (enum_value) //设置
EnumFeature_s.get() //读取
EnumFeature_s.get_range() //获取字典

代码样例：

```
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 获取像素格式数据节点对象
pixel_format_feature =
    remote_device_feature.get_enum_feature("PixelFormat")

# 获取枚举值可设置范围
enum_range = pixel_format_feature.get_range()

# 设置当前枚举值
pixel_format_feature.set("Mono8")

# 打印当前枚举值
enum_PixelFormat_value, enum_PixelFormat_key =
    pixel_format_feature.get()
```

- 布尔型数据节点

相关接口：

BoolFeature_s.set (bool_value) //设置当前值
BoolFeature_s.get() //读取当前值

代码样例：

```
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 获取引脚电平反转数据节点对象
line_inverter_feature =
    remote_device_feature.get_bool_feature("LineInverter")

# 设置当前布尔值
line_inverter_feature.set(True)

# 读取布尔值
bool_LineInverter_value = line_inverter_feature.get()
```

● 字符串型数据节点

相关接口：

StringFeature_s.set (string_value)	//设置当前值
StringFeature_s.get()	//读取当前值
StringFeature_s.get_string_max_length()	//获取字符串型属性参数的最大长度值

代码样例：

```
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 获取用户自定义名称数据节点对象
device_user_id_feature =
    remote_device_feature.get_string_feature("DeviceUserID")

# 读取最长字符串长度
string_max_length = device_user_id_feature.get_string_max_length()

# 读取当前字符串值
current_string = device_user_id_feature.get()

# 设置字符串值
device_user_id_feature.set("MyUserID")
```

● Buffer 型数据节点

相关接口：

RegisterFeature_s.set_buffer (buf)	//设置当前值
RegisterFeature_s.get_buffer()	//读取当前值
RegisterFeature_s.get_register_length()	//获取 Buffer 型属性参数的长度

代码样例：

```
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 获取寄存器数据节点对象
data_field_value_all_feature =
    remote_device_feature.get_register_feature(
        "DataFieldValueAll")

# 读取节点数据长度
buffer_length = data_field_value_all_feature.get_register_length()

# 设置节点数据
string_set = "abcd"
```

```
set_buffer = gx.Buffer.from_string(string_set_1.encode('utf-8'))
data_field_value_all_feature.set_buffer(string_set)
```

读取节点数据

```
buffer_data = data_field_value_all_feature.get_buffer()
print("buffer_data: %s" % (buffer_data.get_data().decode()))
```

● Command 型数据节点

相关接口：

CommandFeature_s.send_command //发送命令

代码样例：

获取远端属性控制器

```
remote_device_feature = cam.get_remote_device_feature_control()
```

获取软触发命令数据节点对象

```
trigger_soft_ware_feature =
    remote_device_feature.get_register_feature(
        "TriggerSoftware")
```

发送命令：软触发

```
trigger_soft_ware_feature.send_command()
```

不同类型的设备具备的属性功能也略有差别。

在附录【[属性参数](#)】中可获取相机所有的属性参数。

2.2.7. 掉线重连

2.2.7.1. 流程

GigE Vision 相机和 U3 Vision 相机提供本地设备事件，包括：设备掉线事件、设备断线事件、设备重连事件。获取事件流程如下：

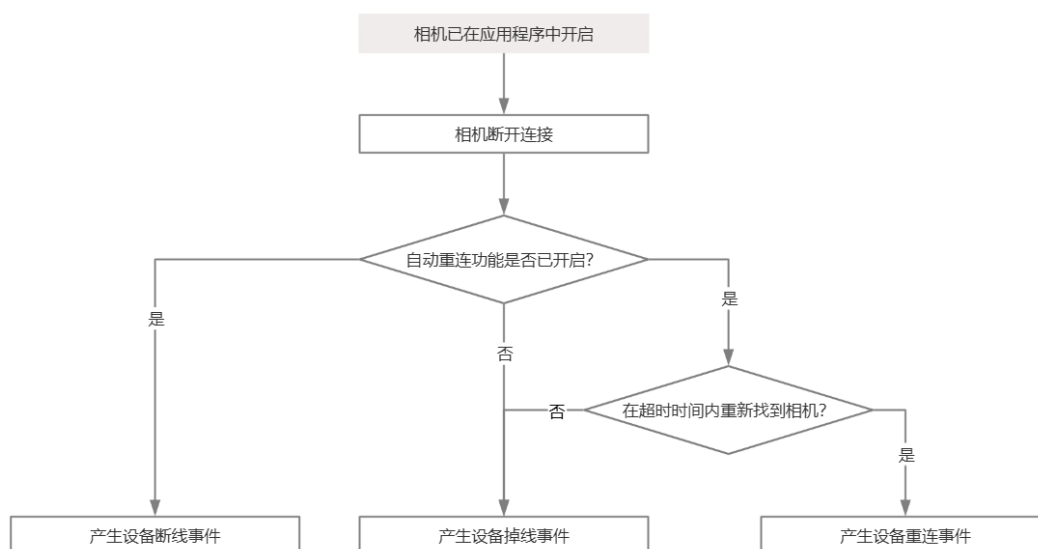


图 2-5 掉线重连流程

自动重连功能未开启时，如果注册了设备掉线事件，就会直接收到设备掉线事件。

自动重连功能开启时，如果注册了设备掉线事件、设备断线重连事件，会在下列情况下收到事件：

1. 设备断开连接的第一时间，API 会调用设备断线事件回调。
2. 在重连超时时间内能够找到相机，API 会调用设备重连事件回调。
3. 在重连超时时间内没有找到相机，API 会调用设备掉线事件回调。

注：如果相机 IP 地址改变，掉线重连功能在此情况下无法使用。

2.2.7.2. 支持范围

	自动重连		恢复图像采集	恢复参数	
	开启	关闭		通过 UserSet	自动
GigE Vision Transport Layer Socket	✓	✓	✓	✓	×
USB3 Vision Transport Layer	✓	✓	✓	✓	×
GxI API C	✓	✓	✓	✓	×
GxI API C++	✓	✓	✓	✓	×
GxI API C#	✓	✓	✓	✓	×
GxI API Python	✓	✓	✓	✓	×

如果用户期望开启自动重连功能，在相机重连成功后，API 自动帮助用户恢复采集状态，则用户必须在开始采集前手动将当前参数保存为 UserSet0，并且启动参数组修改为 UserSet0。否则，如果重连前后相机的图像宽、高、像素格式等属性发生变化，采集可能面临异常，如图像错乱，程序崩溃。

以网络相机为例，设备掉线，但是未掉电时，参数不会发生变化。这种场景下，用户可以不设置 UserSet0 为启动参数组。

2.2.7.3. 属性节点

用户可以通过本地设备层的相关属性节点控制自动重连功能开启、关闭，以及设置重连超时时间。

模块	属性节点	说明
LocalDevice/DeviceAutoConnection	EnableAutoConnection	是否启用自动重连
	DeviceConnectStatus	设备连接状态
	AutoConnectCounts	自动重连次数
	AutoConnectTimeout	自动重连超时时间，单位毫秒

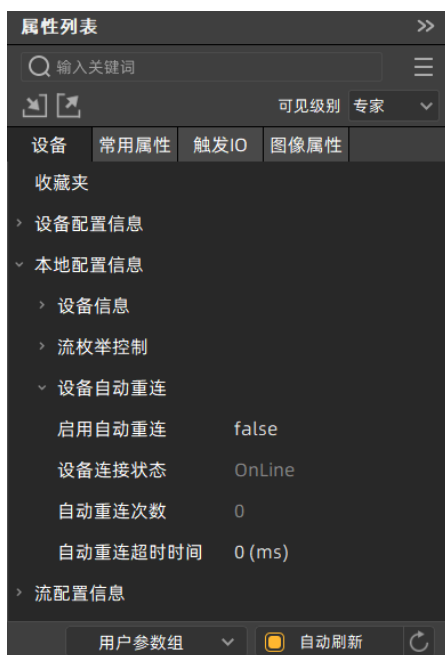


图 2-6 自动重连属性节点

2.2.7.4. 代码样例

GigE Vision 相机和 U3 Vision 相机提供重连通知事件机制、断线通知事件机制，用户可以通过回调通知的方式获取设备重连事件、设备断线事件，打开设备之后就可以注册事件。

获取重连事件分成以下几步：

- 1) 开启断线重连功能；
- 2) 注册重连事件；
- 3) 设备重连之后触发回调函数；
- 4) 注销重连事件。

获取断线事件分成以下几步：

- 1) 开启断线重连功能；
- 2) 注册断线事件；
- 3) 设备断线之后触发回调函数；
- 4) 注销断线事件。

注意：在关闭设备之前一定要注销事件。

在重连回调函数内部不允许执行关闭设备操作，否则抛出非法调用异常。

在断线回调函数内部不允许执行关闭设备操作，否则抛出非法调用异常。

```
import gxipy as gx
from gxipy.gxidef import *

def reconnect_callback():
    # 收到重连通知，用户根据需求进行操作...
```



```
def disconnect_callback():
    # 收到断线通知，用户根据需求进行操作...

def main():
    # 打开设备管理器
    device_manager = gx.DeviceManager()
    dev_num, dev_info_list = device_manager.update_all_device_list()
    if dev_num == 0:
        return

    # 打开相机
    cam = device_manager.open_device_by_index(1)
    remote_device_feature = cam.get_remote_device_feature_control()

    # 注册重连回调、断线回调
    cam.register_device_reconnect_callback(reconnect_callback)
    cam.register_device_disconnect_callback(disconnect_callback)

    # 开启断线重连功能
    local_device_feature = cam.get_local_device_feature_control()
    local_device_feature.get_bool_feature("EnableAutoConnection").set(True)

    # .....

    local_device_feature.get_bool_feature("EnableAutoConnection").set(False)

    # 注销重连回调、断线回调
    cam.unregister_device_reconnect_callback()
    cam.unregister_device_disconnect_callback()

    # 关闭相机
    cam.close_device()

if __name__ == "__main__":
    main()
```

2.2.7.5. 注意事项

- 1) 如果相机重连成功，相机在重连过程中可能会产生一个残帧。
- 2) 如果使用断线重连功能强烈建议用户使用默认参数组，不设置参数。如果未使用默认参数组运行相机，修改过 ROI 属性可能会导致重连后相机无法恢复开采问题。如果断线后修改自动重连使能状态，可能存在依然进行重连问题。
- 3) 相机在重连后，相机的传输数据信息恢复到刚采集状态，例如：交付帧数、残帧等归零。

2.2.8. 导入导出相机配置参数

在接口库中有供用户调用的导入导出设备配置文件的接口代码样例：

```
# 导入设备远端属性控制器配置参数文件
# 获取远端属性控制器
remote_device_feature = cam.get_remote_device_feature_control()
remote_device_feature.feature_load("import_config_file.txt")
# 导出设备远端属性控制器配置参数文件
remote_device_feature.feature_save("export_config_file.txt")
```

2.2.9. 属性设置 UI

GxI API 基于 WxWidget 以弹出式，默认非阻塞的方式提供了属性设置窗口 UI，该功能支持 C/C++/C#/python 这 4 种语言。

2.2.9.1. 功能介绍

功能控件	功能说明
搜索	搜索栏输入相机属性节点关键词后会过滤出相机中所有包含该关键词的节点，点击对应的项即可定位到该节点对应的设置位置。
展开收起	点击展开即可将属性树中全部节点展开显示，点击收起将属性树中节点收起。
自动更新	勾选自动更新后开启 Polling 功能，此时当相机中属性发生变化时，属性树中 UI 界面会有相应的变化。默认不勾选自动更新。
可见级别	可见级别分为 Beginner、Expert、Guru 三个，选中对应的可见级别即可在属性树中筛选出对应的属性节点。
属性树	属性树中依次显示相机设备信息、流信息、本地配置信息、采集卡信息。可选中对应的子项进行设置。



图 2-7 属性设置 UI

2.2.9.2. 代码样例

显示属性窗口步骤：

- 1) 创建属性窗口句柄；
- 2) 设置属性窗口显示位置；
- 3) 设置属性窗口显示模式(BLOCK_SHOW_MODE 模式仅在控制台下设置，GUI 程序中请勿设置否则可能发生未定义行为。)；
- 4) 设置属性窗口标题，如果未设置则默认显示用户自定义名称，如果自定义名称也未设置则显示相机对应的型号、序列号作为标题；
- 5) 销毁属性窗口。

```
import gxipy as gx
from gxipy.gxidef import *

def main():
    # 打开设备管理器
    device_manager = gx.DeviceManager()
    dev_num, dev_info_list = device_manager.update_all_device_list()
    if dev_num == 0:
        return

    # 打开相机
    cam = device_manager.open_device_by_index(1)
    # .....

    #1. 创建窗口操作句柄
    wndh = cam.create_window(GxWindowsID.WINDOW_PROPERTY, 0)

    #2. 设置属性窗口显示位置位于相对屏幕左上角，宽、高均为 700
    cam.set_window_position(wndh, 0, 0, 700, 700)

    #3. 设置属性窗口显示模式为阻塞（当程序是控制台时需要设置
    #BLOCK_SHOW_MODE，GUI 程序则请勿设置）
    cam.set_window_mode(wndh, GxWindowsShowMode.BLOCK_SHOW_MODE)

    #4. 设置属性窗口标题
    cam.set_window_title(wndh, "title")

    #5. 显示窗口
    cam.show_window(wndh, True)

    #6. 销毁窗口
    cam.destroy_window(wndh)

    # 关闭相机
```

```
cam.close_device()

if __name__ == "__main__":
    main()
```

2.2.9.3. 注意事项

1. 此属性窗口不支持逐像素平场校正保存、加载系数功能，如果需要操作该功能请使用 GalaxyView 或平场插件。
2. 阻塞显示模式(BLOCK_SHOW_MODE)仅适用与控制台框架下调用，如果您的程序是 GUI 程序如(Qt、MFC、Winforms 等)请勿调用，否则可能发生未定义行为。

2.2.10. 存图存视频

2.2.10.1. 存图

存图示例关键代码：

```
dev_num, dev_info_list = device_manager.updata_device_list()

_obj_manager = gx.DeviceManager()
# 枚举设备
dev_num, dev_info_list = cls._obj_manager.update_device_list()

_obj_cam = cls._obj_manager.open_device_by_index(1)
_obj_remote_feature_control =
    cls._obj_cam.get_remote_device_feature_control()
_obj_cam_stream = cls._obj_cam.get_stream( 1)
self._obj_cam.stream_on()

image = self._obj_cam.data_stream[0].get_image(1000)

media_manager = MediaProc()
image_info = GxSaveImageInfo()
image_info.image_buf = image.frame_data.image_buf
image_info.width = image.frame_data.width
image_info.height = image.frame_data.height
image_info.src_format = image.get_pixel_format()
image_info.image_format =
    GxImageFormatType.GX_IMAGE_FORMAT_JPEG
image_info.cfa_method = GxCFAMethod.GX_CFA_METHOD_BALANCE
image_info.image_path = b"test_save_image.jpeg"
image_info.image_quality = 99
# 保存图片
media_manager.save_image(image_info)

# 停止采集
_obj_cam.stream_off()
_obj_remote_feature_control.get_command_feature('AcquisitionStop')
.send_command()
```

2.2.10.2. 存视频

存视频示例：

```
dev_num, dev_info_list = device_manager.updata_device_list()

_obj_manager = gx.DeviceManager()
# 枚举设备
dev_num, dev_info_list = cls._obj_manager.update_device_list()

_obj_cam = cls._obj_manager.open_device_by_index(1)
_obj_remote_feature_control =
    cls._obj_cam.get_remote_device_feature_control()
_obj_cam_stream = cls._obj_cam.get_stream( 1)
self._obj_cam.stream_on()

image = _obj_cam.data_stream[0].get_image(1000)
record_param = GxRecordParam()
record_param.pixel_format = image.get_pixel_format()
record_param.video_format =
    GxVideoFormatType.GX_VIDEO_FORMAT_H264_AVI
record_param.width = image.frame_data.width
record_param.height = image.frame_data.height
record_param.frame_rate = 20
record_param.bit_rate = 1
record_param.video_path = b"test_create_video_saver.avi"
media_manager = MediaProc()
#初始化录制参数
saver = media_manager.create_video_saver(record_param)
for i in range(100):
    image = self._obj_cam.data_stream[0].get_image(1000)
    # 添加视频帧
    saver.add_frame(image.frame_data.image_buf)
#停止录制
saver.close()

# 停止采集
_obj_cam.stream_off()
_obj_remote_feature_control.get_command_feature('AcquisitionStop')
.send_command()
```

2.2.11. 错误处理

当调用接口函数内部出现异常时，错误处理机制会检测并抛出不同类型异常，异常类型均继承自Exception。

一个典型的错误处理代码样例：

```
try:
# 调用接口函数时函数内部抛异常
dev_num, dev_info_list = device_manager.updata_device_list()
except Exception as exception:
```

```
print("打印错误信息：%s" % exception)
exit(1)
```

用户也可通过判断捕获的具体错误类型，进行分类处理：

```
if isinstance(exception, OutOfRange):
    print("OutOfRange: %s" % exception)
elif isinstance(exception, OffLine):
    print("OffLine: %s" % exception)
else:
    print("Other Error Type %s" % exception)
```

异常类型：

异常类型	意义
UnexpectedError	未预测
NotFoundTL	没找到 TL
NotFoundDevice	未找到设备
OffLine	掉线
InvalidParameter	参数无效
InvalidHandle	句柄无效
InvalidCall	无效回调
InvalidAccess	无效获取
NeedMoreBuffer	Buffer 不足
FeatureTypeError	功能码错误
OutOfRange	超过范围
NoImplemented	未实现
NotInitApi	未初始化
Timeout	超时
ParameterTypeError	参数类型错误

3. 附录

3.1. 功能类定义

3.1.1. Feature_s(推荐)

Feature_s 类是：[IntFeature_s](#) / [FloatFeature_s](#) / [EnumFeature_s](#) / [BoolFeature_s](#) / [StringFeature_s](#) / [BufferFeature_s](#) / [CommandFeature_s](#) 属性类的父类。

已知：通过 [DeviceManager](#) 已经打开相机相机获取了 [Device](#) 对象，变量名是 cam。

第一步：通过 Device 对象获取对应的 feature_control 对象，比如：

```
obj_feature_control = cam.get_remote_device_feature_control()
```

第二步：通过 feature_control 对象获取指定属性名称的数据节点对象：

比如访问一个整型的数据节点 Width

```
obj_width = remote_device_feature.get_int_feature("Width")
```

```
obj_width.set(2048)
```

再比如访问一个浮点型的数据节点 Gain

```
obj_gain = remote_device_feature.get_float_feature("Gain")
```

```
obj_gain.set(1.0)
```

总之，用户根据节点名称字符串配合对应的数据类型（整形、浮点型、枚举型等）进行属性的访问。

3.1.1.1. IntFeature_s

负责读写、控制相机的整型数据节点，继承自 [Feature_s](#) 类。

接口列表：

[get_range\(\)](#) 获取整型数据节点参数范围

[get\(\)](#) 获取当前值

[set\(value\)](#) 设置当前值

3.1.1.1.1.get_range

声明：IntFeature_s.get_range()

意义：获取整型数据节点参数范围。

返回值：记录整型数据节点参数范围。键包含：min 最小值，max 最大值，step 步长，value 当前值。

异常处理：获取整型数据节点参数范围失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.1.2.get

声明：IntFeature_s.get()

意义：获取当前值。

返回值：获取整型数据节点对应的当前值。

异常处理：如果获取整型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.1.3.set

声明：IntFeature_s.set(int_value)

意义：设置当前值。

形参：设置的整型值。

异常处理：设置整型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.2. FloatFeature_s

负责查看、控制相机的浮点型数据节点，继承自 [Feature_s](#) 类。

接口列表：

get_range()	获取浮点型数据节点参数范围
get()	获取当前值
set(float_value)	设置当前值

3.1.1.2.1.get_range

声明：FloatFeature_s.get_range()

意义：获取浮点型数据节点参数范围。

返回值：记录浮点型数据节点参数范围。键包含：min 最小值，max 最大值，inc 步长，unit 单位，inc_is_valid 单位是否有效，cur_value 当前值。

异常处理：获取浮点型数据节点参数范围失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.2.2.get

声明：FloatFeature_s.get()

意义：获取当前值

返回值：获取的浮点型属性参数值。

异常处理：获取浮点型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.2.3.set

声明：FloatFeature_s.set(float_value)

意义：设置当前值

形参：[in] float_value 设置的浮点型数值

异常处理：设置浮点型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.3. EnumFeature_s

负责查看、控制相机的枚举型数据节点，继承自 [Feature_s](#) 类。

接口列表：

get_range()	获取枚举型数据节点参数范围
get()	获取当前的枚举项值
set(enum_value)	获取当前的枚举项值

3.1.1.3.1.get_range

声明：EnumFeature_s.get_range()

意义：获取枚举型数据节点参数范围。

返回值：记录枚举型数据节点参数范围的。

异常处理：获取枚举型数据节点参数范围失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.3.2.get

声明：EnumFeature_s.get()

意义：获取当前的枚举项值

返回值：

- 1) 枚举型数据节点的当前值。
- 2) 枚举型数据节点参数的字符串。

异常处理：获取枚举型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.3.3.set

声明：EnumFeature_s.set(enum_value)

意义：设置当前的枚举项值

形参：[in] enum_value 设置的枚举型数值

注：可以是枚举值，也可以枚举类型对应的字符串值。

异常处理：设置枚举型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.4. BoolFeature_s

负责查看、控制相机的布尔型数据节点，继承自 [Feature_s](#) 类。

接口列表：

get()	获取当前值
set(bool_value)	设置当前值

3.1.1.4.1.get

声明：BoolFeature_s.get()

意义：获取当前值。

返回值：获取的布尔值。

异常处理：获取布尔型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.4.2.set

声明：BoolFeature_s.set(bool_value)

意义：设置当前值。

形参：[in] bool_value 设置的布尔型当前值

异常处理：设置布尔型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.5. StringFeature_s

负责查看、控制相机的字符串型数据节点，继承自 [Feature_s](#) 类。

接口列表：

get_string_max_length()	获取可设置的最大长度，不包含结束符
get()	获取当前值
set(input_string)	设置当前值

3.1.1.5.1.get_string_max_length

声明：StringFeature_s.get_string_max_length()

意义：获取可设置的最大长度，不包含结束符。

返回值：字符串型数据节点可设置的最大长度。

异常处理：获取字符串型数据节点可设置最大长度失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.5.2.get

声明：StringFeature_s.get()

意义：获取当前值。

返回值：获取的字符串型数据节点当前值。

异常处理：获取字符串型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.5.3.set

声明：StringFeature_s.set(input_string)

意义：设置当前值。

形参：[in] input_string 设置的字符串型节点当前值

异常处理：设置字符串型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.6. RegisterFeature_s

负责查看、控制相机的寄存器数据节点，继承自 [Feature_s](#) 类。

接口列表：

get_register_length()	获取 buffer 的长度。用户读取此长度用来读写 buffer
get_buffer()	获取 buffer 值。输入参数 ptrBuffer 的长度必须等于 get_register_length()接口获取的长度
set_buffer(buf)	设置 buffer 值。输入参数 ptrBuffer 的长度必须等于 get_register_length()接口获取的长度

3.1.1.6.1.get_register_length

声明：RegisterFeature_s.get_register_length()

意义：获取 buffer 的长度。用户读取此长度用来读写 buffer。

返回值：寄存器型属性参数的长度。

异常处理：获取寄存器型数据节点长度失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.6.2.get_buffer

声明：RegisterFeature_s.get_buffer()

意义：获取 buffer 值。输入参数 ptrBuffer 的长度必须等于 get_register_length()接口获取的长度。

返回值：寄存器型当前数据。

异常处理：获取寄存器型数据节点数据失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.6.3.set_buffer

声明：RegisterFeature_s.set_buffer(buf)

意义：设置 buffer 值。输入参数 ptrBuffer 的长度必须等于 get_register_length()接口获取的长度。

形参：[in] buffer 设置的寄存器节点当前数据

异常处理：设置寄存器型数据节点当前数据失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.1.7. CommandFeature_s

负责发送命令型数据节点，继承自 [Feature_s](#) 类。

接口列表：

[send_command\(\)](#) 执行命令

3.1.1.7.1.send_command

声明：CommandFeature_s.send_command()

意义：执行命令。

异常处理：如果发送命令不成功，则抛出异常，异常类型详见[错误处理](#)。

3.1.2. Feature(不再维护)

负责查看各种类型数据节点，判断其是否已实现、可读、可写。

Feature 类是：[IntFeature](#) / [FloatFeature/EnumFeature](#) / [BoolFeature](#) / [StringFeature](#) / [BufferFeature](#) / [CommandFeature](#) 数据节点类的父类。这种方式的数据节点是通过 Device 对象直接访问。

已知：通过 DeviceManager 已经打开相机相机获取了 Device 对象，变量名是 cam。

比如设置一个整型数据节点的当前值 Width：cam.Width.set(2048)；

比如设置一个浮点型数据节点的当前值 Gain：cam.Gain.set(1.0)。

这种方式的弊端是当 cam 有新增数据节点的时候，需要升级 python 库以支持新功能，对旧版本 python 库的用户不友好，所以此方式的属性控制不再维护升级新增功能，已有功能对象仍然可以继续使用。

后续推荐用户使用 feature_control 对象通过数据节点字符串名称获取属性对象访问，这种方式可以不需要升级 python 库就可以兼容新的相机功能。

3.1.2.1. IntFeature

负责查看、控制相机的整型数据节点，继承自 [Feature](#) 类。

接口列表：

[is_implemented\(\)](#) 判断整型数据节点是否已实现

is_readable()	判断整型数据节点是否可读
is_writable()	判断整型数据节点是否可写
get_range()	获取整型数据节点范围字典
get()	读取整型数据节点值
set(int_value)	设置整型数据节点值

3.1.2.1.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.1.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.1.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.1.4.get_range

声明：IntFeature.get_range()

意义：获取整型数据节点参数范围。

返回值：记录整型数据节点参数范围。键包含：min 最小值，max 最大值，step 步长

异常处理：

- 1) 如果该整型数据节点功能未实现，则打印不支持该整型数据节点获取范围的信息，函数返回 None。
- 2) 如果获取该整型数据节点范围不成功，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.1.5.get

声明：IntFeature.get()

意义：读取整型数据节点当前值。

返回值：获取的整型数据节点当前值。

异常处理：

- 1) 如果该整型数据节点未实现或不可读，则打印该整型数据节点不可读的信息，函数返回 None。
- 2) 如果获取该整型数据节点当前值不成功，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.1.6.set

声明：IntFeature.set(self, int_value)

意义：设置整型数据节点当前值

形参：设置的整型数据节点当前值。

异常处理：

- 1) 如果输入参数不是整型值，则抛出 ParameterTypeError 异常。
- 2) 如果该整型数据节点功能未实现或不可写，则打印该整型数据节点不可写的信息，函数返回

None。

- 3) 如果输入参数不在该整型数据节点的参数范围内，则打印超过该整型数据节点范围的信息并打印范围，函数返回 None。
- 4) 如果设置该整型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.2. FloatFeature

负责查看、控制相机的浮点型数据节点，继承自 [Feature](#) 类。

接口列表：

is_implemented()	判断浮点型数据节点是否已实现
is_readable()	判断浮点型数据节点是否可读
is_writable()	判断浮点型数据节点是否可写
get_range()	获取浮点型数据节点范围字典
get()	读取浮点型数据节点当前值
set(float_value)	设置浮点型数据节点当前值

3.1.2.2.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.2.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.2.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.2.4.get_range

声明：FloatFeature.get_range()

意义：获取浮点型数据节点参数范围。

返回值：记录浮点型数据节点参数值范围信息。键包含：min 最小值，max 最大值，inc 步长，unit 单位，inc_is_valid 单位是否有效。

异常处理：

- 1) 如果该浮点型数据节点功能未实现，则打印不支持该浮点型数据节点获取范围的信息，函数返回 None。
- 2) 如果获取该浮点型数据节点范围不成功，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.2.5.get

声明：FloatFeature.get()

意义：读取浮点型数据节点当前值。

返回值：获取的浮点型数据节点当前值。

异常处理：

- 1) 如果该浮点型数据节点未实现或不可读，则打印该浮点型数据节点不可读的信息，函数返回 None。
- 2) 如果获取浮点型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.2.6.set

声明：FloatFeature.set(float_value)

意义：设置浮点型数据节点当前值。

形参：[in] float_value 设置的浮点型数据节点当前值

异常处理：

- 1) 如果输入参数不是浮点型值，则抛出 ParameterTypeError 异常。
- 2) 如果该浮点型数据节点功能未实现或不可写，则打印该浮点型数据节点不可写的信息，函数返回 None。
- 3) 如果输入参数不在该浮点型数据节点的范围内，则打印超过该浮点型数据节点范围的信息并打印范围，函数返回 None。
- 4) 如果设置该浮点型数据节点失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.3. EnumFeature

负责查看、控制相机的枚举型数据节点，继承自 [Feature](#) 类。

接口列表：

is_implemented()	判断枚举型数据节点是否已实现
is_readable()	判断枚举型数据节点是否可读
is_writable()	判断枚举型数据节点是否可写
get_range()	获取枚举型数据节点范围字典
get()	读取枚举型数据节点的当前值和字符串
set(enum_value)	设置枚举型数据节点当前值

3.1.2.3.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.3.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.3.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.3.4.get_range

声明：EnumFeature.get_range()

意义：获取枚举型数据节点参数范围。

返回值：记录枚举型数据节点参数范围。

异常处理：

- 1) 如果该枚举型数据节点功能未实现，则打印不支持该枚举型数据节点获取范围的信息，函数返回 None。
- 2) 如果获取该枚举型数据节点参数范围不成功，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.3.5.get

声明：EnumFeature.get()

意义：读取枚举型数据节点的当前值和字符串。

返回值：

- 1) 枚举型数据节点的当前值。
- 2) 枚举型数据节点的当前值的字符串。

异常处理：

- 1) 如果该枚举型数据节点功能未实现或不可读，则打印该枚举型数据节点不可读的信息，函数返回 None。
- 2) 如果获取枚举型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.3.6.set

声明：EnumFeature.set(enum_value)

意义：设置枚举型数据节点当前值。

形参：[in] enum_value 设置的枚举型当前值

异常处理：

- 1) 如果输入参数不是整型值，则抛出 ParameterTypeError 异常。
- 2) 如果该枚举型数据节点功能未实现或不可写，则打印该枚举型数据节点不可写的信息，函数返回 None。
- 3) 如果输入参数不在枚举型数据节点参数“值”的范围内，则打印超过该枚举型数据节点参数范围的信息并打印范围，函数返回 None。
- 4) 如果设置该枚举型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.4. BoolFeature

负责查看、控制相机的布尔型数据节点，继承自 [Feature](#) 类。

接口列表：

is_implemented()	判断布尔型数据节点是否已实现
is_readable()	判断布尔型数据节点是否可读
is_writable()	判断布尔型数据节点是否可写
get()	读取布尔型数据节点当前值
set()	设置布尔型数据节点当前值

3.1.2.4.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.4.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.4.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.4.4.get

声明： BoolFeature.get()

意义： 读取布尔型数据节点当前值。

返回值： 获取的布尔值。

异常处理：

- 1) 如果该布尔型数据节点功能未实现或不可读，则打印该布尔型数据节点不可读的信息，函数返回 None。
- 2) 如果获取布尔型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.4.5.set

声明： BoolFeature.set(bool_value)

意义： 设置浮点型数据节点当前值。

形参： [in] bool_value 设置的布尔型数值

异常处理：

- 1) 如果输入参数不是布尔型值，则抛出 ParameterTypeError 异常。
- 2) 如果该布尔型数据节点功能未实现或不可写，则打印该布尔型数据节点不可写的信息，函数返回 None。
- 3) 如果设置该布尔型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.5. StringFeature

负责查看、控制相机的字符串型数据节点，继承自 [Feature](#) 类。

接口列表：

is_implemented()	判断字符串型数据节点是否已实现
is_readable()	判断字符串型数据节点是否可读
is_writable()	判断字符串型数据节点是否可写
get_string_max_length()	获取字符串型数据节点值可设置的最长长度
get()	读取字符串型数据节点当前值
set(input_string)	设置字符串型数据节点当前值

3.1.2.5.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.5.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.5.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.5.4.get_string_max_length

声明：StringFeature.get_string_max_length()

意义：获取字符串型数据节点可设置的最大长度。

返回值：字符串型数据节点可设置的最大长度。

异常处理：

- 1) 如果该字符串型数据节点功能未实现，则打印不支持获取该字符串型数据节点的信息，函数返回 None。
- 2) 如果获取字符串型数据节点当前值可设置最大长度失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.5.5.get

声明：StringFeature.get()

意义：读取字符串型数据节点当前值。

返回值：获取的字符串型数据节点当前值。

异常处理：

- 1) 如果该字符串型数据节点功能未实现或不可读，则打印该字符串型数据节点不可读的信息，函数返回 None。
- 2) 如果获取该字符串型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.5.6.set

声明：StringFeature.set(input_string)

意义：设置字符串型数据节点当前值。

形参：[in] input_string 设置的字符串型数据节点当前值

异常处理：

- 1) 如果输入参数不是字符串型值，则抛出 ParameterTypeError 异常。
- 2) 如果该字符串型数据节点功能未实现或不可写，则打印该字符串型数据节点不可写的信息，函数返回 None。
- 3) 如果输入参数长度大于可设置最大长度，则打印超过该字符串型数据节点长度最大值的信息并打印最大值，函数返回 None。
- 4) 如果设置该字符串型数据节点当前值，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.6. BufferFeature

负责查看、控制相机的寄存器型数据节点，继承自 [Feature](#) 类。

接口列表：

is_implemented()	判断寄存器型数据节点是否已实现
is_readable()	判断寄存器型数据节点是否可读
is_writable()	判断寄存器型数据节点是否可写
get_buffer_length()	获取寄存器型数据节点的长度
get_buffer()	读取寄存器型数据节点当前数据
set_buffer(buf)	设置寄存器型数据节点当前数据

3.1.2.6.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.6.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.6.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.6.4.get_buffer_length

声明：BufferFeature.get_buffer_length()

意义：获取型寄存器型数据节点的长度。

返回值：寄存器型数据节点参数的长度。

异常处理：

- 1) 如果该寄存器型数据节点功能未实现，则打印不支持该寄存器型数据节点获取范围的信息，函数返回 None。
- 2) 如果获取寄存器型数据节点参数长度失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.6.5.get_buffer

声明：BufferFeature.get_buffer()

意义：读取寄存器型数据节点当前数据。

返回值：寄存器型数据节点对象。

异常处理：

- 1) 如果该寄存器型数据节点功能未实现或不可读，则打印该寄存器型数据节点不可读的信息，函数返回 None。
- 2) 如果获取该寄存器型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.6.6.set_buffer

声明：BufferFeature.set_buffer(buf)

意义：设置寄存器型数据节点数据。

形参： [in] buffer 设置的寄存器型数据节点当前值

异常处理：

- 1) 如果输入参数不是寄存器类型，则抛出 ParameterTypeError 异常。
- 2) 如果该寄存器型数据节点功能未实现或不可写，则打印该寄存器型数据节点不可写的信息，函数返回 None。
- 3) 如果输入寄存器型数据节点数据长度大于最大长度，则打印超过该寄存器型数据节点最大长度的信息并打印最大值，函数返回 None。
- 4) 如果设置该寄存器型数据节点当前值失败，则抛出异常，异常类型详见[错误处理](#)。

3.1.2.7. CommandFeature

发送命令型数据节点，继承自 [Feature](#) 类。

接口列表：

is_implemented()	判断命令型数据节点是否已实现
is_readable()	判断命令型属数据节点是否可读
is_writable ()	判断命令型数据节点是否可写
send_command()	发送命令

3.1.2.7.1.is_implemented

详见：[Feature::is_implemented\(\)](#)。

3.1.2.7.2.is_readable

详见：[Feature::is_readable\(\)](#)。

3.1.2.7.3.is_writable

详见：[Feature::is_writable\(\)](#)。

3.1.2.7.4.send_command

声明： CommandFeature.send_command()

意义： 发送命令。

异常处理：

- 1) 如果该命令型数据节点功能未实现，则打印不支持该命令型数据节点的信息，函数返回 None。
- 2) 如果发送命令不成功，则抛出异常，异常类型详见[错误处理](#)。

3.2. 数据类型定义

3.2.1. GxLogTypeList

定义	值	解释
GX_LOG_TYPE_OFF	0x00000000	所有类型均发送/不发送
GX_LOG_TYPE_FATAL	0x00000001	FATAL 类型
GX_LOG_TYPE_ERROR	0x00000010	ERROR 类型

GX_LOG_TYPE_WARN	0x00000100	WARN 类型
GX_LOG_TYPE_INFO	0x00001000	INFO 类型
GX_LOG_TYPE_DEBUG	0x00010000	DEBUG 类型
GX_LOG_TYPE_TRACE	0x00100000	TRACE 类型

3.2.2. GxDeviceClassList

定义	值	解释
UNKNOWN	0	未知设备种类
USB2	1	USB2.0 相机
GEV	2	千兆网相机 (GigE Vision)
U3V	3	USB3.0 相机 (USB3 Vision)
SMART	4	网络智能相机
CXP	5	CXP 相机

3.2.3. GxAccessStatus

定义	值	解释
UNKNOWN	0	设备当前状态未知
READWRITE	1	设备当前可读可写
READONLY	2	设备当前仅支持读
NOACCESS	3	设备当前既不支持读，又不支持写

3.2.4. GxAccessMode

定义	值	解释
READONLY	2	以只读方式打开设备
CONTROL	3	以控制方式打开设备
EXCLUSIVE	4	以独占方式打开设备

3.2.5. GxPixelFormatEntry

定义	值	解释
UNDEFINED	0x00000000	-
MONO8	0x01080001	Monochrome 8-bit
MONO8_SIGNED	0x01080002	Monochrome 8-bit signed
MONO10	0x01100003	Monochrome 10-bit unpacked
MONO10_P	0x010A0046	Monochrome 10-bit packed
MONO12	0x01100005	Monochrome 12-bit unpacked
MONO12_P	0x010C0047	Monochrome 12-bit packed
MONO14	0x01100025	Monochrome 14-bit unpacked
MONO14_P	0x010E0104	Monochrome 14-bit packed
MONO16	0x01100007	Monochrome 16-bit
BAYER_GR8	0x01080008	Bayer Green-Red 8-bit
BAYER_RG8	0x01080009	Bayer Red-Green 8-bit

BAYER_GB8	0x0108000A	Bayer Green-Blue 8-bit
BAYER_BG8	0x0108000B	Bayer Blue-Green 8-bit
BAYER_GR10	0x0110000C	Bayer Green-Red 10-bit
BAYER_GR10_P	0x010A0056	Bayer Green-Red 10-bit packed
BAYER_RG10	0x0110000D	Bayer Red-Green 10-bit
BAYER_RG10_P	0x010A0058	Bayer Red-Green 10-bit packed
BAYER_GB10	0x0110000E	Bayer Green-Blue 10-bit
BAYER_GB10_P	0x010A0054	Bayer Green-Blue 10-bit packed
BAYER_BG10	0x0110000F	Bayer Blue-Green 10-bit
BAYER_BG10_P	0x010A0052	Bayer Blue-Green 10-bit packed
BAYER_GR12	0x01100010	Bayer Green-Red 12-bit
BAYER_GR12_P	0x010C0057	Bayer Green-Red 12-bit packed
BAYER_RG12	0x01100011	Bayer Red-Green 12-bit
BAYER_RG12_P	0x010C0059	Bayer Red-Green 12-bit packed
BAYER_GB12	0x01100012	Bayer Green-Blue 12-bit
BAYER_GB12_P	0x010C0055	Bayer Green-Blue 12-bit packed
BAYER_BG12	0x01100013	Bayer Blue-Green 12-bit
BAYER_BG12_P	0x010C0053	Bayer Blue-Green 12-bit packed
BAYER_GR14	0x01100109	Bayer Green-Red 14-bit
BAYER_GR14_P	0x010E0105	Bayer Green-Red 14-bit packed
BAYER_RG14	0x0110010A	Bayer Red-Green 14-bit
BAYER_RG14_P	0x010E0106	Bayer Red-Green 14-bit packed
BAYER_GB14	0x0110010B	Bayer Green-Blue 14-bit
BAYER_GB14_P	0x010E0107	Bayer Green-Blue 14-bit packed
BAYER_BG14	0x0110010C	Bayer Blue-Green 14-bit
BAYER_BG14_P	0x010E0108	Bayer Blue-Green 14-bit packed
BAYER_GR16	0x0110002E	Bayer Green-Red 16-bit
BAYER_RG16	0x0110002F	Bayer Red-Green 16-bit
BAYER_GB16	0x01100030	Bayer Green-Blue 16-bit
BAYER_BG16	0x01100031	Bayer Blue-Green 16-bit
RGB8_PLANAR	0x02180021	Red-Green-Blue 8-bit planar
RGB10_PLANAR	0x02300022	Red-Green-Blue 10-bit unpacked
RGB12_PLANAR	0x02300023	Red-Green-Blue 12-bit unpacked
RGB16_PLANAR	0x02300024	Red-Green-Blue 16-bit planar
RGB8	0x2180014	Red-Green-Blue 8-bit
RGB10	0x2300018	Red-Green-Blue 10-bit
RGB12	0x230001A	Red-Green-Blue 12-bit
RGB14	0x230005E	Red-Green-Blue 14-bit
RGB16	0x2300033	Red-Green-Blue 16-bit

BGR8	0x2180015	Blue-Green-Red 8-bit
BGR10	0x2300019	Blue-Green-Red 10-bit
BGR12	0x230001B	Blue-Green-Red 12-bit
BGR14	0x230004A	Blue-Green-Red 14-bit
BGR16	0x230004B	Blue-Green-Red 16-bit
RGBA8	0x2200016	Red-Green-Blue-Alpha 8-bit
BGRA8	0x2200017	Blue-Green-Red-Alpha 8-bit
ARGB8	0x2200018	Alpha-Red-Green-Blue 8-bit
ABGR8	0x2200019	Alpha-Blue-Green-Red 8-bit
R8	0x010800C9	Red 8-bit
G8	0x010800CD	Green 8-bit
B8	0x010800D1	Blue 8-bit
COORD3D_ABC32F	0x026000C0	3D coordinate A-B-C 32-bit floating point
COORD3D_ABC32F_PLANAR	0x026000C1	3D coordinate A-B-C 32-bit floating point planar
COORD3D_C16	0x011000B8	3D coordinate C 16-bit
COORD3D_C16_I16	0x0110FF02	custom pixel format
COORD3D_C16_S16	0x0110FF03	custom pixel format
COORD3D_C16_I16_S16	0x0110FF04	custom pixel format
YUV444_8	0x2180020	YUV444 8-bit
YUV422_8	0x2100032	YUV422 8-bit
YUV411_8	0x20C001E	YUV411 8-bit
YUV420_8_PLANAR	0x20C0040	YUV420 8-bit planar
YCBCR444_8	0x218005B	YCBCR 444 8-bit
YCBCR422_8	0x210003B	YCBCR 422 8-bit
YCBCR411_8	0x20C005A	YCBCR 411 8-bit
YCBCR601_444_8	0x218003D	YCBCR601 444 8-bit
YCBCR601_422_8	0x210003E	YCBCR601 422 8-bit
YCBCR601_411_8	0x20C003F	YCBCR601 411 8-bit
YCBCR709_444_8	0x2180040	YCBCR709 444 8-bit
YCBCR709_422_8	0x2100041	YCBCR709 422 8-bit
YCBCR709_411_8	0x20C0042	YCBCR709 411 8-bit
MONO10_PACKED	0x010C0004	GigE Vision specific format, Mono 10-bit packed
MONO12_PACKED	0x010C0006	GigE Vision specific format, Mono 12-bit packed
BAYER_BG10_PACKED	0x010C0029	GigE Vision specific format, Bayer Blue-Green 10-bit packed
BAYER_BG12_PACKED	0x010C002D	GigE Vision specific format, Bayer Blue-Green 12-bit packed
BAYER_GB10_PACKED	0x010C0028	GigE Vision specific format, Bayer Green-Blue 10-bit packed
BAYER_GB12_PACKED	0x010C002C	GigE Vision specific format, Bayer Green-Blue 12-bit packed

BAYER_GR10_PACKED	0x010C0026	GigE Vision specific format, Bayer Green-Red 10-bit packed
BAYER_GR12_PACKED	0x010C002A	GigE Vision specific format, Bayer Green-Red 12-bit packed
BAYER_RG10_PACKED	0x010C0027	GigE Vision specific format, Bayer Red-Green 10-bit packed
BAYER_RG12_PACKED	0x010C002B	GigE Vision specific format, Bayer Red-Green 12-bit packed

3.2.6. GxFrameStatusList

定义	值	解释
SUCCESS	0	正常帧
IMCOMPLETE	-1	残帧
INVALID_IMAGE_INFO	-2	图像信息无效

3.2.7. GxDeviceTemperatureSelectorEntry

定义	值	解释
SENSOR	1	传感器温度
MAINBOARD	2	主板温度

3.2.8. GxPixelFormatEntry

定义	值	解释
BPP8	8	像素大小 BPP8
BPP10	10	像素大小 BPP10
BPP12	12	像素大小 BPP12
BPP16	16	像素大小 BPP16
BPP24	24	像素大小 BPP24
BPP30	30	像素大小 BPP30
BPP32	32	像素大小 BPP32
BPP36	36	像素大小 BPP36
BPP48	48	像素大小 BPP48
BPP64	64	像素大小 BPP64

3.2.9. GxPixelFormatColorFilterEntry

定义	值	解释
NONE	0	无
BAYER_RG	1	RG 格式
BAYER_GB	2	GB 格式
BAYER_GR	3	GR 格式
BAYER_BG	4	BG 格式

3.2.10. GxAcquisitionModeEntry

定义	值	解释
SINGLE_FRAME	0	单帧模式
MULTI_FRAME	1	多帧模式
CONTINUOUS	2	连续模式

3.2.11. GxTriggerSourceEntry

定义	值	解释
SOFTWARE	0	软触发
LINE0	1	触发源 0
LINE1	2	触发源 1
LINE2	3	触发源 2
LINE3	4	触发源 3
COUNTER2END	5	COUNTER2END 触发信号
TRIGGER	6	触发信号
MULTISOURCE	7	多源触发
CXPTRIGGER0	8	CXP 触发源 0
CXPTRIGGER1	9	CXP 触发源 1

3.2.12. GxTriggerActivationEntry

定义	值	解释
FALLING_EDGE	0	下降沿触发
RISING_EDGE	1	上升沿触发
ANYEDGE	2	上升或下降沿触发
LEVELHIGH	3	高电平触发
LEVELLOW	4	低电平触发

3.2.13. GxExposureModeEntry

定义	值	解释
TIMED	1	曝光时间寄存器控制曝光时间
TRIGGER_WIDTH	2	触发信号宽度控制曝光时间

3.2.14. GxUserOutputSelectorEntry

定义	值	解释
OUTPUT0	1	输出 0
OUTPUT1	2	输出 1
OUTPUT2	4	输出 2
OUTPUT3	5	输出 3

OUTPUT4	6	输出 4
OUTPUT5	7	输出 5
OUTPUT6	8	输出 6

3.2.15. GxUserOutputModeEntry

定义	值	解释
STROBE	0	闪光灯
USER_DEFINED	1	用户自定义

3.2.16. GxGainSelectorEntry

定义	值	解释
ALL	0	所有增益通道
RED	1	红通道增益
GREEN	2	绿通道增益
BLUE	3	蓝通道增益

3.2.17. GxBlackLevelSelectEntry

定义	值	解释
ALL	0	所有黑电平通道
RED	1	红通道黑电平
GREEN	2	绿通道黑电平
BLUE	3	蓝通道黑电平

3.2.18. GxBalanceRatioSelectorEntry

定义	值	解释
RED	0	红通道
GREEN	1	绿通道
BLUE	2	蓝通道

3.2.19. GxAALightEnvironmentEntry

定义	值	解释
NATURE_LIGHT	0	自然光
AC50HZ	1	50 赫兹日光灯
AC60HZ	2	60 赫兹日光灯

3.2.20. GxUserSetEntry

定义	值	解释
DEFAULT	0	默认参数组
USER_SET0	1	用户参数组 0
USER_SET1	2	用户参数组 1

3.2.21. GxAWBLampHouseEntry

定义	值	解释
ADAPTIVE	0	自适应光源
D65	1	指定色温 6500k
FLUORESCENCE	2	指定荧光灯
INCANDESCENT	3	指定白炽灯
D75	4	指定色温 7500k
D50	5	指定色温 5000k
U30	6	指定色温 3000k

3.2.22. GxUserDataFieldSelectorEntry

定义	值	解释
FIELD_0	0	Flash 数据区域 0
FIELD_1	1	Flash 数据区域 1
FIELD_2	2	Flash 数据区域 2
FIELD_3	3	Flash 数据区域 3

3.2.23. GxTestPatternEntry

定义	值	解释
OFF	0	关闭
GRAY_FRAME_RAMP_MOVING	1	静止灰度递增
SLANT_LINE_MOVING	2	滚动斜条纹
VERTICAL_LINE_MOVING	3	滚动竖条纹
HORIZONTAL_LINE_MOVING	4	滚动横条纹
GREY_VERTICAL_RAMP	5	垂直灰度递增
SLANT_LINE	6	静止斜条纹

3.2.24. GxTriggerSelectorEntry

定义	值	解释
FRAME_START	1	采集一帧
FRAME_BURST_START	2	帧高速连拍开始

3.2.25. GxLineSelectorEntry

定义	值	解释
LINE0	0	引脚 0
LINE1	1	引脚 1
LINE2	2	引脚 2
LINE3	3	引脚 3

LINE4	4	引脚 4
LINE5	5	引脚 5
LINE6	6	引脚 6
LINE7	7	引脚 7
LINE8	8	引脚 8
LINE9	9	引脚 9
LINE10	10	引脚 10
LINE_STROBE	11	专用闪光灯引脚
LINE11	12	引脚 11
LINE12	13	引脚 12
LINE13	14	引脚 13
LINE14	15	引脚 14
TRIGGER	16	硬件触发输入
IO1	17	GPIO 输入
IO2	18	GPIO 输入
FLASH_P	19	闪光灯 flash_B 输出
FLASH_W	20	闪光灯 flash_W 输出

3.2.26. GxDeviceSerialPortBaudRateEntry

定义	值	解释
Baud9600	5	串口波特率为 9600Hz
Baud19200	6	串口波特率为 19200Hz
Baud38400	7	串口波特率为 38400Hz
Baud76800	8	串口波特率为 76800Hz
Baud115200	9	串口波特率为 115200Hz

3.2.27. GxSerialPortStopBitsEntry

定义	值	解释
Bits1	0	Bit1
Bits1AndAHalf	1	Bit1AndHalf
Bits2	2	Bit2

3.2.28. GxLineModeEntry

定义	值	解释
INPUT	0	输入
OUTPUT	1	输出

3.2.29. GxLineSourceEntry

定义	值	解释
OFF	0	关闭
STROBE	1	闪光灯
USER_OUTPUT0	2	用户自定义输出 0
USER_OUTPUT1	3	用户自定义输出 1
USER_OUTPUT2	4	用户自定义输出 2
EXPOSURE_ACTIVE	5	曝光有效
FRAME_TRIGGER_WAIT	6	单帧触发等待
ACQUISITION_TRIGGER_WAIT	7	多帧触发等待
TIMER1_ACTIVE	8	定时器 1 有效
USER_OUTPUT3	9	用户自定义输出 3
USER_OUTPUT4	10	用户自定义输出 4
USER_OUTPUT5	11	用户自定义输出 5
USER_OUTPUT6	12	用户自定义输出 6
TIMER2_ACTIVE	13	计时器 2 激活
TIMER3_ACTIVE	14	计时器 3 激活
FRAME_TRIGGER	15	帧触发
Flash_W	16	Flash_w
Flash_P	17	Flash_P
SERIAL_PORT_0	18	SerialPort0

3.2.30. GxEventSelectorEntry

定义	值	解释
EXPOSURE_END	0x0004	曝光结束
BLOCK_DISCARD	0x9000	图像帧丢弃
EVENT_OVERRUN	0x9001	事件队列溢出
FRAME_START_OVER_TRIGGER	0x9002	触发信号溢出
BLOCK_NOT_EMPTY	0x9003	图像帧存不为空
INTERNAL_ERROR	0x9004	内部错误事件
FRAME_BURST_START_OVERTRIGGER	0x9005	多帧触发屏蔽事件
FRAME_START_WAIT	0x9006	帧等待事件
FRAME_BURST_START_WAIT	0x9007	多帧等待事件

3.2.31. GxLutSelectorEntry

定义	值	解释
LUMINANCE	0	亮度

3.2.32. GxTransferControlModeEntry

定义	值	解释
BASIC	0	基础模式
USER_CONTROLLED	1	用户控制模式

3.2.33. GxTransferOperationModeEntry

定义	值	解释
MULTI_BLOCK	0	指定发送帧数

3.2.34. GxTestPatternGeneratorSelectorEntry

定义	值	解释
SENSOR	0	sensor 的测试图
REGION0	1	FPGA 的测试图

3.2.35. GxChunkSelectorEntry

定义	值	解释
FRAME_ID	1	帧号
TIME_STAMP	2	时间戳
COUNTER_VALUE	3	计数器值

3.2.36. GxBinningHorizontalModeEntry

定义	值	解释
SUM	0	BINNING 水平值和
AVERAGE	1	BINNING 水平值平均值

3.2.37. GxBinningVerticalModeEntry

定义	值	解释
SUM	0	BINNING 垂直值和
AVERAGE	1	BINNING 垂直值平均值

3.2.38. GxSensorShutterModeEntry

定义	值	解释
GLOBAL	0	所有的像素同时曝光且曝光时间相等
ROLLING	1	所有的像素曝光时间相等，但曝光起始时间不同
GLOBALRESET	2	所有的像素曝光起始时间相同，但曝光时间不相等

3.2.39. GxAcquisitionStatusSelectorEntry

定义	值	解释
ACQUISITION_TRIGGER_WAIT	0	采集触发等待
FRAME_TRIGGER_WAIT	1	帧触发等待

3.2.40. GxExposureTimeModeEntry

定义	值	解释
ULTRASHORT	0	极小曝光
STANDARD	1	标准

3.2.41. GxGammaModeEntry

定义	值	解释
SRGB	0	默认 Gamma 校正
USER	1	用户自定义 Gamma 校正

3.2.42. GxLightSourcePresetEntry

定义	值	解释
OFF	0	关闭
CUSTOM	1	用户自定义
DAYLIGHT_6500K	2	标准日光灯 (6500K)
DAYLIGHT_5000K	3	标准日光灯 (5000K)
COOL_WHITE_FLUORESCENCE	4	冷白光源 (4150K)
INCA	5	螺旋钨丝灯 (2856K)

3.2.43. GxColorTransformationModeEntry

定义	值	解释
RGB_TO_RGB	0	默认颜色校正
USER	1	用户自定义颜色校正

3.2.44. GxColorTransformationValueSelectorEntry

定义	值	解释
GAIN00	0	颜色转换分量增益值 GAIN00
GAIN01	1	颜色转换分量增益值 GAIN01
GAIN02	2	颜色转换分量增益值 GAIN02
GAIN10	3	颜色转换分量增益值 GAIN10
GAIN11	4	颜色转换分量增益值 GAIN11
GAIN12	5	颜色转换分量增益值 GAIN12
GAIN20	6	颜色转换分量增益值 GAIN20
GAIN21	7	颜色转换分量增益值 GAIN21
GAIN22	8	颜色转换分量增益值 GAIN22

3.2.45. GxAutoEntry

定义	值	解释
OFF	0	关闭
CONTINUOUS	1	连续
ONCE	2	单次

3.2.46. GxSwitchEntry

定义	值	解释
OFF	0	关闭
ON	1	开启

3.2.47. GxSensorBitDepthEntry

定义	值	解释
BPP8	8	位深 8
BPP10	10	位深 10
BPP12	12	位深 12

3.2.48. GxMultisourceSelectorEntry

定义	值	解释
Software	0	软触发
LINE0	1	多源触发选择 0
LINE2	3	多源触发选择 2
LINE3	4	多源触发选择 3

3.2.49. GxDeviceTapGeometryEntry

定义	值	解释
GEOMETRY_1X_1Y	0	Geometry_1X_1Y
GEOMETRY1_X1_Y2	19	Geometry_1X_1Y2
GEOMETRY1_X2_YE	10	Geometry_1X_2YE

3.2.50. GxEncoderSelectorEntry

定义	值	解释
ENCODER0	0	编码器选择器 0
ENCODER1	1	编码器选择器 1
ENCODER2	2	编码器选择器 2

3.2.51. GxEncoderSourceAEntry

定义	值	解释
OFF	0	编码器 A 相关闭输入
LINE0	1	编码器 A 相输入 Line0
LINE1	2	编码器 A 相输入 Line1
LINE2	3	编码器 A 相输入 Line2

LINE3	4	编码器 A 相输入 Line3
LINE4	5	编码器 A 相输入 Line4
LINE5	6	编码器 A 相输入 Line5

3.2.52. GxEncoderSourceBEntry

定义	值	解释
OFF	0	编码器 B 相关闭输入
LINE0	1	编码器 B 相输入 Line0
LINE1	2	编码器 B 相输入 Line1
LINE2	3	编码器 B 相输入 Line2
LINE3	4	编码器 B 相输入 Line3
LINE4	5	编码器 B 相输入 Line4
LINE5	6	编码器 B 相输入 Line5

3.2.53. GxEncoderModeEntry

定义	值	解释
HIGH_RESOLUTION	0	编码器模式

3.2.54. GxEncoderDirectionEntry

定义	值	解释
FORWARD	0	编码器方向向前
BACKWARD	1	编码器方向向后

3.2.55. GxRegionSendModeEntry

定义	值	解释
SINGLE_ROI	0	单 ROI
MULTI_ROI	1	多 ROI

3.2.56. GxShadingCorrectionModeEntry

定义	值	解释
FLAT_FIELD_CORRECTION	0	平场校正
PARALLAX_CORRECTION	1	视差校正
TAILOR_FLAT_FIELD_CORRECTION	2	定制平场校正
DEVICE_FLAT_FIELD_CORRECTION	3	设备平场校正

3.2.57. GxFFCGenerateStatusEntry

定义	值	解释
IDLE	0	闲置
WAITING_IMAGE	1	等待图像
FINISH	2	完成

3.2.58. GxFFCCoefficientEntry

定义	值	解释
SET0	0	平场校正系数 Set0
SET1	1	平场校正系数 Set1
SET2	2	平场校正系数 Set2
SET3	3	平场校正系数 Set3
SET4	4	平场校正系数 Set4
SET5	5	平场校正系数 Set5
SET6	6	平场校正系数 Set6
SET7	7	平场校正系数 Set7
SET8	8	平场校正系数 Set8
SET9	9	平场校正系数 Set9
SET10	10	平场校正系数 Set10
SET11	11	平场校正系数 Set11
SET12	12	平场校正系数 Set12
SET13	13	平场校正系数 Set13
SET14	14	平场校正系数 Set14
SET15	15	平场校正系数 Set15

3.2.59. GxDSNUSelectorEntry

定义	值	解释
DEFAULT	0	暗场校正系数 Default
SET0	1	暗场校正系数 Set0
SET1	2	暗场校正系数 Set1
SET2	3	暗场校正系数 Set2
SET3	4	暗场校正系数 Set3
SET4	5	暗场校正系数 Set4
SET5	6	暗场校正系数 Set5
SET6	7	暗场校正系数 Set6
SET7	8	暗场校正系数 Set7
SET8	9	暗场校正系数 Set8
SET9	10	暗场校正系数 Set9
SET10	11	暗场校正系数 Set10
SET11	12	暗场校正系数 Set11
SET12	13	暗场校正系数 Set12
SET13	14	暗场校正系数 Set13
SET14	15	暗场校正系数 Set14
SET15	16	暗场校正系数 Set15

3.2.60. GxDSNUGenerateStatusEntry

定义	值	解释
IDLE	0	闲置
WAITING_IMAGE	1	等待图像
FINISH	2	完成

3.2.61. GxPRNUSelectorEntry

定义	值	解释
DEFAULT	0	明场校正系数 Default
SET0	1	明场校正系数 Set0
SET1	2	明场校正系数 Set1
SET2	3	明场校正系数 Set2
SET3	4	明场校正系数 Set3
SET4	5	明场校正系数 Set4
SET5	6	明场校正系数 Set5
SET6	7	明场校正系数 Set6
SET7	8	明场校正系数 Set7
SET8	9	明场校正系数 Set8
SET9	10	明场校正系数 Set9
SET10	11	明场校正系数 Set10
SET11	12	明场校正系数 Set11
SET12	13	明场校正系数 Set12
SET13	14	明场校正系数 Set13
SET14	15	明场校正系数 Set14
SET15	16	明场校正系数 Set15

3.2.62. GxPRNUGenerateStatusEntry

定义	值	解释
IDLE	0	闲置
WAITING_IMAGE	1	等待图像
FINISH	2	完成

3.2.63. GxCXPLinkConfigurationEntry

定义	值	解释
CXP6_X1	0x00010048	CXP 连接配置 CXP6_X1
CXP12_X1	0x00010058	CXP 连接配置 CXP12_X1
CXP6_X2	0x00020048	CXP 连接配置 CXP6_X2

CXP12_X2	0x00020058	CXP 连接配置 CXP12_X2
CXP6_X4	0x00040048	CXP 连接配置 CXP6_X4
CXP12_X4	0x00040058	CXP 连接配置 CXP12_X4
CXP3_X1	0x00010038	CXP 连接配置 CXP3_X1
CXP3_X2	0x00020038	CXP 连接配置 CXP3_X2
CXP3_X4	0x00040038	CXP 连接配置 CXP3_X4

3.2.64. GxCXPLinkConfigurationPreferredEntry

定义	值	解释
CXP12_X4	0x00040058	预设连接配置 CXP12_X4

3.2.65. GxCXPLinkConfigurationStatusEntry

定义	值	解释
CXP6_X1	0x00010048	CXP 连接配置状态 CXP6_X1
CXP12_X1	0x00010058	CXP 连接配置状态 CXP12_X1
CXP6_X2	0x00020048	CXP 连接配置状态 CXP6_X2
CXP12_X2	0x00020058	CXP 连接配置状态 CXP12_X2
CXP6_X4	0x00040048	CXP 连接配置状态 CXP6_X4
CXP12_X4	0x00040058	CXP 连接配置状态 CXP12_X4
CXP3_X1	0x00010038	CXP 连接配置状态 CXP3_X1
CXP3_X2	0x00020038	CXP 连接配置状态 CXP3_X2
CXP3_X4	0x00040038	CXP 连接配置状态 CXP3_X4

3.2.66. GxCXPConectionSelectorEntry

定义	值	解释
SELECTOR0	0	连接选择 0
SELECTOR1	1	连接选择 1
SELECTOR2	2	连接选择 2
SELECTOR3	3	连接选择 3

3.2.67. GxCXPConectionTestModeEntry

定义	值	解释
OFF	0	关闭连接测试模式
MODE1	1	触发连接测试模式

3.2.68. GxSequencerFratureSelectorEntry

定义	值	解释
FLAT_FIELD_CORRECTION	0	序列功能选择

3.2.69. GxSequencerTriggerSourceEntry

定义	值	解释
FRAME_START	7	序列触发源 FrameStart

3.2.70. GxRegionSelectorEntry

定义	值	解释
REGION0	0	区域 0
REGION1	1	区域 1
REGION2	2	区域 2
REGION3	3	区域 3
REGION4	4	区域 4
REGION5	5	区域 5
REGION6	6	区域 6
REGION7	7	区域 7

3.2.71. GxTimerSelectorEntry

定义	值	解释
TIMER1	1	定时器 1
TIMER2	2	定时器 2
TIMER3	3	定时器 3

3.2.72. GxTimerTriggerSourceEntry

定义	值	解释
EXPOSURE_START	1	曝光开始信号开始计时
LINE10	10	接收引脚 10 信号开始计时
LINE14	14	接收引脚 14 信号开始计时
STROBE	16	接收闪光灯信号开始计时

3.2.73. GxNoiseReductionModeEntry

定义	值	解释
OFF	0	关闭 2D 降噪模式
LOW	1	低
MIDDLE	2	中
HIGH	3	高

3.2.74. GxHDRModeEntry

定义	值	解释
OFF	0	关闭 HDR 模式
CONTINUOUS	1	连续 HDR 模式

3.2.75. GxMGCMODEEntry

定义	值	解释
OFF	0	关闭多帧灰度控制模式
TWO_FRAME	1	两帧灰度控制模式
FOUR_FRAME	2	四帧灰度控制模式

3.2.76. GxAcquisitionBurstModeEntry

定义	值	解释
STANDARD	0	标准模式
HIGH_SPEED	1	高速模式

3.2.77. GxSensorSelectorEntry

定义	值	解释
CMOS1	0	选择 CMOS1 传感器
CCD1	1	选择 CCD1 传感器

3.2.78. GxIMUConfigAccRangeEntry

定义	值	解释
ACC_16G	2	加速计测量范围为 16g
ACC_8G	3	加速计测量范围为 8g
ACC_4G	4	加速计测量范围为 4g
ACC_2G	5	加速计测量范围为 2g

3.2.79. GxIMUConfigAccOdrEntry

定义	值	解释
ODR_1000HZ	0	加速计输出数据率为 1000Hz
ODR_500HZ	1	加速计输出数据率为 500Hz
ODR_250HZ	2	加速计输出数据率为 250Hz
ODR_125HZ	3	加速计输出数据率为 125Hz
ODR_63HZ	4	加速计输出数据率为 63Hz
ODR_31HZ	5	加速计输出数据率为 31Hz
ODR_16HZ	6	加速计输出数据率为 16Hz
ODR_2000HZ	8	加速计输出数据率为 2000Hz
ODR_4000HZ	9	加速计输出数据率为 4000Hz
ODR_8000HZ	10	加速计输出数据率为 8000Hz

3.2.80. GxIMUConfigAccOdrLowPassFilterFrequencyEntry

定义	值	解释
ODR040	0	加速计加速计低通截止频率为 $ODR \times 0.40$
ODR025	1	加速计加速计低通截止频率为 $ODR \times 0.25$
ODR011	2	加速计加速计低通截止频率为 $ODR \times 0.11$
ODR004	3	加速计加速计低通截止频率为 $ODR \times 0.04$
ODR002	4	加速计加速计低通截止频率为 $ODR \times 0.02$

3.2.81. GxIMUConfigGyroRangeEntry

定义	值	解释
RANGE_125DPS	2	陀螺仪 X 方向测量范围为 125dps
RANGE_250DPS	3	陀螺仪 X 方向测量范围为 250dps
RANGE_500DPS	4	陀螺仪 X 方向测量范围为 500dps
RANGE_1000DPS	5	陀螺仪 X 方向测量范围为 1000dps
RANGE_2000DPS	6	陀螺仪 X 方向测量范围为 2000dps

3.2.82. GxIMUConfigGyroOdrEntry

定义	值	解释
ODR_1000HZ	0	加速计输出数据率为 1000Hz
ODR_500HZ	1	加速计输出数据率为 500Hz
ODR_250HZ	2	加速计输出数据率为 250Hz
ODR_125HZ	3	加速计输出数据率为 125Hz
ODR_63HZ	4	加速计输出数据率为 63Hz
ODR_31HZ	5	加速计输出数据率为 31Hz
ODR_4KHZ	9	加速计输出数据率为 4KHz
ODR_8KHZ	10	加速计输出数据率为 8KHz
ODR_16KHZ	11	加速计输出数据率为 16KHz
ODR_32KHZ	12	加速计输出数据率为 32KHz

3.2.83. GxIMUConfigGyroOdrLowPassFilterFrequencyEntry

定义	值	解释
GYROLPF2000HZ	2000	加速计加速计低通截止频率为 2000Hz
GYROLPF1600HZ	1600	加速计加速计低通截止频率为 1600Hz
GYROLPF1525HZ	1525	加速计加速计低通截止频率为 1525Hz
GYROLPF1313HZ	1313	加速计加速计低通截止频率为 1313Hz
GYROLPF1138HZ	1138	加速计加速计低通截止频率为 1138Hz
GYROLPF1000HZ	1000	加速计加速计低通截止频率为 1000Hz
GYROLPF863HZ	863	加速计加速计低通截止频率为 863Hz
GYROLPF638HZ	638	加速计加速计低通截止频率为 638Hz

GYROLPF438HZ	438	加速计加速计低通截止频率为 438Hz
GYROLPF313HZ	313	加速计加速计低通截止频率为 313Hz
GYROLPF213HZ	213	加速计加速计低通截止频率为 213Hz
GYROLPF219HZ	219	加速计加速计低通截止频率为 219Hz
GYROLPF363HZ	363	加速计加速计低通截止频率为 363Hz
GYROLPF320HZ	320	加速计加速计低通截止频率为 320Hz
GYROLPF250HZ	250	加速计加速计低通截止频率为 250Hz
GYROLPF200HZ	200	加速计加速计低通截止频率为 200Hz
GYROLPF181HZ	181	加速计加速计低通截止频率为 181Hz
GYROLPF160HZ	160	加速计加速计低通截止频率为 160Hz
GYROLPF125HZ	125	加速计加速计低通截止频率为 125Hz
GYROLPF100HZ	100	加速计加速计低通截止频率为 100Hz
GYROLPF90HZ	90	加速计加速计低通截止频率为 90Hz
GYROLPF80HZ	80	加速计加速计低通截止频率为 80Hz
GYROLPF63HZ	63	加速计加速计低通截止频率为 63Hz
GYROLPF50HZ	50	加速计加速计低通截止频率为 50Hz
GYROLPF45HZ	45	加速计加速计低通截止频率为 45Hz
GYROLPF40HZ	40	加速计加速计低通截止频率为 40Hz
GYROLPF31HZ	31	加速计加速计低通截止频率为 31Hz
GYROLPF25HZ	25	加速计加速计低通截止频率为 25Hz
GYROLPF23HZ	23	加速计加速计低通截止频率为 23Hz
GYROLPF20HZ	20	加速计加速计低通截止频率为 20Hz
GYROLPF15HZ	15	加速计加速计低通截止频率为 15Hz
GYROLPF13HZ	13	加速计加速计低通截止频率为 13Hz
GYROLPF11HZ	11	加速计加速计低通截止频率为 11Hz
GYROLPF10HZ	10	加速计加速计低通截止频率为 10Hz
GYROLPF8HZ	8	加速计加速计低通截止频率为 8Hz
GYROLPF6HZ	6	加速计加速计低通截止频率为 6Hz

3.2.84. GxIMUTemperatureOdrEntry

定义	值	解释
ODR_500HZ	0	温度计输出数据率为 500Hz
ODR_250HZ	1	温度计输出数据率为 250Hz
ODR_125HZ	2	温度计输出数据率为 125Hz
ODR_63HZ	3	温度计输出数据率为 63Hz

3.2.85. GxSerialportSelectorEntry

定义	值	解释
SERIALPOR0	0	串口 0

3.2.86. GxSerialportSourceEntry

定义	值	解释
OFF	0	串口输入源开关
LINE0	1	串口输入源 0
LINE1	2	串口输入源 1
LINE2	3	串口输入源 2
LINE3	4	串口输入源 3

3.2.87. GxSerialportBaundrateEntry

定义	值	解释
BAUNDRATE_9600	5	串口波特率为 9600Hz
BAUNDRATE_19200	6	串口波特率为 19200Hz
BAUNDRATE_38400	7	串口波特率为 38400Hz
BAUNDRATE_76800	8	串口波特率为 76800Hz
BAUNDRATE_115200	9	串口波特率为 115200Hz

3.2.88. GxSerialporeStopBitsEntry

定义	值	解释
ONE	0	Bit1
ONEANDHALF	1	Bit1AndHalf
TWO	2	Bit2

3.2.89. GxSerialportParityEntry

定义	值	解释
NONE	0	None
ODD	1	奇数
EVEN	2	偶数
MARK	3	标记
SPACE	4	空白

3.2.90. GxCounterSelectorEntry

定义	值	解释
COUNTER1	1	计数器 1
COUNTER2	2	计数器 2

3.2.91. GxCounterEventSourceEntry

定义	值	解释
FRAME_START	1	统计 "帧开始" 事件的数量
FRAME_TRIGGER	2	统计 "帧触发" 事件的数量
ACQUISITION_TRIGGER	3	统计 "采集触发" 事件的数量
OFF	4	关闭
SOFTWARE	5	统计 "软触发" 事件的数量
LINE0	6	统计 "Line 0 触发" 事件的数量
LINE1	7	统计 "Line 1 触发" 事件的数量
LINE2	8	统计 "Line 2 触发" 事件的数量
LINE3	9	统计 "Line 3 触发" 事件的数量

3.2.92. GxCounterResetSourceEntry

定义	值	解释
OFF	0	无复位源
SOFTWARE	1	软触发
LINE0	2	引脚 0
LINE1	3	引脚 1
LINE2	4	引脚 2
LINE3	5	引脚 3
COUNTER2END	6	Counter2End
CXPTRIGGER0	8	CxpTrigger0
CXPTRIGGER1	9	CxpTrigger1

3.2.93. GxCounterResetActivationEntry

定义	值	解释
RISINGEDGE	1	上升沿触发

3.2.94. GxCounterTriggerSourceEntry

定义	值	解释
OFF	0	无触发源
SOFTWARE	1	软触发
LINE0	2	引脚 0
LINE1	3	引脚 1
LINE2	4	引脚 2
LINE3	5	引脚 3

3.2.95. GxTimerTriggerActivationEntry

定义	值	解释
RISINGEDGE	1	上升沿触发

3.2.96. GxStopAcquisitionModeEntry

定义	值	解释
GENERAL	0	普通停采
LIGHT	1	轻量级停采

3.2.97. GxDSSStreamBufferHandlingModeEntry

定义	值	解释
OLDEST_FIRST	1	OldestFirst 模式
OLDEST_FIRST_OVERWRITE	2	OldestFirstOverwrite 模式
NEWEST_ONLY	3	NewestOnly 模式

3.2.98. GxResetDeviceModeEntry

定义	值	解释
RESET	2	复位设备

3.2.99. Dx BayerConvertType

定义	值	解释
NEIGHBOUR	0	邻域平均插值算法
ADAPTIVE	1	边缘自适应插值算法
NEIGHBOUR3	2	更大区域的邻域平均插值算法

3.2.100. DxValidBit

定义	值	解释
BIT0_7	0	0-7 位
BIT1_8	1	1-8 位
BIT2_9	2	2-9 位
BIT3_10	3	3-10 位
BIT4_11	4	4-11 位
BIT5_12	5	5-12 位
BIT6_13	6	6-13 位
BIT7_14	7	7-14 位
BIT8_15	8	8-15 位

3.2.101. DxActualBits

定义	值	解释
BITS_8	8	8 位
BITS_10	10	10 位
BITS_12	12	12 位
BITS_14	14	14 位
BITS_16	16	16 位

3.2.102. DxImageMirrorMode

定义	值	解释
HORIZONTAL_MIRROR	0	水平镜像
VERTICAL_MIRROR	1	垂直镜像

3.2.103. DxRGBChannelOrder

定义	值	解释
ORDER_RGB	0	RGB 通道
ORDER_BGR	1	BGR 通道

3.2.104. GxTLClassList

定义	值	解释
TL_TYPE_UNKNOWN	0	未知 TL 类型
TL_TYPE_USB	1	USB TL 类型
TL_TYPE_GEV	2	GEV TL 类型
TL_TYPE_U3V	4	U3V TL 类型
TL_TYPE_CXP	8	CXP TL 类型

3.2.105. GxImageInfo

定义	解释
image_width	图像宽
image_height	图像高
image_buf	图像 buffer
image_pixel_format	图像像素格式

3.2.106. GxIPConfigureModeList

定义	解释
DHCP	启用 DHCP 通过 DHCP 服务分配 IP
LLA	启用 LLA 模式分配 IP
STATIC_IP	启用静态 IP 模式配置 IP 地址
DEFAULT	启用默认模式配置 IP 地址

3.2.107. GxActionCommandResult

定义	解释
device_ip	当前返回 ack 的设备 IP (点分 10 进制的 IPv4)
status	设备返回的 ACTION 状态，错误来源于 GigeVision 协议 0：标识命令发送成功； 0x8013：设备未与主时钟进行时间同步。在执行该接口前，必须启用“ PtpEnable” ，并确保 “PtpStatus” 为 “Master” 或者 “Slave” （表明已经与主时钟同步）； 0x8015：设备队列或数据包数据已溢出。strDeviceAddress 对应的设备正在执行上一个 issue_scheduled_action_command 请求时，再次收到新的 issue_scheduled_action_command 请求，会返回此错误； 0x8016：issue_scheduled_action_command 发出的 Scheduled Action Command 已过时。

3.2.108. GxRegisterStackEntry

定义	解释
address	寄存器地址
buffer	寄存器内存指针
size	寄存器内存大小

3.2.109. ColorTransformFactor

定义	默认值	范围	解释
fGain00	1.0	-4.0~4.0	作用于红色像素的红色通道增益
fGain01	0.0	-4.0~4.0	作用于红色像素的绿色通道增益
fGain02	0.0	-4.0~4.0	作用于红色像素的蓝色通道增益
fGain10	0.0	-4.0~4.0	作用于绿色像素的红色通道增益
fGain11	1.0	-4.0~4.0	作用于绿色像素的绿色通道增益
fGain12	0.0	-4.0~4.0	作用于绿色像素的蓝色通道增益
fGain20	0.0	-4.0~4.0	作用于蓝色像素的红色通道增益
fGain21	0.0	-4.0~4.0	作用于蓝色像素的绿色通道增益
fGain22	1.0	-4.0~4.0	作用于蓝色像素的蓝色通道增益

3.2.110. FlatFieldCorrectionParameter

定义	解释
bright_buf	采集的亮场图像，可通过 DataStream().dq_buf 接口获取
dark_buf	采集的暗场图像，可通过 DataStream().dq_buf 获取，当不需要采集暗场时应将该参数设置为 None。

pixel_format	当前图像像素格式，可通过图像对象中 RawImage().get_pixel_format 接口获取。
width	当前图像的宽度，可通过图像对象中的 RawImage().get_width 接口获取。
height	当前图像的高度，可通过图像对象中的 RawImage().get_height 接口获取。
block_size	平场校正精度，可通过枚举“FFCBlockSize”调用 get 接口获取。
expected_gray	平场校正期望灰度值，可选范围为-1~255，当设置为-1时表示使用当前图像块儿灰度值的最大值。

3.2.111. GxWindowsID

定义	值	解释
WINDOW_PROPERTY	0	显示窗口类型为属性树

3.2.112. GxWindowsShowMode

定义	值	解释
NON_BLOCK_SHOW_MODE	0	以非阻塞模式显示属性窗口，是默认行为。GUI 程序中无需设置。
BLOCK_SHOW_MODE	1	以阻塞模式显示属性窗口，不是模态行为，设置该模式后程序阻塞在 show_window 接口。该模式仅适用与在控制台方式下设置。

3.2.113. GxCFAMethod

定义	值	解释
GX_CFA_METHOD_QUICK	0	快速
GX_CFA_METHOD_BALANCE	1	均衡
GX_CFA_METHOD_OPTIMAL	2	最优

3.2.114. GxImageFormatType

定义	值	解释
GX_IMAGE_FORMAT_JPEG	0	JPEG 格式
GX_IMAGE_FORMAT_PNG	1	PNG 格式
GX_IMAGE_FORMAT_TIFF	2	TIFF 格式
GX_IMAGE_FORMAT_RAW	3	RAW 格式
GX_IMAGE_FORMAT_BMP	4	BMP 格式

3.2.115. GxVideoFormatType

定义	值	解释
GX_VIDEO_FORMAT_H264_AVI	0	H264 AVI 格式
GX_VIDEO_FORMAT_H264_MP4	1	H264 MP4 格式
GX_VIDEO_FORMAT_ORIGINAL_AVI	2	RAW 格式

3.2.116. GxSaveImageInfo

定义	解释
pImageBuffer	采集的图像 buffer，可通过 DQBuf 接口获取图像之后调用 GetBuffer 接口获取
nWidth	保存图像 buffer 的宽度，可通过图像对象中的 GetWidth 接口获取
nHeight	保存图像 buffer 的高度，可通过图像对象中的 GetHeight 接口获取
emSrcFormat	保存图像 buffer 的像素格式，可通过图像对象中的 GetPixelFormat 接口获取
emCfaMethod	保存图像的插值方式
emImgFormat	图片保存格式[bmp/jpg/png/raw/tiff]
pImgPath	图像保存路径
nImgQuality	编码质量, (0-100]，只对 JPEG 有效
nReserved	预留

3.2.117. GxRecordParam

定义	解释
emPixelFormat	保存图像 buffer 的像素格式，可通过图像对象中的 GetPixelFormat 接口获取
emVideoFormat	视频格式[h264_avi/h264_mp4/original_avi]
nWidth	保存图像 buffer 的宽度，可通过图像对象中的 GetWidth 接口获取
nHeight	保存图像 buffer 的高度，可通过图像对象中的 GetHeight 接口获取
nFrameRate	帧率 FPS
nBitRate	码率 kbps
pPathName	保存路径名称
nReserved	预留

3.3. 模块接口定义

3.3.1. DeviceManager

负责相机设备的管理，包括枚举设备、打开设备、获取设备数量信息等。

接口列表：

set_log_type	设置生成的日志类型
get_log_type	获取生成的日志类型
update_device_list	枚举同一网段中的设备
update_all_device_list	枚举不同网段中的设备
update_device_list_ex	按照指定 TL 类型枚举对应设备

get_device_number()	获取设备数量
get_device_info()	获取设备信息
get_interface_number()	获取 Interface 数量
get_interface_info()	获取 Interface 信息
get_interface()	获取 Interface 对象
open_device_by_sn	通过序列号打开设备
open_device_by_user_id	通过用户 ID 号打开设备
open_device_by_index	通过设备索引打开设备
open_device_by_ip	通过 IP 地址打开设备
open_device_by_mac	通过 mac 地址打开设备
gige_reset_device	执行设备复位操作
gige_force_ip	设置设备 ForceIP
gige_ip_configuration	进行 IP 配置
create_image_format_convert()	创建图像像素格式转换对象
create_image_process()	创建图像处理对象
issue_action_command	发送普通 ACTION
issue_scheduled_action_command	发送计划 ACTION
create_decompressor()	创建图像解压对象

3.3.1.1. set_log_type

声明： DeviceManager.set_log_type (log_type)

意义： 设置生成的日志类型。

形参： [in] log_type 待设置的日志类型，参考 [GxLogTypeList](#)

例：GX_LOG_TYPE_INFO | GX_LOG_TYPE_WARN

返回值： None。

异常处理：

如果输入参数不是整型值，则抛出 ParameterTypeError 异常。

3.3.1.2. get_log_type

声明： DeviceManager.get_log_type ()

意义： 设置生成的日志类型。

形参： 无

返回值： 返回获取到的日志类型，日志类型参考 [GxLogTypeList](#)

例：GX_LOG_TYPE_INFO | GX_LOG_TYPE_WARN

异常处理： 无

3.3.1.3. update_device_list

声明： DeviceManager.update_device_list (timeout=200)

意义： 对于非千兆网相机，枚举所有设备；对于千兆网相机，枚举同一网段设备。

形参： [in] timeout 枚举超时【0，0xffffffff】，缺省值为 200 (ms)

返回值： 枚举得到设备数量和记录枚举设备信息的列表 (list)。设备信息列表的元素个数为枚举到的设备个数，列表中元素的数据类型字典，字典中的键名称详见[枚举设备](#)。

异常处理：

- 1) 如果输入参数不是整型值，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数小于 0 或大于无符号整型的最大值 则打印“DeviceManager.update_device_list: Out of bounds, timeout:minimum=0, maximum= 0xffffffff ”，函数返回 None。
- 3) 如果枚举同一网段中设备不成功，则抛出异常，异常类型详见[错误处理](#)。
- 4) 如果获取所有设备基本信息不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.4. update_all_device_list

声明： DeviceManager.update_all_device_list (timeout=200)

意义： 对于非千兆网相机，枚举所有设备；对于千兆网相机，枚举全网设备。

形参： [in] timeout 枚举超时【0，0xffffffff】，缺省值为 200 (ms)

返回值： 枚举得到设备数量和记录枚举设备信息的列表 (list)。设备信息列表的元素个数为枚举到的设备个数，列表中元素的数据类型字典，字典中的键名称详见[枚举设备](#)。

异常处理：

- 1) 如果输入参数不是整型值，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数小于 0 或大于无符号整型的最大值，则打印：
“DeviceManager.update_all_device_list: Out of bounds, timeout:minimum=0, maximum= 0xffffffff ”，函数返回 None。
- 3) 如果枚举不同网段中设备不成功，则抛出异常，异常类型详见[错误处理](#)。
- 4) 如果获取所有设备基本信息不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.5. update_device_list_ex

声明： DeviceManager.update_device_list_ex (tl_type, timeout=2000)

意义： 按照指定 TL 类型枚举对应设备。

形参：

[in] tl_type TL 类型，详见 [GxTLClassList](#)

[in] timeout 枚举超时【0，0xffffffff】，缺省值为 2000 (ms)

返回值： 枚举得到设备数量和记录枚举设备信息的列表 (list)。设备信息列表的元素个数为枚举到的设备个数，列表中元素的数据类型字典，字典中的键名称详见[枚举设备](#)。

异常处理： 如果获取所有设备基本信息不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.6. get_device_number

声明 : DeviceManager.get_device_number()

意义 : 获取设备数量。

返回值 : 设备数量。

3.3.1.7. get_device_info

声明 : DeviceManager.get_device_info()

意义 : 获取设备信息。

返回值 : 设备信息列表。设备信息列表的元素个数为枚举到的设备个数，列表中元素的数据类型为字典，字典的键详见[枚举设备](#)。

3.3.1.8. get_interface_number

声明 : DeviceManager.get_interface_number ()

意义 : 获取 Interface 数量。

返回值 : Interface 数量。

3.3.1.9. get_interface_info

声明 : DeviceManager.get_interface_info()

意义 : 获取 Interface 信息。

返回值 : Interface 信息列表。设备信息列表的元素个数为枚举到的 Interface 个数，列表中元素的数据类型为字典，键包含：

键名称	意义	类型
type	TL 类型	GxTLCClassList
display_name	显示名称	字符串
interface_id	Interface ID	字符串
serial_number	设备序列号	字符串
description	设备显示名称	字符串
init_flag	初始标识（CXP 采集卡特有）	整型
reserved	用户自定义名称	字符串数组

3.3.1.10. get_interface

声明 : DeviceManager.get_interface()

意义 : 获取 Interface 对象。

返回值 : Interface 对象，详细见 [Interface](#)。

3.3.1.11. open_device_by_sn

声明 : DeviceManager.open_device_by_sn (sn, access_mode=GxAccessMode.CONTROL)

意义 : 通过序列号打开设备。

形参：

[in]	sn	序列号[字符串类型]
[in]	access_mode	打开设备模式，缺省值为 GxAccessMode.CONTROL ， 查看 GxAccessMode

返回值：设备对象。

异常处理：

- 1) 如果输入参数 1 不是字符串型，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数 2 不是整型，则抛出 ParameterTypeError 异常。
- 3) 如果输入参数 2 不在打开设备模式 [GxAccessMode](#) 中，则打印接口名称、打开设备方式不在范围内和当前参数所支持的枚举值信息，函数返回 None。
- 4) 如果重复获取设备类不成功，则抛出 NotFoundDevice 异常。
- 5) 如果打开设备不成功，则抛出异常，异常类型详见[错误处理](#)。
- 6) 如果打开获取的设备不是 U3V/USB2/GEV 类中的一种，则抛出 NotFoundDevice 异常。

3.3.1.12. open_device_by_user_id

声明 DeviceManager.open_device_by_user_id (user_id, access_mode=GxAccessMode.CONTROL)

意义：通过用户 ID 号打开设备。

形参：

[in]	user_id	用户 ID 号【字符串类型】
[in]	access_mode	打开设备模式，缺省值为 GxAccessMode.CONTROL ， 查看 GxAccessMode

返回值：设备对象。

异常处理：

- 1) 如果输入参数 1 不是字符串型，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数 2 不是整型，则抛出 ParameterTypeError 异常。
- 3) 如果输入参数 2 不在打开设备模式 [GxAccessMode](#) 中，则打印接口名称、打开设备方式不在范围内和当前参数所支持的枚举值的信息，函数返回 None。
- 4) 如果重复获取设备类不成功，则抛出 NotFoundDevice 异常。
- 5) 如果打开设备不成功，则抛出异常，异常类型详见[错误处理](#)。
- 6) 如果打开获取的设备不是 U3V/USB2/GEV 类中的一种，则抛出 NotFoundDevice 异常。

3.3.1.13. open_device_by_index

声明 DeviceManager.open_device_by_index (index, access_mode=GxAccessMode.CONTROL)

意义：通过设备索引打开设备。

形参：

[in]	index	设备索引【1，2，3...0xffffffff】
[in]	access_mode	打开设备方式，缺省值为 GxAccessMode.CONTROL ， 查看 GxAccessMode

返回值：设备对象。

异常处理：

- 1) 如果输入参数 1 或 2 不是整型值，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数 1 小于 0 或大于无符号整型的最大值，则打印“ DeviceManager.open_device_by_index: index out of bounds, index: minimum=1, maximum= 0xffffffff”，函数返回 None。
- 3) 如果输入参数 2 不在打开设备模式 [GxAccessMode](#) 中，则打印接口名称、打开设备方式不在范围内和当前参数所支持的枚举值的信息，函数返回 None。
- 4) 如果设备数量小于输入参数 1 索引，则抛出 NotFoundDevice 异常。
- 5) 如果打开设备不成功，则抛出异常，异常类型详见[错误处理](#)。
- 6) 如果打开获取的设备不是 U3V/USB2/GEV 类中的一种，则抛出 NotFoundDevice 异常。

3.3.1.14. open_device_by_ip

声明： DeviceManager.open_device_by_ip (ip, access_mode=GxAccessMode.CONTROL)

意义：通过设备 ip 地址打开千兆网相机设备。

形参：

[in]	ip	设备 ip 地址[字符串类型]
[in]	access_mode	打开设备模式，缺省值为 GxAccessMode.CONTROL ， 查看 GxAccessMode

返回值：设备对象。

异常处理：

- 1) 如果输入参数 1 不是字符串型，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数 2 不是整型，则抛出 ParameterTypeError 异常。
- 3) 如果输入参数 2 不在打开设备模式 [GxAccessMode](#) 中，则打印接口名称、打开设备方式不在范围内和当前参数所支持的枚举值的信息，函数返回 None。
- 4) 如果打开设备不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.15. open_device_by_mac

声明： DeviceManager.open_device_by_mac (mac, access_mode=GxAccessMode.CONTROL)

意义：通过设备 mac 地址打开千兆网相机设备。

形参：

[in]	mac	设备 mac 地址[字符串类型]
------	-----	------------------

[in] access_mode 打开设备模式，缺省值为 [GxAccessMode.CONTROL](#)，
查看 [GxAccessMode](#)

返回值：设备对象。

异常处理：

- 1) 如果输入参数 1 不是字符串型，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数 2 不是整型，则抛出 ParameterTypeError 异常。
- 3) 如果输入参数 2 不在打开设备模式 [GxAccessMode](#) 中，则打印接口名称、打开设备方式不在范围内和当前参数所支持的枚举值的信息，函数返回 None。
- 4) 如果打开设备不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.16. gige_reset_device

声明：DeviceManager.gige_reset_device (mac_address, reset_device_mode)

意义：执行设备复位操作。设备复位通常应用在调试千兆网相机时，设备已经被打开，此时程序异常，而后立即重新打开设备报错（因为调试心跳 5 分钟，设备依然处于打开状态），这时可通过设备复位功能，使设备处于未打开状态，而后再次打开设备即可成功。

设备复位通常应用在相机状态异常，用户不具备给设备重新上电的条件，可尝试使用设备复位功能，使设备掉电再上电。设备复位后，需要重新执行枚举、打开设备操作。

注意：

- 1) 设备复位时间需要 1s 左右，因此需要确保 1s 后再调用枚举接口。
- 2) 如果设备正在正常采集，禁止使用设备复位功能，否则会导致设备掉线。

形参：

[in] mac_address 设备 mac 地址【字符串类型】
[in] reset_device_mode 重置设备模式，参考 [GxResetDeviceModeEntry](#)。

返回值：None。

异常处理：如果执行命令失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.17. gige_force_ip

声明：DeviceManager.gige_force_ip(mac_address, ip_address, subnet_mask, default_gate_way)

意义：执行 force ip 操作。

形参：

[in] mac_address mac 地址
[in] ip_address ip 地址
[in] subnet_mask 子网掩码
[in] default_gate_way 默认网关

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.1.18. gige_ip_configuration

声明 : DeviceManager.gige_ip_configuration(mac_address, ipconfig_flag,
ip_address, subnet_mask,
default_gateway, user_id)

意义 : 执行 ip 配置操作。

形参 :

[in]	mac_address	mac 地址
[in]	ip_address	ip 地址
[in]	subnet_mask	子网掩码
[in]	default_gate_way	默认网关
[in]	ipconfig_flag	IP 配置模式, 详见 GxIPConfigureModeList
[in]	user_id	用户 ID

异常处理 : 如果调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.1.19. create_image_format_convert

声明 : DeviceManager.create_image_format_convert()

意义 : 创建图像像素格式转换对象。

返回值 : 像素格式转换对象, 详见 [ImageFormatConvert](#)。

3.3.1.20. create_image_process()

声明 : DeviceManager.create_image_process()

意义 : 创建图像处理对象。

返回值 : 图像处理对象, 详见 [ImageProcess](#)。

3.3.1.21. issue_action_command

声明 : DeviceManager.issue_action_command(self, device_key, group_key,
group_mask, broadcast_address,
special_address, time_out,
expect_ack_number_res)

意义 : 通过发送 ActionCommond 命令使网络上的相机同时执行 action 动作。

返回值 : 实际返回的 ack 列表, 详见 [GxActionCommandResult](#)。

形参 :

[in]	device_key	设备密钥, 对应设备的 “ActionDeviceKey” 属性
[in]	group_key	组密钥, 对应设备的 “ActionGroupKey” 属性
[in]	group_mask	组掩码, 对应设备的 “ActionGroupMask” 属性, 该值不能为 0
[in]	broadcast_address	发送 action 命令的目的 IP, 可为广播、子网广播、单播
[in]	special_address	发送 action 命令的源 IP, 用于明确的表明从哪个为网络适配器上发送命令。如果不指定则所有网络适配器的每一个 IP 都

		会发送当前 action 命令
[in]	time_out	0 表示不需要等待 ack 返回；非 0 表明最大等待 ack 返回时间 时间到后有参数 pNumResults 返回实际收到的 ack 个数。
[in]	expect_ack_number_res	表明期望返回的 ack 个数。此值表示为用户期望得到设备执行 action 命令反馈的预期 ack 个数。如果 time_out 为 0，则忽略此参数。因此，如果 time_out 为 0，则此参数可以为 NULL。

异常处理：如果调用不成功，则抛出异常，异常类型详见错误处理。

3.3.1.22. issue_scheduled_action_command

声明：DeviceManager.issue_scheduled_action_command(self, device_key, group_key, group_mask, action_time, broadcast_address, special_address, time_out, expect_ack_number_res)

意义：通过发送 ActionCommond 命令在某个绝对时间点，使网络上的相机同时执行 action 动作。

返回值：实际返回的 ack 列表，详见 [GxActionCommandResult](#)。

形参：

[in]	device_key	设备密钥，对应设备的 “ActionDeviceKey” 属性
[in]	group_key	组密钥，对应设备的 “ActionGroupKey” 属性
[in]	group_mask	组掩码，对应设备的 “ActionGroupMask” 属性，该值不能为 0
[in]	action_time	执行操作的时间（以纳秒为单位）。实际值可以为所使用的主时钟值加上预计的延时时间。主时钟值可以通过在从一组相机设备中锁存时间戳值 get_command_feature("TimestampLatch").send_command() 后读取时间戳值 get_int_feature("TimestampLatchValue").get()来获取一组同步相机设备的主时钟值。
[in]	broadcast_address	发送 action 命令的目的 IP，可为广播、子网广播、单播
[in]	special_address	发送 action 命令的源 IP，用于明确的表明从哪个为网络适配器上发送命令。如果不指定则所有网络适配器的每一个 IP 都会发送当前 action 命令
[in]	time_out	0 表示不需要等待 ack 返回；非 0 表明最大等待 ack 返回时间 时间到后有参数 pNumResults 返回实际收到的 ack 个数。
[in]	expect_ack_number_res	表明期望返回的 ack 个数。此值表示为用户期望得到设备执行 action 命令反馈的预期 ack 个数。如果 time_out 为 0，

则忽略此参数。因此，如果 time_out 为 0，则此参数可以为 NULL。

异常处理：如果调用不成功，则抛出异常，异常类型详见错误处理。

3.3.1.23. create_decompressor()

声明：DeviceManager.create_decompressor()

意义：创建图像解压对象。

3.3.2. Device

负责相机设备的采集控制、设备关闭、配置文件导入导出和获取设备句柄等。

接口列表：

get_stream_channel_num()	获取当前设备支持的流通道个数
get_parent_interface()	获取设备所属的 Interface 句柄
stream_on()	发送开始命令，相机开始传送图像数据
stream_off()	发送结束命令，相机结束传送图像数据
export_config_file()	导出当前配置文件
import_config_file()	导入配置文件
close_device()	关闭设备，销毁设备句柄，将句柄置为空
register_device_offline_callback()	注册掉线回调函数
unregister_device_offline_callback()	注销设备掉线回调函数
register_device_reconnect_callback()	注册重连回调函数
unregister_device_reconnect_callback()	注销设备重连回调函数
register_device_disconnect_callback()	注册断线回调函数
unregister_device_disconnect_callback()	注销设备断线回调函数
get_stream_channel_num()	获取流通道数量
get_stream()	获取流对象
get_local_device_feature_control()	获取本地属性控制器
get_remote_device_feature_control()	获取远端属性控制器
register_device_feature_callback_by_string()	通过字符串注册功能属性更新回调函数
unregister_device_feature_callback_by_string()	通过字符串注销功能属性更新回调函数
create_image_process_config()	创建图像处理配置对象
set_device_persistent_ip_address()	设置设备的永久 IP 信息
get_device_persistent_ip_address()	获取设备的永久 IP 信息
create_window()	创建设备对应的窗口操作句柄
destroy_window()	销毁对应的窗口
set_window_position()	设置窗口显示位置跟大小

set_window_mode()	设置窗口显示模式
show_window()	设置窗口显示或隐藏
set_window_title()	设置窗口标题
read_remote_device_port()	读远端寄存器地址值（不再维护）
write_remote_device_port()	写远端寄存器地址值（不再维护）
read_remote_device_port_stacked()	批量读用户指定寄存器的值（不再维护）
write_remote_device_port_stacked()	批量写用户指定寄存器的值（不再维护）
register_device_feature_callback()	通过功能码注册功能属性更新回调函数（不再维护）
unregister_device_feature_callback()	通过功能码注销功能属性更新回调函数（不再维护）

3.3.2.1. get_stream_channel_num

声明： Device.get_stream_channel_num()

意义： 获取当前设备支持的流通道个数。

返回值： 流通道个数。

注意： 目前千兆网相机、USB3.0、USB2.0 相机均不支持多流通道。

3.3.2.2. get_parent_interface

声明： Device.get_parent_interface()

意义： 获取设备所属的 Interface 句柄。

返回值： 设备所属的 Interface 句柄。

异常处理： 如果获取失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.3. stream_on

声明： Device.stream_on()

意义： 发送开始命令，相机开始传送图像数据。

返回值： None。

异常处理： 如果发送开始命令不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.4. stream_off

声明： Device.stream_off()

意义： 发送停止命令，相机停止传送图像数据。

返回值： None。

异常处理： 如果发送停止命令不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.5. export_config_file

声明： Device.export_config_file(file_path)

意义： 导出当前配置文件。

形参： [in] file_path 文件路径

返回值：None。

异常处理：

- 1) 如果输入参数不是字符串型，则抛出 ParameterTypeError 异常。
- 2) 如果导出当前配置文件不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.6. import_config_file

声明：Device.import_config_file(file_path, verify=False)

意义：导入配置文件。

形参：

[in]	file_path	文件路径
[in]	verify	是否所有导入值将被验证一致性，缺省值为 False

返回值：None。

异常处理：

- 1) 如果输入参数 1 不是字符串型，则抛出 ParameterTypeError 异常。
- 2) 如果输入参数 2 不是布尔型，则抛出 ParameterTypeError 异常。
- 3) 如果导入配置文件不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.7. close_device

声明：Device.close_device()

意义：关闭设备，销毁设备句柄，将句柄置为空。

返回值：None。

异常处理：如果关闭设备不成功，则抛出异常，异常类型详见[错误处理](#)。

注意：当执行关闭设备后，如果还想使用此相机，请重新打开后再操作。

3.3.2.8. register_device_offline_callback

声明：Device.register_device_offline_callback(callback_func)

意义：注册掉线回调函数。

形参：[in] callback_func 掉线回调函数

返回值：None。

异常处理：

- 1) 如果输入参数不是函数类型，则抛出 ParameterTypeError 异常。
- 2) 如果注册掉线回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.9. unregister_device_offline_callback

声明：Device.unregister_device_offline_callback()

意义：注销设备掉线回调函数。

返回值：None。

异常处理：如果注销掉线回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.10. register_device_reconnect_callback

声明：Device.register_device_reconnect_callback(callback_func)

意义：注册重连回调函数，使用前需要开启断线重连功能。

形参：[in] callback_func 重连回调函数

返回值：None。

异常处理：

- 1) 如果输入参数不是函数类型，则抛出 ParameterTypeError 异常。
- 2) 如果注册重连回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.11. unregister_device_reconnect_callback

声明：Device.unregister_device_reconnect_callback()

意义：注销设备重连回调函数。

返回值：None。

异常处理：如果注销重连回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.12. register_device_disconnect_callback

声明：Device.register_device_disconnect_callback(callback_func)

意义：注册断线回调函数，使用前需要开启断线重连功能。

形参：[in] callback_func 断线回调函数

返回值：None。

异常处理：

- 1) 如果输入参数不是函数类型，则抛出 ParameterTypeError 异常。
- 2) 如果注册掉线回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.13. unregister_device_disconnect_callback

声明：Device.unregister_device_disconnect_callback()

意义：注销设备断线回调函数。

返回值：None。

异常处理：如果注销断线回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.14. get_stream_channel_num

声明：Device.get_stream_channel_num()

意义：获取流通道数量。

返回值：流通道数量。

3.3.2.15. get_stream

声明：Device.get_stream(stream_index)

意义：获取流对象。

形参： [in] stream_index 流数组索引值，从 1 开始

返回值： 流对象，详见 [DataStream](#)。

异常处理： 如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.16. get_local_device_feature_control

声明： Device.get_local_device_feature_control()

意义： 获取本地属性控制器。

返回值： 属性控制器对象，详见 [FeatureControl](#)。

异常处理： 如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.17. get_remote_device_feature_control

声明： Device.get_remote_device_feature_control()

意义： 获取远端属性控制器。

返回值： 属性控制器对象，详见 [FeatureControl](#)。

3.3.2.18. register_device_feature_callback_by_string

声明： Device.register_device_feature_callback_by_string(callback_func, feature_name, args)

意义： 通过字符串注册功能属性更新回调函数。

形参：

[in]	callback_func	属性更新回调函数
[in]	feature_name	功能字符串节点
[in]	args	用户参数可为 None

返回值： 回调函数句柄。

异常处理： 如果注册回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.19. unregister_device_feature_callback_by_string

声明： Device.unregister_device_feature_callback_by_string(feature_name, feature_callback_handle)

意义： 通过功能码注销功能属性更新回调函数。

形参：

[in]	feature_name	功能字符串节点
[in]	feature_callback_handle	回调函数句柄

返回值： None。

异常处理： 如果注销回调函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.20. create_image_process_config()

声明： Device.create_image_process_config()

意义： 创建图像处理配置对象。

返回值： 图像处理配置对象，详见 [ImageProcessConfig](#)。

异常处理：如果输入参数不支持功能节点“ColorCorrectionParam”，或支持但不可读则抛出 UnexpectedError 异常。

3.3.2.21. set_device_persistent_ip_address

声明：Device.set_device_persistent_ip_address(ip, subnet_mask, default_gate_way)

意义：设置设备的永久 IP 信息。

形参：

[in]	ip	ip 地址
[in]	subnet_mask	子网掩码
[in]	default_gate_way	默认网关

返回值：None。

异常处理：如果设置 IP 失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.22. get_device_persistent_ip_address

声明：Device.get_device_persistent_ip_address()

意义：获取设备的永久 IP 信息。

返回值：设备的 IP 地址、子网掩码和网关。

异常处理：如果获取设备 IP 失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.23. create_window

声明：Device.create_window(window_id, parent_window_handle)

意义：创建设备对应的窗口操作句柄。

形参：

[in]	window_id	窗口类型。详见 GxWindowsID
[in]	parent_window_handle	父窗口句柄，仅支持传 0 窗口将会弹出显示。 (预留此参数后面支持嵌入显示)

返回值：窗口操作句柄。

异常处理：如果创建失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.24. destroy_window

声明：Device.destroy_window(operate_wnd_handle)

意义：销毁对应的窗口。

形参：

[in]	operate_wnd_handle	窗口操作句柄
------	--------------------	--------

返回值：None。

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.25. set_window_position

声明：Device.set_window_position (operate_wnd_handle, pos_x, pos_y, width, height):

意义：设置窗口显示位置跟大小。

形参：

[in]	operate_wnd_handle	窗口操作句柄
[in]	pos_x	窗口相对屏幕水平位置
[in]	pos_y	窗口相对屏幕垂直位置
[in]	width	窗口宽度，最小宽度 450
[in]	height	窗口对应的高度，最小高度 600

返回值：None。

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.26. set_window_mode

声明：Device.set_window_mode(operate_wnd_handle, mode):

意义：设置窗口显示模式。

形参：

[in]	operate_wnd_handle	窗口操作句柄
[in]	mode	显示模式。 详见： GxWindowsShowMode

返回值：None。

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

注意：当设置为 NON_BLOCK_SHOW_MODE 时不会阻塞后面要执行的操作，当设置为 BLOCK_SHOW_MODE 时会阻塞后面要执行的操作。如果您的程序为控制台模式启动在显示窗口之前必须调用该接口将模式设置为 BLOCK_SHOW_MODE ;如果您的程序为 GUI 程序(Qt、MFC、Winforms 等) 请勿调用此接口设置 BLOCK_SHOW_MODE 模式，否则可能发生未定义行为。

3.3.2.27. show_window

声明：Device.show_window(operate_wnd_handle, visible):

意义：设置窗口显示或隐藏。

形参：

[in]	operate_wnd_handle	窗口操作句柄
[in]	visible	true 显示；false 隐藏。

返回值：None。

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.28. set_window_title

声明：Device.set_window_title(operate_wnd_handle, title):

意义：设置窗口标题。

形参：

[in]	operate_wnd_handle	窗口操作句柄
[in]	title	标题字符串指针

返回值：None。

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.29. read_remote_device_port(不再维护)

推荐用 [FeatureControl.read_port\(\)](#)

声明：Device.read_remote_device_port(address, buff, size)

意义：读用户指定寄存器的值。

形参：

[in]	address	寄存器地址
[out]	buff	返回寄存器内容的缓存地址
[in out]	size	用户申请的 Buffer 大小,调用完后返回实际大小

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.30. write_remote_device_port(不再维护)

推荐用 [FeatureControl.write_port\(\)](#)

声明：Device.write_remote_device_port(address, buf, size)

意义：向用户指定的寄存器中写入用户给定的数据。

形参：

[in]	address	寄存器地址
[in]	buff	要写的寄存器内容的缓存地址
[in]	size	用户申请的 Buffer 大小,调用完后返回实际大小

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.31. read_remote_device_port_stacked(不再维护)

推荐用 [FeatureControl.read_port_stacked\(\)](#)

声明：Device.read_remote_device_port_stacked(entries, size)

意义：批量读用户指定寄存器的值（仅限命令值为 4 字节长度的寄存器）。

形参：

[in]	entries	批量读寄存器结构体地址
[in out]	size	读取设备寄存器的个数

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.2.32. write_remote_device_port_stacked(不再维护)

推荐用 [FeatureControl.write_port_stacked\(\)](#)

声明 : Device.write_remote_device_port_stacked(entries, size)

意义 : 批量向用户指定的寄存器中写入用户给定的数据 (仅限命令值为 4 字节长度的寄存器)

形参 :

[in]	entries	批量写寄存器结构体地址
[in]	size	写设备寄存器的个数

异常处理 : 如果调用函数失败, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.2.33. register_device_feature_callback(不再维护)

推荐用 [register_device_feature_callback_by_string\(\)](#)

声明 : Device.register_device_feature_callback(callback_func, feature_id, args)

意义 : 通过功能码注册功能属性更新回调函数。

形参 :

[in]	callback_func	属性更新回调函数
[in]	feature_id	功能码节点
[in]	args	用户参数可为 None

返回值 : 回调函数句柄。

异常处理 : 如果调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.2.34. unregister_device_feature_callback(不再维护)

推荐用 [unregister_device_feature_callback_by_string\(\)](#)

声明 : Device.unregister_device_feature_callback(feature_id, feature_callback_handle)

意义 : 通过功能码注销功能属性更新回调函数。

形参 :

[in]	feature_id	功能码节点
[in]	feature_callback_handle	回调函数句柄

返回值 : None。

异常处理 : 如果注销回调函数失败, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.3. Interface

封装 Interface 类。

接口列表 :

get_interface_info()	获取 Interface 信息
get_feature_control()	获取 Interface 属性控制对象

3.3.3.1. get_interface_info

声明 : get_interface_info()

意义 : 获取 Interface 信息。

返回值 : 返回 Interface 信息字典，键包含：

键名称	意义	类型
type	TL 类型	GxTlClassList
display_name	显示名称	字符串
interface_id	Interface ID	字符串
serial_number	设备序列号	字符串
description	设备显示名称	字符串
init_flag	初始标识（CXP 采集卡特有）	整型
reserved	用户自定义名称	字符串数组

3.3.3.2. get_feature_control

声明 : get_feature_control()

意义 : 获取 Interface 属性控制对象。

返回值 : Interface 属性控制对象，详细见 [FeatureControl](#)。

3.3.4. DataStream

负责相机设备的数据流设置、控制，获取图像等。

接口列表：

set_acquisition_buffer_number()	设置采集缓冲的大小
get_image()	获取图像，成功创建图像类对象
dq_buf()	零拷贝获取图像
q_buf()	将 image 对象交还给采集系统
flush_queue()	清除相机采集缓冲队列
register_capture_callback()	注册采集回调函数
unregister_capture_callback()	注销采集回调函数
get_feature_control()	获取属性控制对象
get_payload_size()	获取 Payloadsize
register_buffer()	注册用户申请的 buffer 作为内部采集使用
unregister_buffer()	注销使用 register_buffer()接口注册的 buffer

3.3.4.1. set_acquisition_buffer_number

声明 : DataStream.set_acquisition_buffer_number(buf_num)

意义 : 设置采集缓冲的大小。

形参 : [in] buf_num 缓冲区地址的长度【1，0xffffffff】

返回值：None。

异常处理：

- 1) 如果输入参数不是整型，则抛出 `ParameterTypeError` 异常。
- 2) 如果输入参数小于 1 或大于无符号整型的最大值，则打印
“`DataStream.set_acquisition_buffer_number: buf_num out of bounds, minimum=1, maximum=0xffffffff`”，函数返回 `None`。
- 3) 如果设置采集缓冲大小不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.4.2. get_image

声明：`DataStream.get_image(timeout=1000)`

意义：获取图像，成功创建图像类对象

形参：`[in] timeout` 获取超时【0，0xffffffff】，缺省值为 1000 (ms)

返回值：

图像对象：获取成功

None：超时

抛出异常：其他错误

异常处理：

- 1) 如果输入参数不是整型，则抛出 `ParameterTypeError` 异常。
- 2) 如果输入参数小于 0 或大于 0xffffffff 则打印“`DataStream.get_image: timeout out of bounds, minimum=0, maximum=0xffffffff`”，函数返回 `None`。
- 3) 如果获取数据大小不成功，则抛出异常，异常类型详见[错误处理](#)。
- 4) 如果超时导致未获取图像成功，则函数返回 `None`。
- 5) 如果未超时但获取图像失败，则打印“`status, DataStream, get_image`”，函数返回 `None`。

3.3.4.3. dq_buf

声明：`DataStream.dq_buf(timeout=1000)`

意义：零拷贝获取图像，成功创建图像类对象。

形参：`[in] timeout` 获取超时【0，0xffffffff】，缺省值为 1000 (ms)

返回值：

图像对象：获取成功

None：超时

抛出异常：其他错误

异常处理：

- 1) 如果输入参数不是整型，则抛出 `ParameterTypeError` 异常；
- 2) 如果输入参数小于 0 或大于 0xffffffff，则打印“`DataStream.dq_buf: timeout out of bounds,`

minimum=0, maximum=0xffffffff” , 函数返回 None ;

- 3) 如果获取数据大小不成功, 则抛出异常, 异常类型详见[错误处理](#);
- 4) 如果超时导致未获取图像成功, 则函数返回 None ;
- 5) 如果未超时但获取图像失败, 则打印 “status, DataStream, dq_buf” , 函数返回 None。

3.3.4.4. q_buf

声明 : DataStream.q_buf(image)

意义 : 将 image 对象交还给采集系统。

形参 : [in] image dq_buf 获取到的 image 对象

返回值 : None

异常处理 :

- 1) 如果输入参数不是 RawImage , 则抛出 ParameterTypeError 异常 ;
- 2) 如果未开采, 打印错误信息, 直接返回 ;
- 3) 如果已注册回调函数, 则抛出 InvalidCall 异常。

3.3.4.5. flush_queue

声明 : DataStream.flush_queue()

意义 : 清除相机采集缓冲队列。

异常处理 : 如果清除相机采集缓冲队列不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.4.6. register_capture_callback

声明 : DataStream.register_capture_callback(callback_func)

意义 : 注册采集回调函数, 回调方式采集使用方式见 : [采集控制-回调方式](#)

形参 : [in] callback_func 采集回调函数

返回值 : None。

异常处理 :

- 1) 如果输入参数不是函数类型, 则抛出 ParameterTypeError 异常。
- 2) 如果注册采集回调函数失败, 则抛出异常, 异常类型详见[错误处理](#)。

注意 : 需要先注册采集回调函数, 再执行 stream_on 开采。

3.3.4.7. unregister_capture_callback

声明 : DataStream.unregister_capture_callback()

意义 : 注销采集回调函数。

返回值 : None。

异常处理 : 如果注销采集回调函数失败, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.4.8. get_feature_control

声明 : DataStream.get_feature_control()

意义 : 获取流属性控制对象。

返回值：流属性控制对象，详见 [FeatureControl](#)。

3.3.4.9. get_payload_size

声明：DataStream.get_payload_size()

意义：获取 payloadsize。

返回值：返回 payloadsize。

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

3.3.4.10. register_buffer

声明：DataStream.register_buffer(user_buf, user_param)

意义：注册用户申请的 buffer 作为内部采集使用，用户可自主管理 buffer 内存。

形参：

[in] user_buf 用户申请的 buffer 指针

[in] user_param 用户自定义参数，可通过 [RawImage.get_user_param\(\)](#) 获取

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

注意：

1) 用户申请的 buffer 大小不能自行计算，需要从 [get_payload_size\(\)](#) 接口获取，否则可能导致异常。POLS 采集卡等产品基于性能考虑要求采集使用的内存大小和首地址对齐，因此用户在使用该接口时需要按照对齐字节数属性 StreamBufferAlignment 的要求对首地址和大小对齐。

2) 注册 buffer 个数较少时，采集驱动完成了单个采集(buffer 处于消费者队列)，驱动内无 buffer 可用可能导致丢帧。因此建议注册 buffer 数量至少是 3 个，并且尽快完成图像的使用。另外 POLS 等硬件要求最小 AnnounceBuffer 数量必须大于等于 3，如果用户 register 的 Buffer 数量不足，则会导致开采失败。

3) 调用 [register_buffer\(\)](#) 接口后，如果用户修改了相机或采集卡的参数，导致 PayloadSize 发生变化，需要重新注销掉已注册的 buffer，根据新长度重新申请内存并注册。

4) [register_buffer\(\)](#) 为可选接口，若不调用此接口注册 buffer，开采后 API 库会自动申请 buffer 并开采，不影响采集流程。

代码样例：

创建一个字节对齐的 buffer

```
def create_aligned_buffer(size, alignment):
    buf = mmap.mmap(-1, size + alignment, access=mmap.ACCESS_WRITE)
    address = (ctypes.c_char * size).from_buffer(buf)
    return address
```

获取 buffer 大小和对齐字节数

```
payload_size = cam.data_stream[0].get_payload_size()
stream_feature_control = cam.data_stream[0].get_feature_control()
alignment=stream_feature_control.get_int_feature(
    "StreamBufferAlignment").get()
```

```
# 创建 buffer，并注册到采集系统
buf1 = create_aligned_buffer(payload_size, alignment)
buf2 = create_aligned_buffer(payload_size, alignment)
buf3 = create_aligned_buffer(payload_size, alignment)
cam.data_stream[0].register_buffer(buf1, 1)
cam.data_stream[0].register_buffer(buf2, 3.14)
cam.data_stream[0].register_buffer(buf3, "test")

# 开始采集
cam.stream_on()
num = 3
for i in range(num):
    # 打开第 0 通道数据流
    raw_image = cam.data_stream[0].get_image()
    print('user_param: ', raw_image.get_user_param())

# 停止采集
cam.stream_off()

# 注销采集 buffer
cam.data_stream[0].unregister_buffer(buf1)
cam.data_stream[0].unregister_buffer(buf2)
cam.data_stream[0].unregister_buffer(buf3)
```

3.3.4.11. unregister_buffer

声明：DataStream.unregister_buffer (user_buf)

意义：注销用户使用 [register_buffer\(\)](#)接口注册的 buffer。

形参：

[in] user_buf 用户申请的 buffer 指针

异常处理：如果调用函数失败，则抛出异常，异常类型详见[错误处理](#)。

注意：此接口需要在停止采集后调用，开采过程中调用会影响正常采集流程。

3.3.5. FeatureControl

负责查询各节点访问属性是否支持、可读、可写以及进行设置数据、获取数据。

接口列表：

is_implemented()	查询当前功能是否支持
is_readable()	查询当前功能是否可读
is_writable()	查询当前功能是否可写
get_int_feature()	获取一个整型数据节点对象
get_enum_feature()	获取一个枚举型数据节点对象
get_float_feature()	获取一个浮点型数据节点对象
get_bool_feature()	获取一个布尔型数据节点对象

get_string_feature()	获取一个字符串型数据节点对象
get_command_feature()	获取一个命令型数据节点对象
get_register_feature()	获取一个寄存器类型数据节点对象
feature_save()	保存用户参数组
feature_load()	加载用户参数组
read_port()	读指定寄存器的值
write_port()	写指定寄存器的值
read_port_stacked()	批量读取指定的寄存器值
write_port_stacked()	批量写入指定的寄存器值

3.3.5.1. is_implemented

声明 : `is_implemented(feature_name)`

意义 : 查询当前功能是否支持。

形参 : [in] `feature_name` 属性字符串

返回值 : 支持返回 true, 否则返回 false。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.2. is_readable

声明 : `is_readable(feature_name)`

意义 : 查询当前功能是否可读。

形参 : [in] `feature_name` 属性字符串

返回值 : 可读返回 true, 否则返回 false。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.3. is_writable

声明 : `is_writable(feature_name)`

意义 : 查询当前功能是否可写。

形参 : [in] `feature_name` 属性字符串

返回值 : 可写返回 true, 否则返回 false。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.4. get_int_feature

声明 : `get_int_feature(feature_name)`

意义 : 获取一个整型数据节点对象。

形参 : [in] `feature_name` 属性字符串

返回值 : 返回整型数据节点对象, 详见 [IntFeature_s](#)。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.5. get_enum_feature

声明 : get_enum_feature(feature_name)

意义 : 获取一个枚举型数据节点对象。

形参 : [in] feature_name 属性字符串

返回值 : 返回枚举型数据节点对象, 详见 [EnumFeature_s](#)。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.6. get_float_feature

声明 : get_float_feature (feature_name)

意义 : 获取一个浮点型数据节点对象。

形参 : [in] feature_name 属性字符串

返回值 : 返回浮点型数据节点对象, 详见 [EnumFeature_s](#)。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.7. get_bool_feature

声明 : get_bool_feature(feature_name)

意义 : 获取一个布尔型数据节点对象。

形参 : [in] feature_name 属性字符串

返回值 : 返回布尔型数据节点对象, 详见 [BoolFeature_s](#)。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.8. get_string_feature

声明 : get_string_feature(feature_name)

意义 : 获取一个字符串型数据节点对象。

形参 : [in] feature_name 属性字符串

返回值 : 返回字符串型数据节点对象, 详见 [StringFeature_s](#)。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.9. get_command_feature

声明 : get_command_feature(feature_name)

意义 : 获取一个命令型数据节点对象, 详见 [CommandFeature_s](#)。

形参 : [in] feature_name 属性字符串

返回值 : 返回命令型数据节点对象。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.5.10. get_register_feature

声明 : get_register_feature(feature_name)

意义 : 获取一个寄存器类型数据节点对象, 详见 [RegisterFeature_s](#)。

形参：

[in]

feature_name

属性字符串

返回值：

返回寄存器类型数据节点对象。

异常处理：

如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.5.11. feature_save

声明：

feature_save(file_path)

意义：

保存当前设备配置参数到文本文件。

形参：

[in]

file_path

导出文件路径名

异常处理：

如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.5.12. feature_load

声明：

feature_load(file_path, verify=False)

意义：

配置文件中的参数加载到设备。

形参：

[in]

file_path

加载文件路径名

异常处理：

如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.5.13. read_port

声明：

read_port(address, size)

意义：

读取相机指定寄存器的值。

注意：

读取到千兆网相机属性值为大端数据。

形参：

[in]

address

属性寄存器地址

[in]

size

要读取属性的字节大小

返回值：

返回指定寄存器的值。

异常处理：

如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.5.14. write_port

声明：

write_port(address, buff, size)

意义：

设置相机指定寄存器的值。

注意：

- 1) 千兆网相机设置的属性值需转换为大端后设置。
- 2) 调用当前接口后，使用 [get_enum_feature\(\)](#)、[get_int_feature\(\)](#)等接口获取到的节点值仍为修改前值，可使用 [read_port\(\)](#)或 [read_port_stacked\(\)](#)接口获取最新的属性值。

形参：

[in]

address

属性寄存器地址

[out]

buff

要写入的属性缓存

[in]

size

要写入的属性大小

异常处理：

如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.5.15. read_port_stacked

声明： read_port_stacked(entries, size)

意义： 批量读取相机指定的多个寄存器的值，当前仅支持整型（4 字节）。

注意： 获取千兆网相机属性值为大端数据。

形参：

[in]	entries	指向 GxRegisterStackEntry 类型的指针数组
[in]	size	数组大小

异常处理： 如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

代码样例：

```
# 获取属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 结构体个数
i = 2

# 为 buffer 申请内存
outbuffer = 64
buffer = c_void_p()
buffer.value = id(outbuffer)

outbuffer1 = 1024
buffer1 = c_void_p()
buffer1.value = id(outbuffer1)

# 定义结构体列表
struct_list = [c_ulonglong(0x00900018), buffer, c_uint(4)]
struct_list1 = [c_ulonglong(0x00900008), buffer1, c_uint(4)]

# 结构体初始化
struct_Reg = GxRegisterStackEntry(*struct_list)
struct_Reg1 = GxRegisterStackEntry(*struct_list1)

# 创建多链表
list = [struct_Reg, struct_Reg1]

# 定义结构体数组，类型为 GxRegisterStackEntry，长度为 2
struct_array = GxRegisterStackEntry * 2

# 结构体数组初始化
array_instance = struct_array(*list)

# 转换为指针批量读取寄存器
struct = pointer(array_instance)
status = remote_device_feature.read_port_stacked(struct, i)
```



```
assert status == 0

# 获取内存中的数据
length = c_int
width_value = cast(array_instance[0].buffer,
                    POINTER(length)).contents.value
width_value1 = cast(array_instance[1].buffer,
                     POINTER(length)).contents.value
```

3.3.5.16. write_port_stacked

声明： write_port_stacked(entries, size)

意义： 批量设置相机指定的多个寄存器的值，当前仅支持整型（4 字节）。

注意：

- 1) 千兆网相机设置的属性值需转换为大端后设置。
- 2) 调用当前接口后，使用 [get_enum_feature\(\)](#)、[get_int_feature\(\)](#)、[get_bool_feature\(\)](#) 等接口获取到的节点值仍为修改前值，可使用 [read_port\(\)](#) 或 [read_port_stacked\(\)](#) 接口获取最新的属性值。

形参：

[in]	entries	指向 GxRegisterStackEntry 类型的指针数组
[in]	size	数组大小

异常处理： 如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

代码样例：

```
# 获取属性控制器
remote_device_feature = cam.get_remote_device_feature_control()

# 结构体个数
i = 2

size = 2
value = c_int(64)
point = byref(value)
bufer = cast(point, c_void_p)
buffer_c = c_void_p()
buffer_c = bufer

# 定义结构体列表
struct_list = [c_ulonglong(0x00900008), buffer_c, c_uint(4)]
struct_list1 = [c_ulonglong(0x0090000C), buffer_c, c_uint(4)]

struct_Reg = GxRegisterStackEntry(*struct_list)
struct_Reg1 = GxRegisterStackEntry(*struct_list1)
list = [struct_Reg, struct_Reg1]

struct_array = GxRegisterStackEntry * 2
array_instance = struct_array(*list)
```

```
struct = pointer(array_instance)

# 批量写寄存器
status = remote_device_feature.write_port_stacked(struct, size)
assert status == 0
assert status == 0
```

3.3.6. RGBImage(不再维护)

推荐用 ImageProcess。

负责 RGB 图像操作。

接口列表：

image_improvement()	图像质量提高
brightness()	图像进行亮度调节
contrast()	图像进行对比度调节
saturation()	图像进行饱和度调节
sharpen()	图像进行锐化处理
get_white_balance_ratio()	获取图像白平衡系数
get_numpy_array()	将 RGB 数据转换为 numpy 对象
get_image_size()	获取 RGB 数据大小

3.3.6.1. image_improvement

声明： RGBImage.image_improvement(color_correction_param=0,
contrast_lut=None, gamma_lut=None,
channel_order=DxRGBChannelOrder.ORDER_RGB)

意义： 图像质量提高。

形参：

[in]	contrast_lut	对比度 LUT
[in]	gamma_lut	gamma LUT
[in]	color_collect	颜色校正
[in]	channel_order	图像通道顺序，缺省值为 DxRGBChannelOrder.ORDER_RGB， 参考 DxRGBChannelOrder

异常处理：

- 1) 如果参数 1、2 不是 Buffer 类型或 None，则抛异常 ParameterTypeError。
- 2) 如果参数 3 不是整型或 None，则抛异常 ParameterTypeError。
- 3) 如果参数 4 不是整型，则抛异常 ParameterTypeError。
- 4) 如果提高图像质量不成功，则抛出异常 UnexpectedError。
- 5) 如果参数 1、2、3 都是默认缺省值，则不进行图像质量提升处理，函数退出。

3.3.6.2. brightness

声明 : RGBImage.brightness(factor)

意义 : 对 RGB24 图像进行亮度调节。

形参 : [in] factor 亮度调节因子, 值范围-150~150。其中 : 等于 0 时亮度没有变化 ;
大于 0 : 增加亮度 ; 小于 0 : 减小亮度 ;

返回值 : None。

异常处理 :

- 1) 如果输入参数不是整型, 则抛出 ParameterTypeError 异常。
- 2) 如果亮度调节失败, 则抛出 UnexpetedError 异常。

3.3.6.3. contrast

声明 : RGBImage.contrast(factor)

意义 : 对 RGB24 图像进行对比度调节。

形参 : [in] factor 对比度调节因子, 值范围-50~150。其中, 等于 0 时对比度没有变化 ; 大于 0 : 增加对比度 ; 小于 0 : 减小对比度 ;

返回值 : None。

异常处理 :

- 1) 如果输入参数不是整型, 则抛出 ParameterTypeError 异常。
- 2) 如果对比度调节失败, 则抛出 UnexpetedError 异常。

3.3.6.4. saturation

声明 : RGBImage.saturation(factor)

意义 : 对 RGB 图像进行饱和度调节。

形参 : [in] factor 饱和度调节参数, 范围 : 0 ~ 128, 其中 :
64 : 饱和度没有变化 ; 大于 64 : 增加饱和度 ;
小于 64 : 减小饱和度 ; 128 : 饱和度为当前两倍 ;
0 : 黑白图像

返回值 : None。

异常处理 : 如果图像饱和度调节失败, 则抛出异常 UnexpectedError。

3.3.6.5. sharpen

声明 : RGBImage.sharpen()

意义 : 对 RGB 图像进行锐化处理。

形参 : [in] factor 锐化调节参数, 范围 : 0.1 ~ 5.0

返回值 : None。

异常处理 : 如果图像锐化处理失败, 则抛出异常 UnexpectedError。

3.3.6.6. get_white_balance_ratio

声明 : RGBImage.get_white_balance_ratio()

意义 : 获取白平衡系数。

返回值 : 返回 RGB 分量系数元组。

3.3.6.7. get_numpy_array

声明 : RGBImage.get_numpy_array()

意义 : 获取 numpy 对象。

返回值 : numpy 对象。

3.3.6.8. get_image_size

声明 : RGBImage.get_image_size()

意义 : 获取 RGB 数据大小。

返回值 : RGB 图的大小。

3.3.7. RawImage

负责 Raw 图像操作。

接口列表 :

[defective_pixel_correct\(\)](#)

图像坏点校正

[raw8_rotate_90_cw\(\)](#)

将 8 位图像顺时针旋转 90 度

[raw8_rotate_90_ccw\(\)](#)

将 8 位图像逆时针旋转 90 度

[mirror\(\)](#)

对 8 位图像进行镜像处理

[get_ffc_coefficients\(\)](#)

计算图像平场校正系数

[flat_field_correction\(\)](#)

对图像进行平场校正处理

[get_numpy_array\(\)](#)

将 raw 数据转换为 numpy 对象

[get_data\(\)](#)

获取 raw 数据

[save_raw\(\)](#)

保存 raw 图数据

[get_status\(\)](#)

获取 raw 图状态

[get_width\(\)](#)

获取 raw 图宽度

[get_height\(\)](#)

获取 raw 图高度

[get_pixel_format\(\)](#)

获取图像像素格式

[get_image_size\(\)](#)

获取 raw 图数据大小

[get_frame_id\(\)](#)

获取帧 ID

[get_timestamp\(\)](#)

获取时间戳

[rgb8_to_numpy_array\(\)](#)

将 rgb8 数据转换为 numpy 对象

[is_color_cam\(\)](#)

是否是彩色相机

[get_output_pixel_format\(\)](#)

获取输出像素格式

get_chunkdata()	获取帧信息数据
get_user_param()	获取用户自定义参数
convert()	图像格式转换（不再维护）
brightness()	对 8 位灰度图像进行亮度调节（不再维护）
contrast()	对 8 位灰度图像进行对比度调节(不再维护)
get_chunk_data_feature_control()	获取当前图像的帧信息属性控制器

3.3.7.1. defective_pixel_correct

声明：RawImage.defective_pixel_correct()

意义：对 raw 数据进行坏点校正。

返回值：None。

异常处理：如果坏点校正不成功，则抛出异常 UnexpetedError。

3.3.7.2. raw8_rotate_90_cw

声明：RawImage.raw8_rotate_90_cw()

意义：对 8 位图像顺时针旋转 90 度。

返回值：旋转后的 RawImage 图像对象。

异常处理：如果旋转失败，则抛出异常 UnexpetedError。

3.3.7.3. raw8_rotate_90_ccw

声明：RawImage.raw8_rotate_90_ccw()

意义：对 8 位图像逆时针旋转 90 度。

返回值：旋转后的 RawImage 图像对象。

异常处理：如果旋转失败，则抛出异常 UnexpetedError。

3.3.7.4. mirror

声明：RawImage.mirror(mirror_mode)

意义：为 8 位图像产生一个与原图像在水平方向或者垂直方向相对称的镜像图像。

形参：[in] mirror_mode 图像镜像翻转方式，参考 [DxImageMirrorMode](#)。

返回值：镜像后的 RawImage 图像对象。

异常处理：

- 1) 如果输入参数不是整型，则抛出 ParameterTypeError 异常。
- 2) 如果不是 8 位图像格式，则抛出 InvalidParameter 异常。
- 3) 如果图像镜像失败，则抛出 UnexpetedError 异常。

3.3.7.5. get_ffc_coefficients

声明：RawImage.get_ffc_coefficients(dark=None, target_value=None)

意义：计算图像平场校正系数，仅支持 8~12 位 Raw 图。

形参：

[in]	dark	暗场图像
[in]	target_value	期望灰度值

返回值：平场校正系数。

异常处理：

- 1) 如果 dark 类型不是 RawImage，则抛出 ParameterTypeError 异常。
- 2) 如果 target_value 不是 INT 类型，则抛出 ParameterTypeError 异常。
- 3) 如果图像格式不是 8/10/12 位，则抛出 InvalidParameter 异常。
- 4) 如果获取平场校正系数调用失败，则抛出 UnexpetedError 异常。

3.3.7.6. flat_field_correction

声明：RawImage.flat_field_correction(ffc_coefficients)

意义：为 8~12 位的 Raw 图进行平场校正操作。

形参：[in] ffc_coefficients 平场校正系数。

返回值：None。

异常处理：

- 1) 如果 ffc_coefficients 不是 Buffer 类型，则抛出 ParameterTypeError 异常。
- 2) 如果图像格式不是 8/10/12 位图像格式，则抛出 InvalidParameter 异常。
- 3) 如果图像平场校正失败，则抛出 UnexpetedError 异常。

3.3.7.7. get_numpy_array

声明：RawImage.get_numpy_array()

意义：将 raw 数据转换为 numpy 对象。

返回值：

numpy 对象：成功

None：失败

异常处理：

- 1) 如果帧信息状态不成功，则打印错误信息“RawImage.get_numpy_array:This is a incomplete image”，函数返回 None。
- 2) 如果像素格式不为 8 位或 16 位，则返回 None。

3.3.7.8. get_data

声明：RawImage.get_data()

意义：获取 raw 数据。

返回值：raw 数据【字符串型】

3.3.7.9. save_raw

声明 : RawImage.save_raw(file_path)

意义 : 保存 raw 图数据。

形参 : [in] file_path 文件路径。例如 : file_path = 'raw_image.raw' , 则将 raw 图保存到当前工程路径下 ; file_path = 'E://python_gxiapi/raw_image.raw' , 则将 raw 图保存到绝对路径'E://python_gxiapi/'下。

返回值 : None。

异常处理 :

- 1) 如果参数不是字符串格式 , 则抛出异常 ParameterTypeError。
- 2) 如果保存 raw 图数据未成功 , 则抛异常 UnexpectedError。

3.3.7.10. get_status

声明 : RawImage.get_status()

意义 : 获取 raw 图状态。

返回值 : raw 图状态 , 数据类型参考 [GxFrameStatusList](#)。

3.3.7.11. get_width

声明 : RawImage.get_width()

意义 : 获取 raw 图宽度。

返回值 : raw 图宽度。

3.3.7.12. get_height

声明 : RawImage.get_height()

意义 : 获取 raw 图高度。

返回值 : raw 图高度。

3.3.7.13. get_pixel_format

声明 : RawImage.get_pixel_format()

意义 : 获取图像像素格式。

返回值 : 像素格式。

3.3.7.14. get_image_size

声明 : RawImage.get_image_size()

意义 : 获取 raw 图数据大小。

返回值 : Raw 图的大小。

3.3.7.15. get_frame_id

声明 : RawImage.get_frame_id()

意义 : 获取帧 ID。

返回值：帧 ID。

3.3.7.16. get_timestamp

声明：RawImage.get_timestamp()

意义：获取时间戳。

返回值：时间戳。

3.3.7.17. rgb8_to_numpy_array

声明：RawImage.rgb8_to_numpy_array()

意义：将 rgb8 数据转换为 numpy 对象。

返回值：

numpy 对象：成功

None：失败

3.3.7.18. is_color_cam

声明：RawImage.is_color_cam()

意义：是否是彩色相机。

返回值：是否是彩色相机。

3.3.7.19. get_output_pixel_format

声明：RawImage.get_output_pixel_format()

意义：获取输出像素格式。

返回值：输出像素格式。

3.3.7.20. get_chunkdata

声明：RawImage.get_chunkdata()

意义：获取帧信息数据。

返回值：帧信息数据。

3.3.7.21. get_user_param

声明：RawImage.get_user_param()

意义：获取用户通过 [register_buffer\(\)](#) 设置的用户参数，若未调用该接口，则返回值空。

返回值：返回用户自定义参数。

3.3.7.22. get_chunk_data_feature_control

声明：RawImage.get_chunk_data_feature_control()

意义：获取当前图像的帧信息属性控制器。

返回值：图像的帧信息属性控制器。

注意：不支持帧信息时调用该接口会抛出不支持异常。

3.3.7.23. convert(不再维护)

推荐用 [ImageFormatConvert.convert\(\)](#)或 [ImageFormatConvert.convert_ex\(\)](#)

声明： `RawImage.convert(mode, flip=False,
valid_bits=DxValidBit.BIT4_11,
convert_type=DxBayerConvertType.NEIGHBOUR,
channel_order=DxRGBChannelOrder.ORDER_RGB)`

意义： 图像格式转换。

- 1) 当 mode = 'RAW8' 模式时，将 16 位 raw 图转换为 8 位 raw 图，截取的有效位默认为当前像素格式的高 8 位。用户也可通过参数 valid_bits 手动设置有效位。仅支持 10/12 位的 Raw 图。
- 2) 当 mode = 'RGB' 模式时，将 raw 图转换为 RGB 图。如果输入为 10/12 位 raw 图，先转换为 8 位 raw 图，再转换为 RGB 图。

形参：

[in]	mode	'RAW8'：将 16 位 raw 图转换为 8 位 raw 图 'RGB'：将 raw 图转换为 RGB24 图
[in]	flip	输出的 RGB 图像是否上下翻转，缺省值为 False，该功能仅支持 mode = 'RGB' 模式
[in]	valid_bits	有效位数，缺省值为当前像素格式的高 8 位，参考 DxValidBit
[in]	convert_type	转换类型，缺省值为 DxBayerConvertType.NEIGHBOUR，参考 DxBayerConvertType ，仅对 mode = 'RGB' 模式有效
[in]	channel_order	图像通道顺序，缺省值为 DxRGBChannelOrder.ORDER_RGB，参考 DxRGBChannelOrder

返回值： RGB 图像对象

异常处理：

- 1) 如果帧信息状态不成功，则打印错误信息“ RawImage.convert:This is a incomplete image” ，函数返回 None。
- 2) 如果参数 1 不是字符串型，则抛异常 ParameterTypeError。
- 3) 如果参数 2 不是布尔型，则抛异常 ParameterTypeError。
- 4) 如果参数 3、4 不是整型，则抛异常 ParameterTypeError。
- 5) 如果参数 4 不在 [DxBayerConvertType](#) 中，则打印：提示参数越界、当前参数所支持的枚举值，函数返回 None。
- 6) 如果参数 4 不在 [DxValidBit](#) 中，则打印：提示参数越界、当前参数所支持的枚举值，函数返回 None。
- 7) 如果像素不是 8/10/12 位，则打印错误信息“ RawImage.convert:This pixel format is not support” ，函数返回 None。

- 8) 如果参数 1 为 'RAW8' 且参数 2 为 True，则打印错误信息" RawImage.convert:mode = 'RAW8' don't support flip = True"，函数返回 None。
- 9) mode = 'RAW8'，位深不是 10/12 位，则打印错误信息" RawImage.convert: mode="RAW8" only support 10bit and 12bit"，函数返回 None。
- 10) 如果参数 1 不为 'RAW8' 或 'RGB'，则打印接口名称和输入的 mode 不在范围内的信息，函数返回 None。

3.3.7.24. brightness(不再维护)

推荐用 [ImageProcessConfig.set_contrast_param\(\)](#) 设置，用 [ImageProcess.image_improvement\(\)](#) 处理。

声明： RawImage. brightness(factor)

意义： 对 8 位灰度图像进行亮度调节。

形参： [in] factor 亮度调节因子，值范围 -150~150。其中：
0：亮度没有变化；大于 0：增加亮度；小于 0：减小亮度；

返回值： None。

异常处理：

- 1) 如果输入参数不是整型，则抛出 ParameterTypeError 异常。
- 2) 如果图像类型不是 Mono8，则打印 "RawImage.brightness only support mono8 image"，抛出 InvalidParameter 异常。
- 3) 如果亮度调节失败，则抛出 UnexpetedError 异常。

3.3.7.25. contrast(不再维护)

推荐用 [ImageProcessConfig.set_lightness_param\(\)](#) 设置后用 [ImageProcess.image_improvement\(\)](#) 处理。

声明： RawImage.contrast(factor)

意义： 对 8 位灰度图像进行对比度调节。

形参： [in] factor 对比度调节因子，值范围 -50~150。其中：
0：对比度没有变化；大于 0：增加对比度；小于 0：减小对比度；

返回值： None。

异常处理：

- 1) 如果输入参数不是整型，则抛出 ParameterTypeError 异常。
- 2) 如果图像类型不是 MONO8，则打印 "RawImage.brightness only support mono8 image"，抛出 InvalidParameter 异常。
- 3) 如果对比度调节失败，则抛出 UnexpetedError 异常。

3.3.8. Buffer

负责 Buffer 类的操作。Buffer 类将在图像质量提升的部分使用，[Utility.get_gamma_lut\(gamma\)](#)和[Utility.get_contrast_lut\(contrast\)](#)接口返回的 Buffer 类型对象将作为参数传给[RGBImage.image_improvement\(color_correction_param=0, contrast_lut=None, gamma_lut=None\)](#)接口。

接口列表：

from_file()	从文件获取 Buffer 对象
from_string()	从字符串获取 Buffer 对象
get_data()	返回 Buffer 对象的字符串数据
get_ctype_array()	返回 Buffer 对象的数据数组
get_numpy_array()	返回 Buffer 对象的 numpy 数组
get_length()	返回 Buffer 对象的数据数组长度

3.3.8.1. from_file (静态函数)

声明： Buffer.from_file(file_name)

意义： 从文件获取 Buffer 对象。

形参： [in] file_name 文件路径

返回值： Buffer 对象。

3.3.8.2. from_string (静态函数)

声明： Buffer.from_string(string_data)

意义： 从字符串获取 Buffer 对象。

形参： [in] string_data 字符串

返回值： Buffer 对象。

3.3.8.3. get_data

声明： Buffer.get_data()

意义： 返回 Buffer 对象的字符串数据。

返回值： string_data 字符串数据。

注意： python2.7 返回字符串类型；python3.5 返回 bytes 类型。

3.3.8.4. get_ctype_array

声明： Buffer.get_ctype_array()

意义： 返回 Buffer 对象的数据数组。

返回值： Buffer 对象的数据数组【ctype 类型】

3.3.8.5. get_numpy_array

声明： Buffer.get_numpy_array()

意义：返回 Buffer 对象的 numpy 数组。

返回值：Buffer 对象的数据数组【numpy 类型】。

3.3.8.6. get_length

声明：Buffer.get_length()

意义：返回 Buffer 对象的数据数组长度。

返回值：数据数组长度。

3.3.9. ImageProcessConfig

负责图像处理属性配置。

接口列表：

set_valid_bits()	选择获取指定 8 位有效数据位，此接口设置指定哪 8 位
get_valid_bits()	查询当前指定的哪 8 位有效位
enable_defective_pixel_correct()	使能坏点校正
is_defective_pixel_correct()	查询当前是否使能坏点校正
enable_sharpen()	使能锐化
is_sharpen()	查询当前是否使能锐化
set_sharpen_param()	设置锐化强度因子
get_sharpen_param()	查询当前使用的锐化强度因子
set_contrast_param()	设置对比度调节参数
get_contrast_param()	查询当前使用的对比度调节参数
set_gamma_param()	设置 Gamma 系数
get_gamma_param()	获取 Gamma 系数
set_lightness_param()	设置亮度调节参数
get_lightness_param()	获取亮度调节参数
enable_denoise()	使能降噪
is_denoise()	查询当前是否使能降噪
set_saturation_param()	设置饱和度系数
get_saturation_param()	获取饱和度系数
set_convert_type()	设置图像格式转换算法
get_convert_type()	获取图像格式转换算法
enable_convert_flip()	使能图像格式转换翻转
is_convert_flip()	查询当前是否使能图像格式转换翻转
enable_accelerate()	使能加速图像处理

is_accelerate()	查询当前是否工作在加速使能状态
enable_color_correction()	使能颜色校正
is_color_correction()	查询是否使能颜色校正
enable_user_set_ccparam()	使能颜色校正用户设置模式
is_user_set_ccparam()	查询颜色校正是否用户设置模式
set_user_ccparam()	设置颜色转换因子结构体
get_user_ccparam()	获取颜色转换因子结构体
reset()	恢复默认调节参数

3.3.9.1. set_valid_bits

声明： set_valid_bits(valid_bits)

意义：选择获取指定 8 位有效数据位，此接口针对非 8 位原始数据设立。

形参： [in] param 有效位数，缺省值为当前像素格式的高 8 位，参考 [DxValidBit](#)

异常处理：如果设置有效位不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.9.2. get_valid_bits()

声明： get_valid_bits()

意义：获取指定 8 位有效数据位，此接口针对非 8 位原始数据设立。

返回值：有效数据位，详见 [DxValidBit](#)。

3.3.9.3. enable_defective_pixel_correct

声明： enable_defective_pixel_correct(enable)

意义：使能坏点校正。bEnable 为 true 则使能坏点校正；为 false 则禁用坏点校正。

形参： [in] enable 使能坏点校正。enable 为 true 则使能坏点校正；为 false 则禁用坏点校正。

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.9.4. is_defective_pixel_correct

声明： is_defective_pixel_correct()

意义：获取坏点校正状态。

返回值：坏点校正状态。

3.3.9.5. enable_sharpen

声明： enable_sharpen(enable)

意义：使能锐化。

形参： [in] enable 使能锐化，bEnable 为 true 则使能锐化；为 false 则禁用锐化

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.9.6. is_sharpen

声明 : is_sharpen()

意义 : 查询当前是否使能锐化。

返回值 : 使能锐化状态。

3.3.9.7. set_sharpen_param

声明 : set_sharpen_param(param)

意义 : 设置锐化强度因子。

形参 : [in] param 锐化强度因子。

异常处理 :

- 1) 如果输入参数不是浮点型, 则抛出 ParameterTypeError 异常。
- 2) 如果输入参数范围不在 0.1~5.0, 则抛出 UnexpectedError 异常。

3.3.9.8. get_sharpen_param

声明 : get_sharpen_param()

意义 : 查询当前使用的锐化强度因子。

返回值 : 返回锐化强度因子。

3.3.9.9. set_contrast_param

声明 : set_contrast_param(param)

意义 : 设置对比度调节参数。

形参 : [in] param 对比度调节参数。

异常处理 :

- 1) 如果输入参数不是整型, 则抛出 ParameterTypeError 异常。
- 2) 如果输入参数范围不在-50~100, 则抛出 UnexpectedError 异常。

3.3.9.10. get_contrast_param

声明 : get_contrast_param()

意义 : 查询当前使用的对比度调节参数。

返回值 : 返回对比度调节参数。

3.3.9.11. set_gamma_param

声明 : set_gamma_param(param)

意义 : 设置 Gamma 系数。

形参 : [in] param Gamma 系数。

异常处理 :

- 1) 如果输入参数不是整型、浮点型, 则抛出 ParameterTypeError 异常。
- 2) 如果输入参数范围不在-0.1~10.0, 则抛出 UnexpectedError 异常。

3.3.9.12. get_gamma_param

声明 : get_gamma_param()

意义 : 获取 Gamma 系数。

返回值 : 返回 Gamma 系数。

3.3.9.13. set_lightness_param

声明 : set_lightness_param(param)

意义 : 设置亮度调节参数。

形参 : [in] param 亮度调节参数

异常处理 :

- 1) 如果输入参数不是整型, 则抛出 ParameterTypeError 异常。
- 2) 如果输入参数范围不在-150~150, 则抛出 UnexpectedError 异常

3.3.9.14. get_lightness_param

声明 : get_lightness_param()

意义 : 获取亮度调节参数。

返回值 : 返回亮度调节参数。

3.3.9.15. enable_denoise

声明 : enable_denoise(enable)

意义 : 使能降噪。

形参 : [in] param 使能降噪开关 (黑白相机不支持)。enable 为 true 则使能降噪功能; 为 false 则禁用降噪功能。

异常处理 : 如果函数调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.9.16. is_denoise()

声明 : is_denoise()

意义 : 获取降噪开关 (黑白相机不支持)。

返回值 : 返回降噪开关标志。

3.3.9.17. set_saturation_param

声明 : set_saturation_param(param)

意义 : 设置饱和度调节系数 (黑白相机不支持)。

形参 : [in] param 饱和度调节系数。

异常处理 :

- 1) 如果输入参数不是整型、浮点型, 则抛出 ParameterTypeError 异常。
- 2) 如果输入参数范围不在 0~128, 则抛出 UnexpectedError 异常。

3.3.9.18. get_saturation_param

声明 : get_saturation_param()

意义：获取饱和度调节系数（黑白相机不支持）。

返回值：返回饱和度调节系数。

3.3.9.19. set_convert_type

声明：set_convert_type(cv_type)

意义：设置图像插值处理算法（黑白相机不支持）。

形参：[in] cv_type 转换类型，缺省值为 Dx BayerConvertType.NEIGHBOUR，参考 [Dx BayerConvertType](#)

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.9.20. get_convert_type

声明：get_convert_type()

意义：获取图像插值处理算法（黑白相机不支持）。

返回值：获取图像插值处理算法（黑白相机不支持），详见 [Dx BayerConvertType](#)。

3.3.9.21. enable_convert_flip

声明：enable_convert_flip(flip)

意义：使能插值翻转（黑白相机不支持）。

形参：[in] flip flip 为 true 则使能翻转；为 false 则禁用翻转功能。

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.9.22. is_convert_flip

声明：is_convert_flip()

意义：获取插值翻转标识（黑白相机不支持），true 表示翻转，false 表示不翻转。

返回值：返回插值翻转标识。

3.3.9.23. enable_accelerate

声明：enable_accelerate(accelerate)

意义：图像处理加速使能。设置 true 为使能加速，为 false 禁用加速。

形参：[in] accelerate accelerate 为 true 则使能加速；为 false 则禁用加速功能。

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.9.24. is_accelerate

声明：is_accelerate()

意义：查询当前是否工作在加速使能状态，true 表示加速，false 表示不加速。

返回值：加速使能状态。

3.3.9.25. enable_color_correction

声明：enable_color_correction(enable)

意义：使能颜色校正（黑白相机不支持），true 表示使能颜色校正，false 表示禁用颜色校正。

形参 : [in] enable enable 为 true 则使能颜色校正 ; 为 false 则禁用颜色校正功能。

异常处理 : 如果函数调用不成功 , 则抛出异常 , 异常类型详见[错误处理](#)。

3.3.9.26. is_color_correction

声明 : is_color_correction()

意义 : 查询是否使能颜色校正 (黑白相机不支持) 。

返回值 : 使能颜色校正状态。

3.3.9.27. enable_user_set_ccparam

声明 : enable_user_set_ccparam(enable)

意义 : 使能颜色校正用户设置模式 (黑白相机不支持) , true 表示用户模式 , false 表示非用户模式。

形参 : [in] enable enable 为 true 则使能颜色校正用户设置模式 ; 为 false 则禁用颜色校正用户设置模式功能

异常处理 : 如果函数调用不成功 , 则抛出异常 , 异常类型详见[错误处理](#)。

3.3.9.28. is_user_set_ccparam

声明 : is_user_set_ccparam()

意义 : 查询颜色校正是否用户设置模式 (黑白相机不支持) , true 表示用户模式 , false 表示非用户模式。

返回值 : 校正是否用户设置模式。

3.3.9.29. set_user_ccparam

声明 : set_user_ccparam(color_transform_factor)

意义 : 设置颜色转换因子结构体 (颜色校正结构体参数建议范围-4~4 , 如果参数设置小于-4 , 校正效果与-4 一致 , 如果参数设置大于 4 , 校正效果与 4 一致) 。

形参 : [in] color_transform_factor 设置颜色转换因子结构体。

异常处理 : 如果输入参数不是 [ColorTransformFactor](#) 型 , 则抛出 ParameterTypeError 异常。

3.3.9.30. get_user_ccparam

声明 : get_user_ccparam()

意义 : 获取颜色转换因子结构体。

返回值 : 转换因子结构体。

3.3.9.31. reset

声明 : reset()

意义 : 恢复最佳默认参数配置。

3.3.10. Utility

负责参数 gamma 和 contrast 的操作。

接口列表 :

[get_gamma_lut\(\)](#) 通过 gamma 值获取 gamma 查找表的 Buffer 类型对象

<code>get_contrast_lut()</code>	通过对比度值获取对比度查找表的 Buffer 类型对象
<code>get_lut()</code>	计算图像处理 8 位查找表
<code>calc_cc_param()</code>	计算图像处理色彩调节数组
<code>calc_user_set_cc_param()</code>	根据用户设置计算图像处理色彩调节数组

3.3.10.1. `get_gamma_lut` (静态函数)

声明： `Utility.get_gamma_lut(gamma=1)` (静态函数)

意义：通过 `gamma` 值获取 `gamma` 查找表的 Buffer 类型对象。

形参： [in] `gamma` 整型或浮点型，范围【0.1，10.0】，缺省值为 1

返回值：`gamma` 查找表的 Buffer 类型对象。

异常处理：

- 1) 如果输入参数不是整型或浮点型，则抛出 `ParameterTypeError` 异常。
- 2) 如果输入参数不在 0.1~10.0 范围内，则打印错误信息“ `Utility.get_gamma_lut:gamma out of bounds, range:[0.1, 10.0]`”，函数返回 `None`。
- 3) 如果获取 `gamma` 查找表失败，则打印接口名称、获取 `gamma lut` 失败和错误码的信息，函数返回 `None`。

3.3.10.2. `get_contrast_lut` (静态函数)

声明： `Utility.get_contrast_lut(contrast=0)` (静态函数)

意义：通过对比度值获取对比度查找表的 Buffer 类型对象。

形参： [in] `contrast` 整型，范围【-50，100】，缺省值为 0

返回值：对比度查找表的 Buffer 类型对象。

异常处理：

- 1) 如果输入参数不是整型，则抛出 `ParameterTypeError` 异常。
- 2) 如果输入参数不在 -50~100 范围内，则打印错误信息“ `Utility.get_contrast_lut:contrast out of bounds, range:[-50, 100]`”，函数返回 `None`。
- 3) 如果获取对比度查找表失败，则打印接口名称、获取 `contrast lut` 失败和错误码的信息，函数返回 `None`。

3.3.10.3. `get_lut` (静态函数)

声明： `Utility.get_lut(contrast=0, gamma=1, lightness=0)` (静态函数)

意义：计算图像处理 8 位查找表。

形参：

[in]	<code>contrast</code>	对比度调节参数，整型，范围【-50，100】，缺省值为 0
[in]	<code>gamma</code>	调节参数，浮点型，范围【0.1，10】，缺省值为 1

[in] lightness 亮度调节参数，整型，范围【-150，150】，缺省值为 0

返回值：查找表的 Buffer 类型对象。

异常处理：

- 1) 如果 contrast/gamma/lightness 不是整型/浮点型/整型，则抛出 ParameterTypeError 异常。
- 2) 如果获取查找表失败，则打印接口名称、获取 lut 失败和错误码的信息，函数返回 None。

3.3.10.4. calc_cc_param (静态函数)

声明：Utility.calc_cc_param(color_correction_param, saturation=64) (静态函数)

意义：计算图像处理色彩调节数组。

形参：

[in] color_correction_param 颜色校正值，整型，可以从相机获取或者设为 0

[in] saturation 饱和度调节参数，整型，范围【0，128】，缺省值为 64

返回值：色彩调节参数的 Buffer 类型对象。

异常处理：

- 1) 如果输入参数不是整型，则抛出 ParameterTypeError 异常。
- 2) 如果计算色彩调节参数失败，则打印接口名称、获取调节参数失败和错误码的信息，函数返回 None。

3.3.10.5. calc_user_set_cc_param (静态函数)

声明：Utility.calc_user_set_cc_param(color_transform_factor, saturation=64) (静态函数)

意义：根据用户设置计算图像处理色彩调节数组。

形参：

[in] color_transform_factor 颜色校正/颜色转换数组，列表或者元组类型

[in] saturation 饱和度调节参数，整型，范围【0，128】，缺省值为 64

返回值：色彩调节参数的 Buffer 类型对象。

异常处理：

- 1) 如果输入参数不是列表（元组）、整型，则抛出 ParameterTypeError 异常。
- 2) 如果计算色彩调节参数失败，则打印接口名称、获取调节参数失败和错误码的信息，函数返回 None

3.3.11. ImageProcess

对当前图像做图像效果增强，返回效果增强之后的图像数据

注意：

- 1) 如果 [ImageProcessConfig.is_accelerate\(\)](#) 返回 true，说明此时将以加速方式处理图像，加速方式处理图像需要图像的高度必须是 4 的整倍数，否则接口报错；
- 2) 黑白相机 void* 返回的是 8bit 图像数据，图像大小为图像宽×图像高；

3) 彩色相机 void*返回的是 RGB 图像数据，图像大小为图像宽×图像高×3。

接口列表：

image_improvement()	图像质量提升处理
static_defect_correction()	图像静态坏点矫正
calcula_lut()	计算相机查找表
read_lut_file()	读取查找表文件

3.3.11.1. image_improvement

声明： image_improvement(image, output_address, image_process_config)

意义：对输入的图像做图像质量提升，注：Bayer 格式的图片图像质量提升处理之后的格式为 RGB。

形参：

[in]	image	源图像（可变类型），类型为 RawImage，可通过 DataStream.get_image() 、 DataStream.dq_buf() 获得，类型为 GxImageInfo
[out]	output_address	返回图像质量提升后的图像 Buffer
[in]	image_process_config	图像质量提升辅助配置对象 可控制图像锐化、亮度等信息。

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.11.2. static_defect_correction

声明： static_defect_correction(input_address, output_address, defect_correction, defect_pos_buffer_address, defect_pos_buffer_size)

意义：对输入的图像做静态坏点矫正。

形参：

[in]	input_address	要矫正的原始图片缓存地址
[out]	output_address	存放输出图像缓存地址
[in]	defect_correction	图像静态坏点矫正参数
[in]	defect_pos_buffer_address	检测静态坏点文件 buffer
[in]	defect_pos_buffer_size	检测静态坏点文件 buffer 位置

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.11.3. calcula_lut

声明： calcula_lut(contrast_param, gamma, light_ness, lut_address, lut_length_address)

意义：计算查找表。

形参：

[in]	contrast_param	contrast 参数
[in]	gamma	gamma 参数
[in]	light_ness	亮度

[out]	lut_address	查找表地址
[out]	lut_length_address	查找表长度地址

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.11.4. read_lut_file

声明：read_lut_file(lut_file_path, lut_address, lut_length_address)

意义：从文件读取查找表到程序内存。

形参：

[in]	lut_file_path	查找表文件路径
[out]	lut_address	要存放的查找表地址
[out]	lut_length_address	查找表长度

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12. ImageFormatConvert

负责图像格式转换，可通过此对象转换图像格式。

接口列表：

set_dest_format()	设置目标像素格式
get_dest_format()	获取目标像素格式
set_interpolation_type()	设置转换算法
get_interpolation_type()	获取转换算法
set_alpha_value()	设置 Alpha 通道
get_alpha_value()	获取 Alpha 通道
set_valid_bits()	设置要转换像素格的有效位
get_valid_bits()	获取要转换像素格的有效位
get_buffer_size_for_conversion_ex()	获取转换像素格式对应的 Buffer 的大小
get_buffer_size_for_conversion()	获取转换像素格式对应的 Buffer 的大小
convert_ex()	转换像素格式
convert()	转换像素格式
set_3d_calib_param()	该函数用于加载 3D 标定参数，将高度图转换为点云图
get_3d_calib_param()	获取 3D 标定参数
set_y_step(y_step)	设置高度图转点云的 Y 方向步长
get_y_step()	获取高度转点云的 Y 方向步长

3.3.12.1. set_dest_format

声明：set_dest_format(dest_pixel_format)

意义：设置期望转换格式，详见 [GxPixelFormatEntry](#)。

形参：[in] dest_pixel_format 期望的转换格式，详见

[GxPixelFormatEntry 支持转换类型](#)

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.2. get_dest_format

声明：get_dest_format()

意义：获取期望转换格式。

返回值：返回期望转换格，详见 [GxPixelFormatEntry](#)。

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.3. set_interpolation_type

声明：set_interpolation_type(cvt_type)

意义：设置图像格式转换算法。

形参：[in] cvt_type 转换算法，详见 [DxBayerConvertType](#)

默认值为：[DxBayerConvertType.NEIGHBOUR](#)

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.4. get_interpolation_type

声明：get_interpolation_type()

意义：获取图像格式转换算法。

返回值：返回图像格式转换算法，详见 [DxBayerConvertType](#)。

3.3.12.5. set_alpha_value

声明：set_alpha_value(alpha_value)

意义：设置带有 Alpha 通道图像的 Alpha 值。

形参：[in] alpha_value Alpha 通道值，取值范围 0~255，默认值为：255。

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.6. get_alpha_value

声明：get_alpha_value()

意义：获取图像格式转换算法。

返回值：获取 Alpha 通道值。

3.3.12.7. set_valid_bits

声明：set_valid_bits(valid_bits)

意义：设置转换有效位。

形参：[in] valid_bits 有效位数，缺省值为当前像素格式的高 8 位，参考 [DxValidBit](#)

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.8. get_valid_bits

声明：get_valid_bits()

意义：获取转换有效位。

返回值：有效位，详见 [DxValidBit](#)。

3.3.12.9. get_buffer_size_for_conversion_ex

声明：get_buffer_size_for_conversion_ex(width, height, pixel_format)

意义：根据输入图像数据获取期望格式的图像 Buffer 长度。

形参：

[in]	width	源图像宽。
[in]	height	源图像高。
[in]	pixel_format	源图像格式。

返回值：期望格式的图像 Buffer 长度。

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.10. get_buffer_size_for_conversion

声明：get_buffer_size_for_conversion(raw_image)

意义：根据输入图像指针获取期望格式的图像 Buffer 长度。

形参：[in] raw_image 源图像，可通过 [DataStream.get_image\(\)](#)、
[DataStream.dq_buf\(\)](#)获得

返回值：期望格式的图像 Buffer 长度。

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.11. convert_ex

声明：convert_ex(input_address, input_width,
input_height, src_fixel_format,
output_address, output_length, flip)

意义：将输入源图像转换为期望的图像格式，可转换类型参考 [GxPixelFormatEntry](#)

形参：

[in]	input_address	源图像 Buffer 指针
[in]	input_width	源图像宽度
[in]	input_height	源图像高度
[in]	src_fixel_format	源图像格式
[out]	output_address	目的图像 Buffer 指针，该内存由用户申请，长度为 output_length
[in]	output_length	目的图像 Buffer 长度，可通过 get_buffer_size_for_conversion_ex() 获得
[in]	flip	图像是否翻转（true-翻转，false-不翻转）

异常处理：如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.12. convert

声明：convert(raw_image, output_address, output_length, flip)

意义：将输入源图像转换为期望的图像格式，可转换类型参考 [GxPixelFormatEntry](#)

形参：

[in]	raw_image	源图像，可通过 DataStream.get_image() 、 DataStream.dq_buf() 获得
[in]	output_address	目的图像 Buffer 指针，该内存由用户申请，长度为 output_length
[in]	output_length	目的图像 Buffer 长度，可通过 get_buffer_size_for_conversion() 获得
[in]	flip	图像是否翻转（true-翻转，false-不翻转）

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.13. set_3d_calib_param

声明：set_3d_calib_param(self, buffer_address, buffer_size)

意义：该函数用于加载 3D 标定参数，将高度图转换为点云图。

形参：

[in]	pCalibParamBuffer	3D 标定参数内存
[in]	nBufferSize	3D 标定参数内存长度

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.14. get_3d_calib_param

声明：get_3d_calib_param()

意义：获取 3D 标定参数。

形参：无

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.15. set_y_step

声明：set_y_step(y_step)

意义：设置高度图转点云的 Y 方向步长。

形参：[in] dY Y 方向步长

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.12.16. get_y_step

声明：get_y_step()

意义：获取高度转点云的 Y 方向步长。

形参：无

异常处理：如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.13. FlatFieldCorrection

平场校正功能类，可通过此对象获取图像平场校正系数，并根据此系数获取平场校正后的图像。具体是使用方式参考 GxFlatFieldCorrection 示例程序。

接口列表：

set_frame_count	设置融合帧数
get_coefficients_size	获取平场校正系数大小
calculate	计算平场校正系数
flat_field_correction	应用平场校正系数获取平场校正后的图像

3.3.13.1. set_frame_count

声明： set_frame_count(frame_count)

意义： 设置融合帧数，该接口需在 Calculate 接口之前调用。

形参： [in] frame_count 平场校正融合帧数(取值范围 1、2、4、8、16)
表明计算平场系数时融合的图像数，
如果图像噪声较大，建议选择大的值。

异常处理： 如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.13.2. get_coefficients_size

声明： get_coefficients_size(ffc_param)

意义： 获取平场系数大小。该接口需在 [calculate](#) 接口之前调用，用于获取系数大小之后分配平场系数缓存。

形参： [in] ffc_param 平场校正参数，构造方式参见 [FlatFieldCorrectionParameter](#)

返回值： 返回平场系数大小。

异常处理： 如果函数调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.13.3. calculate

声明： calculate(ffc_param, coefficients_buffer, coefficients_buffer_size)

意义： 计算平场校正系数。

形参： [in] ffc_param 平场校正参数，构造方式参见 [FlatFieldCorrectionParameter](#)
[out] coefficients_buffer 平场校正系数缓存
[in] coefficients_buffer_size 平场校正系数缓存大小，通过调用 [get_coefficients_size](#)
接口获取

得到平场系数后，可以通过功能节点“FFCValueAll”先查询相机是否支持设置平场，如果支持则可以直接通过该节点设置给相机，如果不支持则可以通过调用接口 [flat_field_correction](#) 获取每帧图像平场之后的图像。

异常处理： 如果调用不成功，则抛出异常，异常类型详见[错误处理](#)。

3.3.13.4. flat_field_correction

声明： flat_field_correction(input_address, output_address, actual_bits, width, height, coefficients_buffer, coefficients_buffer_size)

意义：应用平场校正系数获取平场校正后的图像。

形参：

[in]	input_address	源图像, 通过 DataStream.get_image() 、 DataStream.dq_buf() 获得
[out]	output_address	平场后图像, 通过 DataStream.get_image() 、 DataStream.dq_buf() 获得
[in]	actual_DxValidBit	源图像有效位, 见 DxActualBits
[in]	width	当前图像的宽度, 通过图像对象中的 RawImage().get_width 接口获取
[in]	height	当前图像的高度, 通过图像对象中 RawImage().get_height 接口获取
[in]	coefficients_buffer	平场校正系数, 通过调用 calculate 获取
		长度为 coefficients_buffer_size
[in]	coefficients_buffer_size	平场校正系数长度, 通过调用 get_coefficients_size 接口获取

返回值：返回平场处理后的图像。

3.3.14. Decompressor

图像解压功能类, 可通过此对象将压缩数据解压成裸数据。具体使用方式参考 GxDecompression 示例程序。

接口列表：

[decompression](#) 将压缩数据解压成裸数据

3.3.14.1. decompression

声明：decompression(compression_address, compression_address_size, decompression_address, decompression_address_size, img_pixel_format, img_width, img_height, compression_method)

意义：将获取到的压缩数据解压成裸数据（包含帧信息）。

形参：

[in]	compression_address	压缩图像 buffer 地址
[in]	compression_address_size	压缩图像 buffer 长度
[in out]	decompression_address	解压图像 buffer 地址
[in out]	decompression_address_size	解压图像 buffer 长度
[in]	img_pixel_format	图像像素格式
[in]	img_width	图像宽度
[in]	img_height	图像高度
[in]	compression_method	压缩方法

异常处理：如果调用不成功, 则抛出异常, 异常类型详见[错误处理](#)。

3.3.15. MediaProc

图像存储功能类,可通过此对象将图像数据保存为本地图片或者视频。具体使用方式参考快速上手中存图存视频示例程序。

接口列表：

save_image	将图像数据保存为本地图片
create_video_saver	创建视频保存实例对象

3.3.15.1. save_image

声明： save_image(self, stSaveImageInfo)

意义： 将图像数据保存为本地图片。

形参： [in] stSaveImageInfo 保存图像信息

3.3.15.2. create_video_saver

声明： create_video_saver(self, record_param)

意义： 创建视频保存实例对象。

形参： [in] record_param 图像录制参数

3.3.16. VideoSaver

视频存储功能类,可通过此对象将图像数据保存视频。具体使用方式参考快速上手中存图存视频示例程序。

接口列表：

__init__	初始化录制参数
add_frame	添加一帧图像
close	关闭录制

3.3.16.1. __init__

声明： __init__(self, record_param)

意义： 初始化录制参数。

形参： [in] record_param 录制参数

3.3.16.2. add_frame

声明： add_frame(self, image_buffer)

意义： 添加一帧图像。

形参： [in] image_buffer 图像数据

3.3.16.3. close

声明： close(self)

意义： 关闭录制。

3.4. 属性参数（不再维护，推荐使用 [Feature_s](#)）

3.4.1. 设备属性参数

属性参数	解释	属性类
DeviceInformation Section		
DeviceVendorName	厂商名称	StringFeature
DeviceModelName	设备型号	StringFeature
DeviceFirmwareVersion	设备固件版本	StringFeature
DeviceVersion	设备版本	StringFeature
DeviceSerialNumber	设备序列号	StringFeature
FactorySettingVersion	出厂参数版本	StringFeature
DeviceUserID	用户自定义名称	StringFeature
DeviceLinkSelector	设备链路选择，详见： C 软件开发说明书	IntFeature
DeviceLinkThroughputLimit	设备链路带宽限制	IntFeature
DeviceLinkThroughputLimitMode	设备带宽限制模式，详见： GxSwitchEntry	EnumFeature
DeviceLinkCurrentThroughput	当前设备采集带宽	IntFeature
DeviceReset	设备复位	CommandFeature
TimestampTickFrequency	时间戳时钟频率	IntFeature
TimestampLatch	时间戳锁存	CommandFeature
TimestampReset	重置时间戳	CommandFeature
TimestampLatchReset	重置时间戳锁存	CommandFeature
TimestampLatchValue	时间戳锁存值	IntFeature
DevicePHYVersion	设备网络芯片版本	StringFeature
DeviceTemperatureSelector	设备温度选择，详见 GxDeviceTemperatureSelectorEntry	EnumFeature
DeviceTemperature	设备温度	FloatFeature
ImageFormat Section		
SensorWidth	传感器宽度	IntFeature

SensorHeight	传感器高度	IntFeature
WidthMax	最大宽度	IntFeature
HeightMax	最大高度	IntFeature
OffsetX	水平偏移	IntFeature
OffsetY	垂直偏移	IntFeature
Width	图像宽度	IntFeature
Height	图像高度	IntFeature
BinningHorizontal	水平像素 Binning	IntFeature
BinningVertical	垂直像素 Binning	IntFeature
DecimationHorizontal	水平像素抽样	IntFeature
DecimationVertical	垂直像素抽样	IntFeature
PixelSize	像素位深, 详见 GxPixelSizeEntry	EnumFeature
PixelColorFilter	Bayer 格式, 详见 : GxPixelColorFilterEntry	EnumFeature
PixelFormat	像素格式, 详见 : GxPixelFormatEntry	EnumFeature
ReverseX	水平翻转	BoolFeature
ReverseY	垂直翻转	BoolFeature
TestPattern	测试图, 详见 GxTestPatternEntry	EnumFeature
TestPatternGeneratorSelector	测试图源选择, 详见 : C 软件开发说明书和 GxTestPatternGeneratorSelectorEntry	EnumFeature
RegionSendMode	ROI 输出模式, 详见 GxRegionSendModeEntry	EnumFeature
RegionMode	区域开关, 详见 GxSwitchEntry	EnumFeature
RegionSelector	区域选择, 详见 : C 软件开发说明书和 GxRegionSelectorEntry	EnumFeature
CenterWidth	窗口宽度	IntFeature
CenterHeight	窗口高度	IntFeature
BinningHorizontalMode	水平像素 Binning 模式, 详见 : GxBinningHorizontalModeEntry	EnumFeature

BinningVerticalMode	垂直像素 Binning 模式，详见： GxBinningVerticalModeEntry	EnumFeature
SensorShutterMode	Sensor 曝光时间模式，详见： GxSensorShutterModeEntry	EnumFeature
TransportLayer Section		
PayloadSize	数据大小	IntFeature
CenterWidth	窗口宽度	IntFeature
CenterHeight	窗口高度	IntFeature
GevCurrentIPConfigurationLLA	LLA 方式配置 IP	BoolFeature
GevCurrentIPConfigurationDHCP	DHCP 方式配置 IP	BoolFeature
GevCurrentIPConfigurationPersistentIP	永久 IP 方式配置 IP	BoolFeature
EstimatedBandwidth	预估带宽	IntFeature
GevHeartbeatTimeout	心跳超时时间	IntFeature
GevSCPSPacketSize	流通道包长	IntFeature
GevSCPD	流通道包间隔	IntFeature
GevLinkSpeed	连接速度	IntFeature
DigitalIO Section		
UserOutputSelector	用户自定义输出选择，详见： C 软件开发说明书和 GxUserOutputSelectorEntry	EnumFeature
UserOutputValue	用户自定义输出值	BoolFeature
UserOutputMode	用户 IO 输出模式，详见： GxUserOutputModeEntry	EnumFeature
StrobeSwitch	闪光灯开关，详见： GxSwitchEntry	EnumFeature
LineSelector	引脚选择，详见： C 软件开发说明书和 GxLineSelectorEntry	EnumFeature
LineMode	引脚方向，详见 GxLineModeEntry	EnumFeature
LineSource	引脚输出源，详见： GxLineSourceEntry	EnumFeature
LineInverter	引脚电平反转	BoolFeature
LineStatus	引脚状态	BoolFeature

LineStatusAll	所有引脚的状态	IntFeature
AnalogControls Section		
GainAuto	自动增益，详见： GxAutoEntry	EnumFeature
GainSelector	增益通道选择，详见： C 软件开发说明书和 GxGainSelectorEntry	EnumFeature
BlackLevelAuto	自动黑电平，详见： GxAutoEntry	EnumFeature
BlackLevelSelector	黑电平通道选择，详见： C 软件开发说明书和 GxBlackLevelSelectEntry	EnumFeature
BalanceWhiteAuto	自动白平衡，详见： GxAutoEntry	EnumFeature
BalanceRatioSelector	白平衡通道选择，详见： C 软件开发说明书和 GxBalanceRatioSelectorEntry	EnumFeature
BalanceRatio	白平衡系数	FloatFeature
DeadPixelCorrect	坏点校正，详见： GxSwitchEntry	EnumFeature
Gain	增益	FloatFeature
BlackLevel	黑电平	FloatFeature
GammaEnable	Gamma 使能	BoolFeature
GammaMode	Gamma 模式，详见： GxGammaModeEntry	EnumFeature
Gamma	Gamma	FloatFeature
DigitalShift	数字移位	IntFeature
LightSourcePreset	环境光源预设，详见： GxLightSourcePresetEntry	EnumFeature
CustomFeature Section		
ADCLevel	AD 转换级别	IntFeature
HBlanking	水平消隐	IntFeature
VBlanking	垂直消隐	IntFeature
UserPassword	用户加密区密码	StringFeature
VerifyPassword	用户加密区校验密码	StringFeature
UserData	用户加密区内容	BufferFeature

ExpectedGrayValue	期望灰度值	IntFeature
AALightEnvironment	自动曝光、自动增益，光照环境类型，详见： GxAALightEnvironmentEntry	EnumFeature
ImageGrayRaiseSwitch	图像亮度拉伸开关，详见： GxSwitchEntry	EnumFeature
AAROIOffsetX	自动调节感兴趣区域 X 坐标	IntFeature
AAROIOffsetY	自动调节感兴趣区域 Y 坐标	IntFeature
AAROIWidth	自动调节感兴趣区域宽度	IntFeature
AAROIHeight	自动调节感兴趣区域高度	IntFeature
AutoGainMin	自动增益最小值	FloatFeature
AutoGainMax	自动增益最大值	FloatFeature
AutoExposureTimeMin	自动曝光最小值	FloatFeature
AutoExposureTimeMax	自动曝光最大值	FloatFeature
ContrastParam	对比度参数	IntFeature
ColorCorrectionParam	颜色校正系数	IntFeature
AWBROIOffsetX	自动白平衡感兴趣区域 X 坐标	IntFeature
AWBROIOffsetY	自动白平衡感兴趣区域 Y 坐标	IntFeature
AWBROIWidth	自动白平衡感兴趣区域宽度	IntFeature
AWBROIHeight	自动白平衡感兴趣区域高度	IntFeature
GammaParam	伽马参数	FloatFeature
AWBLampHouse	自动白平衡光源，详见： GxAWBLampHouseEntry	EnumFeature
SharpnessMode	锐化模式，详见： GxSwitchEntry	EnumFeature
Sharpness	锐度	FloatFeature
FrameInformation	图像帧信息	BufferFeature
DataFieldSelector	用户选择 Flash 数据区域，详见： GxUserDataFieldSelectorEntry	EnumFeature
DataFieldValue	用户区内容	BufferFeature
FlatFieldCorrection	平场校正开关，详见 GxSwitchEntry	EnumFeature
NoiseReductionMode	降噪开关，详见： GxSwitchEntry	EnumFeature

NoiseReduction	降噪	FloatFeature
FFCLoad	获取平场校正参数	BufferFeature
FFCSave	设置平场校正参数	BufferFeature
StaticDefectCorrection	静态坏点校正开关，详见： GxSwitchEntry	EnumFeature
UserSetControl Section		
UserSetLoad	加载参数组	CommandFeature
UserSetSave	保存参数组	CommandFeature
UserSetSelector	参数组选择，详见 C 软件开发说明书 和 GxUserSetEntry	EnumFeature
UserSetDefault	启动参数组，详见 GxUserSetEntry	EnumFeature
Event Section		
EventSelector	事件源选择，详见： GxEventSelectorEntry	EnumFeature
EventNotification	事件使能开关，详见 GxSwitchEntry	EnumFeature
EventExposureEnd	曝光结束事件 ID	IntFeature
EventExposureEndTimestamp	曝光结束事件时间戳	IntFeature
EventExposureEndFrameID	曝光结束事件帧 ID	IntFeature
EventBlockDiscard	数据块丢失事件 ID	IntFeature
EventBlockDiscardTimestamp	数据块丢失事件时间戳	IntFeature
EventOverrun	事件队列溢出事件 ID	IntFeature
EventOverrunTimestamp	事件队列溢出事件时间戳	IntFeature
EventFrameStartOvertrigger	触发信号被屏蔽事件 ID	IntFeature
EventFrameStartOvertriggerTimestamp	触发信号被屏蔽事件时间戳	IntFeature
EventBlockNotEmpty	帧存不为空事件 ID	IntFeature
EventBlockNotEmptyTimestamp	帧存不为空事件时间戳	IntFeature
EventInternalError	内部错误事件 ID	IntFeature
EventInternalErrorTimestamp	内部错误事件时间戳	IntFeature
EventFrameBurstStartOvertrigger	多帧触发屏蔽事件 ID	IntFeature
EventFrameBurstStartOvertriggerFrameID	多帧触发屏蔽事件帧 ID	IntFeature

EventFrameBurstStartOvertriggerTimestamp	多帧触发屏蔽事件时间戳	IntFeature
EventFrameStartWait	帧等待事件 ID	IntFeature
EventFrameStartWaitTimestamp	帧等待事件时间戳	IntFeature
EventFrameBurstStartWait	多帧等待事件 ID	IntFeature
EventFrameBurstStartWaitTimestamp	多帧等待事件时间戳	IntFeature
EventBlockDiscardFrameID	数据块丢失事件帧 ID	IntFeature
EventFrameStartOvertriggerFrameID	触发信号被屏蔽事件帧 ID	IntFeature
EventBlockNotEmptyFrameID	帧存不为空事件帧 ID	IntFeature
EventFrameStartWaitFrameID	帧等待事件帧 ID	IntFeature
EventFrameBurstStartWaitFrameID	多帧等待事件帧 ID	IntFeature
LUT Section		
LUTValueAll	查找表内容	BufferFeature
LUTSelector	查找表选择，详见 C 软件开发说明书和 GxLutSelectorEntry	EnumFeature
LUTEnable	查找表使能	BoolFeature
LUTIndex	查找表索引	IntFeature
LUTValue	查找表值	IntFeature
Color Transformation Control		
ColorTransformationMode	颜色转换模式，详见： GxColorTransformationModeEntry	EnumFeature
ColorTransformationEnable	颜色转换使能	BoolFeature
ColorTransformationValueSelector	颜色转换矩阵元素选择，详见： GxColorTransformationValueSelectorEntry	EnumFeature
ColorTransformationValue	颜色转换矩阵元素	FloatFeature
SaturationMode	饱和度模式开关，详见： GxSwitchEntry	EnumFeature
Saturation	饱和度	IntFeature
ChunkData Section		
ChunkModeActive	帧信息使能	BoolFeature
ChunkEnable	单项帧信息使能	BoolFeature

ChunkSelector	帧信息项选择，详见 C 软件开发说明书和 GxChunkSelectorEntry	EnumFeature
Device Feature		
DeviceCommandTimeout	命令超时	IntFeature
DeviceCommandRetryCount	命令重试次数	IntFeature
AcquisitionTrigger Section		
FrameBufferOverwriteActive	帧存覆盖使能	BoolFeature
AcquisitionStart	开始采集	CommandFeature
AcquisitionStop	停止采集	CommandFeature
TriggerSoftware	软触发	CommandFeature
TransferStart	开始传输	CommandFeature
AcquisitionMode	采集模式，详见： GxAcquisitionModeEntry	EnumFeature
TriggerMode	触发模式，详见 GxSwitchEntry	EnumFeature
TriggerActivation	触发极性，详见： GxTriggerActivationEntry	EnumFeature
ExposureAuto	自动曝光，详见	EnumFeature
TriggerSource	触发源，详见 GxTriggerSourceEntry	EnumFeature
ExposureMode	曝光模式，详见： GxExposureModeEntry	EnumFeature
TriggerSelector	触发类型选择，详见 C 软件开发说明书和 GxTriggerSelectorEntry	EnumFeature
TransferControlMode	传输控制模式，详见： GxTransferControlModeEntry	EnumFeature
TransferOperationMode	传输操作模式，详见： GxTransferOperationModeEntry	EnumFeature
AcquisitionFrameRateMode	采集帧率调节模式，详见： GxSwitchEntry	EnumFeature
FixedPatternNoiseCorrectMode	模板噪声校正，详见 GxSwitchEntry	EnumFeature
ExposureTime	曝光时间	FloatFeature
TriggerFilterRaisingEdge	上升沿触发滤波	FloatFeature
TriggerFilterFallingEdge	下降沿触发滤波	FloatFeature
TriggerDelay	触发延迟	FloatFeature

AcquisitionFrameRate	采集帧率	FloatFeature
CurrentAcquisitionFrameRate	当前采集帧率	FloatFeature
TransferBlockCount	传输帧数	IntFeature
TriggerSwitch	外触发开关，详见 GxSwitchEntry	EnumFeature
AcquisitionSpeedLevel	采集速度级别	IntFeature
AcquisitionFrameCount	多帧采集帧数	IntFeature
AcquisitionBurstFrameCount	高速连拍帧数	IntFeature
AcquisitionStatusSelector	采集状态选择，详见： C 软件开发说明书和 GxAcquisitionStatusSelectorEntry	EnumFeature
AcquisitionStatus	采集状态	BoolFeature
ExposureDelay	曝光延迟	FloatFeature
ExposureOverlapTimeMax	交叠曝光时间最大值	FloatFeature
ExposureTimeMode	曝光时间模式，详见： GxExposureTimeModeEntry	EnumFeature
CounterAndTimerControl Section		
TimerSelector	计时器选择，详见： GxTimerSelectorEntry	EnumFeature
TimerDuration	计时器持续时间	FloatFeature
TimerDelay	计时器延迟	FloatFeature
TimerTriggerSource	计时器触发源，详见： GxTimerTriggerSourceEntry	EnumFeature
CounterSelector	计数器选择，详见： GxCounterSelectorEntry	EnumFeature
CounterEventSource	计数器事件触发源，详见： GxCounterEventSourceEntry	EnumFeature
CounterResetSource	计数器复位源，详见： GxCounterResetSourceEntry	EnumFeature
CounterResetActivation	计数器复位信号极性，详见： GxCounterResetActivationEntry	EnumFeature
CounterReset	计数器复位	CommandFeature
CounterTriggerSource	计数器触发源，详见： GxCounterTriggerSourceEntry	EnumFeature
CounterDuration	计数器持续时间	IntFeature

TimerTriggerActivation	计时器触发极性，详见： GxTimerTriggerActivationEntry	EnumFeature
RemoveParameterLimitControl Section		
RemoveParameterLimit	取消参数范围限制开关，详见： GxSwitchEntry	EnumFeature

3.4.2. 流属性参数

属性参数	解释	属性类
StreamAnnouncedBufferCount	声明的 Buffer 个数	IntFeature
StreamDeliveredFrameCount	接收帧个数(包括残帧)	IntFeature
StreamLostFrameCount	buffer 不足导致的丢帧个数	IntFeature
StreamIncompleteFrameCount	接收的残帧个数	IntFeature
StreamDeliveredPacketCount	接收到的包数	IntFeature
StreamResendPacketCount	重传包个数	IntFeature
StreamRescuedPacketCount	重传成功包个数	IntFeature
StreamResendCommandCount	重传命令次数	IntFeature
StreamUnexpectedPacketCount	异常包个数	IntFeature
MaxPacketCountInOneBlock	数据块最大重传包数	IntFeature
MaxPacketCountInOneCommand	一次重传命令最大包含的包数	IntFeature
ResendTimeout	重传超时时间	IntFeature
MaxWaitPacketCount	最大等待包数	IntFeature
ResendMode	重传模式，详见 GxSwitchEntry	EnumFeature
StreamMissingBlockIDCount	BlockID 丢失个数	IntFeature
BlockTimeout	数据块超时时间	IntFeature
MaxNumQueueBuffer	采集队列最大 Buffer 个数	IntFeature
PacketTimeout	包超时时间	IntFeature
StreamTransferSize	传输数据块大小	IntFeature
StreamTransferNumberUrb	传输数据块数量	IntFeature
SocketBufferSize	套接字缓冲区大小	IntFeature
StopAcquisitionMode	停采模式，详见： GxStopAcquisitionModeEntry	EnumFeature
StreamBufferHandlingMode	Buffer 处理模式，详见： GxDSSStreamBufferHandlingModeEntry	EnumFeature

4. 常见问题解答

序号	常见问题	解决办法
1	程序运行中出现如下错误： <pre>" NotInitApi: DeviceManager.update_device_list: {-13}{Not init API}"</pre>	1) 请检查并删除程序中调用 DeviceManager 类对象的 <code>__del__()</code> 函数的语句。因为 Python 的垃圾回收机制会自动调用 <code>__del__()</code> 函数销毁对象，所以不需要、不允许用户显示调用 <code>__del__()</code> 函数，如： <pre>" device_manager.__del__()"</pre>

5. 版本说明

序号	修订版本号	API 版本	所做改动	发布日期
1	V1.0.0	1.0.1808.x 及以上版本	初始发布	2018-08-10
2	V1.0.1	1.0.1810.x 及以上版本	添加水星二代相机新增功能说明	2018-10-31
3	V1.0.2	1.0.1904.x 及以上版本	修改部分标题，更正了部分不准确的描述	2019-04-12
4	V1.0.3	1.0.1905.x 及以上版本	补充了部分描述	2019-05-07
5	V1.0.4	1.0.2103.x 及以上版本	添加了掉线回调和采集回调的注册和注销接口，同步新的功能码和对应的数据类型	2021-03-04
6	V1.0.5	1.0.2103.x 及以上版本	修改部分描述的问题，去掉不准确的描述	2021-03-08
7	V1.0.6	1.0.2105.x 及以上版本	添加设备重连接口以及调节亮度、镜像等图像处理库接口	2021-05-27
8	V1.0.7	2.0.2303.x 及以上版本	1. 修改 2.2.2 节，更新新增枚举接口 2. 修改 2.2.5 节，更新图像格式转换接口，图像质量提升接口 3. 修改 2.2.6 节，更新属性读写查询为新接口 4. 修改 2.2.7 节，更新导入导出相机配置为新接口 5. 新增 3.1.1 节，基础数据类型读写说明 6. 修改 3.2.1 节，更新增加 cxp 类型 7. 修改 3.2.5 节，更新增加像素格式 MONO10P，MONO12P，R8，B8，G8，RGB8，BGR8 8. 修改 3.2.100 节，新增有效位 9. 新增 3.2.104 节，TL 类型说明 10. 新增 3.2.105，图像处理辅助结构说明 11. 新增 3.2.106，IP 配置模式结构说明 12. 修改 3.3.1，增加 get_interface_number、get_interface_info、get_interface、gige_force_ip、gige_ip_configuration、create_image_format_convert、create_image_process 接口 13. 修改 3.3.2 节，增加 get_stream_channel_num、get_stream、get_local_device_feature_control、get_remote_device_feature_control、	2023-03-12

序号	修订版本号	API 版本	所做改动	发布日期
			register_device_feature_callback 、 register_device_feature_callback_by_string 、 unregister_device_feature_callback 、 unregister_device_feature_callback_by_string 、 read_remote_device_port 、 write_remote_device_port 、 read_remote_device_port_stacked 、 write_remote_device_port_stacked 、 create_image_process_config 接口 14.新增 3.3.3 节，Interface 相关说明 15.修改 3.3.4 节，增加 get_feature_control 、 get_payload_size、set_payload_size 接口 16.新增 3.3.5 节，查询读写各节点属性相关说明 17.新增 3.3.9，3.3.11 节，图像处理相关说明 18.新增 3.3.12 节，图像格式转换相关说明	
9	V1.0.8	2.0.2404.x 及以上版本	1. 扩展定义 3.2.5.GxPixelFormatEntry 像素格式 2. 新增 3.3.12.13、3.3.12.14、3.3.12.15、3.3.12.16 小节	2024-04-03
10	V1.0.9	2.0.2405.x 及以上版本	1. 新增 GxLogTypeList 数据类型定义 2. 新增 3.3.1.1、3.3.1.2 小节	2024-05-23
11	V1.0.10	2.0.2406.x 及以上版本	1. 新增 3.3.4.3 节 dq_buf 接口 2. 新增 3.3.4.4 节 q_buf 接口	2024-06-24
12	V1.0.11	2.0.2412.x 及以上版本	1. 新增 3.2.107.GxActionCommandResult 2. 新增 3.3.1.21.issue_action_command 3.3.1.22.issue_scheduled_action_command	2024-12-10
13	V1.0.12	2.0.2412.x 及以上版本	1. 校对修改 3.2 节缺失接口和结构体定义 2. 新增 3.2.108 节 GxRegisterStackEntry 结构体 3. 新增 3.2.109 节 ColorTransformFactor 结构体 4. 新增 3.3.4.10 节 register_buffer 接口 5. 新增 3.3.4.11 节 unregister_buffer 接口 6. 新增 3.3.7.21 节 get_user_param 接口 7. 优化部分描述信息	2025-01-08
14	V1.0.13	2.0.2412.x 及以上版本	1. 修改 2.2.4 新增帧信息描述 2. 新增 3.3.7.22 get_chunk_data_feature_control 接口	2025-05-08
15	V1.0.14	2.0.2412.x 及以上版本	1. 新增 3.3.13 章节介绍平场相关接口 2. 新增 3.2.110 FlatFieldCorrectionParameter	2025-06-11

序号	修订版本号	API 版本	所做改动	发布日期
16	V1.0.15	2.0.2412.x 及以上版本	1. 新增 2.2.7 节介绍自动重连功能 2. 新增 3.2.101 DxActualBits	2025-09-02
17	V1.0.16	2.0.2509.x 及以上版本	1. 新增 3.3.1.23 节，create_decompressor 接口 2. 新增 3.3.14 节，Decompressor 功能类	2025-09-26
18	V1.0.17	2.0.2509.x 及以上版本	1. 新增 2.2.9 节，介绍属性 UI 功能 2. 新增 2.2.10 节，介绍存图存视频功能 3. 修改 3.2 节，新增 GxWindowsID、 GxWindowsShowMode、GxCFAMethod、 GxImageFormatType、GxVideoFormatType、 GxSaveImageInfo、GxRecordParam 结构体 4. 修改 3.3.2 节，新增 create_window、 destroy_window、set_window_position、 set_window_mode、show_window、 set_window_title 接口 5. 新增 3.3.15 节，MediaProc 功能类 6. 新增 3.3.16 节，VideoSaver 功能类	2026-01-12